LAPORAN PRAKTIKUM
REKAYASA PERANGKAT LUNAK



Disusun Oleh :

SURYA SAPUTRA

F 521 25 30 116

C

PROGRAM STUDI S1 SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS TADULAKO

TAHUN

2026

DAFTAR ISI

BAB I
PENDAHULUAN

1.1 Latar Belakang

1.2 Tujuan Praktikum

1.2.1 Konsep Dasar RPL, Pemodelan Aliran Data (DFD), Unified Modeling Language (UML) Menggunakan *Platform Draw.io*

1. Memahami Konsep Dasar RPL & SDLC: Menguasai konsep dasar Rekayasa Perangkat Lunak, peran *stakeholders*, serta tahapan siklus hidup pengembangan sistem (*System Development Life Cycle*) menggunakan paradigma *Waterfall* maupun *Agile Scrum*.
2. Menganalisis Kebutuhan Perangkat Lunak: Mampu mengidentifikasi, mengklasifikasikan, dan merumuskan Kebutuhan Fungsional (*Functional Requirements*) serta Kebutuhan Non-Fungsional (*Non-Functional Requirements*) dari suatu sistem informasi.
3. Merancang Pemodelan Aliran Data Terstruktur (DFD): Mampu menyusun dan memetakan alur data sistem secara mendetail melalui pembuatan Diagram Konteks (Level 0), DFD Level 1, hingga DFD Level 2 dengan menggunakan standar notasi DFD.
4. Memahami Dasar Pemodelan Berorientasi Objek (UML): Mengenal fungsi dan simbolsimbol dasar diagram UML yang meliputi *Use Case Diagram*, *Sequence Diagram*, *Activity Diagram*, dan *Class Diagram*.
5. Menguasai Alat Bantu Perancangan (Draw.io): Mampu mengoperasikan *Platform Draw.io* secara efektif untuk mengimplementasikan pemodelan diagram sistem terstruktur maupun berorientasi objek.

1.2.2 Perancangan Arsitektur Dan Implementasi Perangkat Lunak

1. Memahami dan merancang arsitektur perangkat lunak berbasis pola Model-*View*Controller (MVC).

2. Merancang struktur basis data relasional menggunakan Entity Relationship Diagram (ERD) dengan memperhatikan penetapan Primary Key, Foreign Key, dan aturan integritas data.
3. Mengimplementasikan basis data relasional dan skema antartabel pada sistem manajemen basis data *Mysql* /MariaDB menggunakan instruksi DDL (Data Definition Language.
4. Mengorganisasikan struktur proyek perangkat lunak secara modular berbasis arsitektur MVC dan menghubungkannya ke layanan basis data.
5. Menerapkan dan menguji operasi Create, Read, Update, dan Delete (CRUD) serta penanganan kesalahan (exception handling) pada antarmuka aplikasi.

1.2.3 *Setup Environment, Code Editor,* Dan *Version Control System* (VCS)

1. Mahasiswa mampu menginstalasi dan mengkonfigurasi *Code Editor (Visual Studio Code).*
2. Mahasiswa mampu membuat dan mengelola lingkungan virtual *(*Virtual *Environment)* menggunakan bahasa pemrograman *Python*.
3. Mahasiswa memahami dasar penggunaan Git dan dapat menghubungkan repositori lokal ke platform GitHub

1.2.4 Perancangan Model Python Dan Integrasi Basis Data *Mysql*

1. Mahasiswa mampu menghubungkan lingkungan kerja Python dengan layanan basis data *Mysql* yang berjalan pada Laragon.
2. Mahasiswa mampu mengimplementasikan komponen Model pada arsitektur MVC menggunakan bahasa pemrograman Python.
3. Mahasiswa mampu melakukan operasi CRUD *(Create, Read, Update, Delete)* dari Python ke dalam tabel basis data yang telah dibuat pada Modul sebelumnya.

1.2.5 Pengembangan Antarmuka Pengguna (*View*) Berbasis *Desktop*

1. Mahasiswa memahami peran dan tanggung jawab komponen *View* dalam arsitektur perangkat lunak berbasis MVC.

2. Mahasiswa mampu merancang dan membangun antarmuka pengguna grafis (*Graphical User Interface*/GUI) aplikasi desktop menggunakan Python.
3. Mahasiswa mampu mengimplementasikan tata letak (*layouting*), formulir masukan data (*input form*), dan tabel visual secara statis sebelum diintegrasikan dengan *Controller*.

1.3 Alat dan Bahan

1.3.1 Konsep Dasar RPL, Pemodelan Aliran Data (DFD), *Unified Modeling Language* (UML) Menggunakan *Platform Draw.io*

1. Laptop / PC
2. *Draw.io*
3. Modul Praktikum

1.3.2 Perancangan Arsitektur Dan Implementasi Perangkat Lunak

1. PC/Laptop
2. *Draw.io*
3. *Visual Studio Code*
4. *Laragon*

1.3.3 *Setup Environment, Code Editor,* Dan *Version Control System* (VCS)

1. PC/Laptop
2. *Visual Studi Code*
3. Python 3.x
4. Git
5. *Web Browser* Dan Akun Git

1.3.4 Perancangan Model Python Dan Integrasi Basis Data *Mysql*

1. PC/Laptop
2. *Visual Studio Code*
3. Python 3.x
4. Aplikasi *Laragon*

1.3.5 Pengembangan Antarmuka Pengguna (*View*) Berbasis *Desktop*

1. PC/Laptop

2. *Code Editor (Visual Studio Code)*
3. Python 3.x
4. *GitHub*

BAB II
LANDASAN TEORI

2.1 Teori Dasar

2.1.1 Konsep Dasar RPL, Pemodelan Aliran Data (DFD), *Unified Modeling Language* (UML) Menggunakan *Platform Draw.io*

1. Konsep Rekayasa Perangkat Lunak (RPL)

   Rekayasa Perangkat Lunak (RPL) mengkaji pendekatan sistematis, disiplin, dan terukur dalam pengembangan, pengoperasian, serta pemeliharaan perangkat lunak guna menghasilkan produk berkualitas tinggi secara efisien. Disiplin ini berkembang sebagai jawaban atas fenomena krisis perangkat lunak (*software crisis*), di mana proyek sering kali melebihi anggaran, melampaui batas waktu, sulit dipelihara, dan tidak memenuhi kebutuhan pengguna.

   Pemangku Kepentingan (*Stakeholders*) mencakup berbagai pihak yang memiliki kepentingan langsung maupun tidak langsung terhadap keberhasilan proyek, yaitu:

   a. Pengguna Akhir (*End-Users*): Pihak yang berinteraksi langsung dengan sistem untuk menyelesaikan tugas sehari-hari.
   b. Klien (*Client*): Pihak yang mendanai proyek dan menetapkan kebutuhan bisnis tingkat tinggi.
   c. Pengembang (*Developers*): Tim teknis yang bertugas menganalisis, merancang, dan menulis kode program.
   d. Penjamin Kualitas (*Quality Assurance*/QA): Pihak yang menguji perangkat lunak untuk memastikan pemenuhan standar kualitas.
   e. Operasional (*Operations*): Tim yang mengelola infrastruktur, *deployment*, dan pemeliharaan sistem pasca-rilis.

2. Metodologi Siklus Hidup Pengembangan Sistem (SDLC)

   System Development Life Cycle (SDLC) adalah kerangka kerja terstruktur yang mendefinisikan fase-fase utama dalam pengembangan perangkat lunak, mulai dari analisis awal,

perancangan, pengkodean, pengujian, hingga pemeliharaan sistem. Terdapat dua paradigma populer dalam SDLC:

a. Model *Waterfall*: Pendekatan sekuensial linier di mana setiap tahap (analisis, desain, implementasi, pengujian, pemeliharaan) harus diselesaikan secara penuh sebelum beralih ke tahap berikutnya. Sangat cocok untuk proyek dengan kebutuhan yang stabil dan terdokumentasi dengan baik sejak awal.
b. Model *Agile* (*Scrum*): Pendekatan iteratif dan inkremental yang berfokus pada kolaborasi tim, adaptasi cepat terhadap perubahan, dan pengiriman produk fungsional secara bertahap melalui siklus pendek yang disebut Sprint (umumnya berlangsung 1-4 minggu).

3. Analisis Kebutuhan Sistem

   Analisis Kebutuhan merupakan tahapan krusial untuk mengidentifikasi masalah, memahami batasan domain, serta memetakan apa yang harus dilakukan oleh perangkat lunak. Analisis ini menghasilkan spesifikasi kebutuhan yang terbagi menjadi:

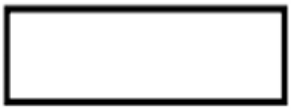
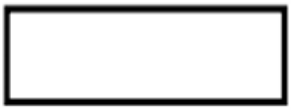
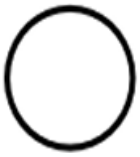
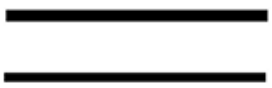
   a. Kebutuhan Fungsional (*Functional Requirements*): Deskripsi layanan, fitur, atau fungsi spesifik yang harus disediakan oleh sistem sebagai respons terhadap input tertentu (misalnya, sistem harus dapat menyimpan data transaksi penjualan).
   b. Kebutuhan Non-Fungsional (*Non-Functional Requirements*): Batasan operasional, karakteristik kualitas, atau standar performa sistem (seperti aspek keamanan, kecepatan akses, keandalan, dan kemudahan pemeliharaan).

4. *Data Flow Diagram* (DFD)

   Data Flow Diagram (DFD) merupakan alat pemodelan grafis yang menggambarkan aliran informasi, proses transformasi data, serta tempat penyimpanan data secara logis dalam suatu sistem tanpa mempertimbangkan aspek fisik penyimpanan atau perangkat keras. DFD terdiri dari beberapa tingkatan:

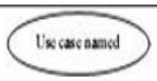
a. Diagram Konteks (*Context Diagram* / Level 0): Diagram tingkat tertinggi yang menggambarkan batas sistem secara keseluruhan, menunjukkan hubungan *input-output* utama antara sistem tunggal (proses nomor 0) dengan entitas-entitas eksternal di sekelilingnya.
b. DFD Level 1: Dekomposisi dari diagram konteks yang memecah proses utama menjadi subsub proses fungsional yang lebih detail, serta menampilkan interaksi data dengan data store (simpanan data).
c. DFD Level 2: Dekomposisi lebih lanjut dari proses yang ada pada DFD Level 1 untuk menjelaskan aktivitas pengolahan data yang sangat spesifik.

Tabel berikut menyajikan pembandingan notasi simbol DFD yang umum digunakan, yaitu versi DeMarco-Yourdon dan versi Gane-Sarson:

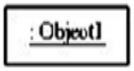
Simbol DeMarco dan Yourdan	**Ketrangan**	**Simbol Gane dan Sarson**
	Source (Kesatuan Luar)	
	Proses	
	Data Flow (Arus Data)	
	*Data Store* (Simpanan Data)	

Simbol DeMarco & Yourdan	Keterangan Simbol	Simbol Gane & Sarson
Lingkaran (*Circle*)	Proses (*Process*): Menunjukkan transformasi data *input* menjadi *output.*	Persegi Panjang dengan Sudut Tumpul (*Rounded Rectangle*)
Persegi Panjang (*Rectangle*)	Entitas Luar (*External Entity / Terminator/Source*): Sumber *input* atau tujuan akhir *output* data.	Persegi Panjang (*Rectangle*)
Garis Berpanah (*Arrow Line*)	Arus Data (*Data Flow*): Menunjukkan arah aliran data dalam *system*.	Garis Berpanah (*Arrow Line*)
Garis Sejajar Ganda (*Double Parallel Lines*)	Simpanan Data (*Data Store*): Tempat data disimpan secara logis.	Persegi Panjang Terbuka Ujung Kanan (*Open-ended-Rectangle*)

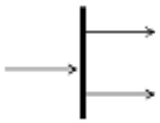
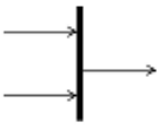
5. *Unified Modeling Language* (UML)
   a. *Diagram Use Case* menggambarkan apa saja aktifitas yang dilakukan oleh suatu sistem dari sudut pandang pengamatan luar. Yang menjadi persoalan itu apa yang dilakukan bukan bagaimana melakukannya. Sebuah *use case* merepresentasikan sebuah interaksi antara aktor dengan sistem. Seorang/sebuah aktor adalah sebuah entitas manusia atau mesin yang berinteraksi dengan sistem untuk melakukan pekerjaan pekerjaan tertentu. Berikut adalah simbol-simbol yang terdapat dalam *Use Case Diagram*:

Simbol	Keterangan
	*Use case* digambarkan sebagai lingkaran elips dengan nama *use case* dituliskan didalam elips tersebut.
	*Actor* adalah pengguna sistem. *Actor* tidak terbatas hanya manusia saja, jika sebuah sistem berkomunikasi dengan aplikasi lain dan membutuhkan *input* atau memberikan *output*, maka aplikasi tersebut juga bisa dianggap sebagai *actor*.
	Asosiasi digunakan untuk menghubungkan *actor* dengan *use case*. Asosiasi digambarkan dengan sebuah garis yang menghubungkan antara *Actor*dengan *Use Case*.

b. *Sequence diagram* menggambarkan interaksi antar objek di dalam dan di sekitar sistem (termasuk pengguna, *display*/*form*) berupa *message* yang digambarkan terhadap waktu. *Sequence diagram* biasa digunakan untuk menggambarkan skenario atau rangkaian langkah- langkah yang dilakukan sebagai respons dari sebuah *event* untuk menghasilkan *output* tertentu. Diawali dari apa yang men-*trigger* aktivitas tersebut, proses dan perubahan apa saja yang terjadi secara internal dan *output* apa yang dihasilkan. Berikut adalah simbol-simbol yang terdapat dalam *Sequence Diagram*:

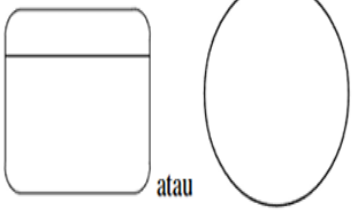
Simbol	Keterangan
	*Object* merupakan *instance* dari sebuah *class* dan dituliskan tersusun secara *horizontal*. Digambarkan sebagai sebuah *class* (kotak) dengan nama *object* di dalamnya yang diawali dengan sebuah titik koma.
	*Actor* juga dapat berkomunikasi dengan *object* , maka *actor* juga dapat diurutkan sebagai kolom. Simbol *Actor* sama dengan simbol pada *Actor* Use Case Diagram.
	Lifeline mengindikasikan keberadaan sebuah *object* dalam basis waktu. Notasi untuk *Lifeline* adalah garis putus-putus *vertikal* yang ditarik dari sebuah *object* .
	*Activation* dinotasikan sebagai sebuah kotak segi empat yang digambar pada sebuah *lifeline*. mengindikasikan sebuah obyek yang akan melakukan sebuah aksi.
	*Message*, digambarkan dengan anak panah horizontal antara *Activation* *Message* mengindikasikan komunikasi antara *object -object* .

c. *Activity Diagram* (AD) diperlukan untuk menggambarkan proses bisnis dan urutan aktivitas dalam sebuah proses. Merupakan salah satu jenis diagram dalam *Unified Modeling Language* (UML) yang digunakan untuk memodelkan alur kerja atau urutan aktivitas salam suatu proses. Sangat bermanfaat dalam menggambarkan proses dalam suatu prosedur. Berikut adalah simbol-simbol yang terdapat dalam *Activity Diagram*:

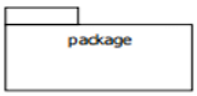
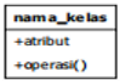
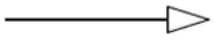
Simbol	Keterangan
	Start Point
	End Point
	Activities
	Fork (Percabangan)
	Join (Penggabungan)
	Decision
Swimlane	Sebuah cara untuk mengelompokkan activity berdasarkan Actor (mengelompokkan activity dalam sebuah urutan yang sama)

d. *Context Diagram* adalah tingkatan tertinggi dalam diagram aliran data yang hanya menampilkan satu proses utama untuk menggambarkan keseluruhan sistem. Proses ini diberi nomor nol. Diagram ini menunjukkan entitas eksternal beserta aliran data

utama yang masuk ke dan keluar dari sistem. Berikut adalah simbol yang terdapat dalam *Context Diagram* :

Simbol	Arti	Contoh
	Terminator	
	Aliran Data/ Data flow	Informasi mahasiswa baru 
	Proses/Process	

e. *Class Diagram* merupakan salah satu jenis diagram struktural utama dalam *Unified Modeling Language* (UML) yang menggambarkan struktur statis dari suatu sistem perangkat lunak. Diagram ini memperlihatkan kumpulan *class*, *package*, antarmuka (*interface*), dan objek beserta hubungan fungsional maupun struktural antara satu sama lain, seperti relasi *containment*, pewarisan (*inheritance* / generalisasi), asosiasi, agregasi, hingga kebergantungan (*dependency*). Secara konseptual, sebuah *class* adalah cetak biru (*blueprint*) atau spesifikasi yang mendefinisikan entitas tertentu. Ketika sebuah *class* di-instansiasi pada saat program berjalan, ia akan menghasilkan sebuah objek nyata (*instance*). *Class* menjadi inti dari paradigma Analisis dan Perancangan Berorientasi Objek (*Object-Oriented Analysis and Design* / OOAD). Di dalam pemodelan *Class Diagram*, setiap *class* umumnya digambarkan dalam bentuk kotak yang terbagi ke dalam tiga kompartemen utama

Simbol	Deskripsi
Package 	*Package* merupakan sebuah bungkusan dari satu atau lebih kelas
Kelas 	Kelas pada struktur sistem
Antarmuka / *interface* 	Sama dengan konsep *interface* dalam pemrograman berorientasi objek
Asosiasi / *association* 	Relasi antar kelas dengan makna umum, asosiasi biasanya juga disertai dengan *multiplicity*
Asosiasi berarah / *directed association* 	Relasi antar kelas dengan makna kelas yang satu digunakan oleh kelas yang lain, asosiasi biasanya juga disertai dengan *multiplicity*
Generalisasi 	Relasi antar kelas dengan makna generalisasi-spesialisasi (umum khusus)
Kebergantungan / *dependency* 	Relasi antar kelas dengan makna kebergantungan antar kelas
Agregasi / *aggregation* 	Relasi antar kelas dengan makna semua-bagian (*whole-part*)

6. Draw.io

   Draw.io adalah sebuah aplikasi atau platform yang memungkinkan siapa saja untuk merancang berbagai macam diagram secara visual. Mulai dari diagram alur yang sederhana hingga model sistem yang kompleks. Sebagai sebuah alat visualisasi yang kuat, draw.io dilengkapi dengan berbagai bentuk, konektor, dan tema yang dapat disesuaikan. Dari perencanaan proyek hingga pengembangan perangkat lunak dan kemampuan untuk membuat berbagai jenis diagram, mulai dari diagram UML hingga peta konsep. Lalu diagram yang dibuat dapat diekspor ke berbagai format, seperti PNG, JPEG, SVG, PDF. Draw.io memungkinkan penyimpanan diagram langsung ke berbagai platform cloud seperti Google Drive, OneDrive, dan Dropbox serta pengguna dapat menyimpan diagram secara lokal di perangkat.

2.1.2 Perancangan Arsitektur Dan Implementasi Perangkat Lunak

1. Arsitektur Perangkat Lunak & Pola MVC

   Arsitektur perangkat lunak mendefinisikan struktur komponen utama sistem dan cara komponen tersebut saling berinteraksi. Perancangan arsitektur membantu pengembang memisahkan antarmuka, logika aplikasi, dan pengelolaan data agar sistem lebih terstruktur dan mudah dipelihara.

   Pola arsitektur Model-*View*-Controller (MVC) membagi aplikasi menjadi tiga bagian utama (Separation of Concerns):

   a. Model: Mengelola data dan logika bisnis aplikasi. Layer ini bertanggung jawab atas enkapsulasi struktur data, manipulasi data, eksekusi transaksi, serta interaksi langsung dengan basis data.

   b. *View*: Menampilkan antarmuka pengguna (User Interface). Layer ini bertugas menyajikan informasi dan data hasil olahan Model kepada pengguna.

   c. Controller: Menerima masukan dari pengguna melalui *View* dan mengatur aliran proses antara *View* dan Model. Controller menentukan bagaimana data dari Model diproses dan menentukan tampilan *View* mana yang akan disajikan kembali.

2. Pemodelan Data Relasional & ERD

   Basis data menyimpan data secara terstruktur dalam bentuk tabel relasional. Entity Relationship Diagram (ERD) digunakan untuk menggambarkan entitas, atribut, kunci utama (primary key), kunci asing (foreign key), serta hubungan antartabel sebelum basis data diimplementasikan pada DBMS

   Elemen utama dalam perancangan ERD meliputi:

   a. Entitas: Objek nyata atau konsep dalam dunia nyata yang datanya perlu disimpan dalam sistem (misalnya: Buku, Anggota, Peminjaman).

b. Atribut: Properti atau karakteristik yang menjelaskan deskripsi detail dari suatu entitas.
c. Primary Key (PK): Atribut khusus yang bersifat unik untuk membedakan secara pasti setiap baris (record) dalam suatu tabel.
d. Foreign Key (FK): Atribut pada suatu tabel yang merujuk pada Primary Key tabel lain untuk membangun relasi antar entitas.
e. Kardinalitas Relasi: Aturan yang menentukan batasan jumlah entitas yang dapat saling terhubung. Contoh kardinalitas umum adalah One-to-Many (1:N), di mana satu data entitas utama dapat terhubung dengan banyak data pada entitas transaksi.

3. Bahasa Kueri Terstruktur (SQL) dan Operasi CRUD

Structured Query Language (SQL) adalah bahasa standar yang digunakan untuk mengelola dan memanipulasi basis data relasional. SQL dibagi menjadi beberapa sub-bahasa utama:

a. Data Definition Language (DDL): Instruksi untuk membangun dan mendefinisikan skema atau struktur basis data, seperti perintah CREATE DATABASE, CREATE TABLE, ALTER, dan *DROP*.
b. *Data Manipulation Language* (DML) : Perintah untuk mengolah isi data di dalam tabel. Operasi CRUD terdiri atas:
    1) *Create* (*INSERT*): Digunakan untuk memasukkan atau menambah baris data baru ke dalam tabel.
    2) *Read* (*SELECT*): Digunakan untuk membaca, menampilkan, atau mengambil data dari satu atau beberapa tabel (dapat digabungkan dengan perintah *JOIN*).
    3) *Update* (*UPDATE*): Digunakan untuk mengedit atau memperbarui nilai data yang sudah tersimpan pada tabel.
    4) *Delete* (*DELETE*): Digunakan untuk menghapus baris data dari tabel berdasarkan kriteria tertentu.

4. Integrasi Basis Data, *Parameterization, dan Exception Handling*

Dalam pengimplementasian arsitektur MVC berbasis pemrograman berorientasi objek, Layer Model mengeksekusi kueri SQL dengan memperhatikan keamanan dan keandalan data:

a. *Prepared Statement & Parameterization Query:*Penggunaan *placeholder* (seperti ? atau :param) untuk menyisipkan nilai variabel ke dalam perintah SQL secara terpisah dari sintaksis kuerinya. Mekanisme ini mencegah bahaya *SQL Injection*, yaitu celah keamanan akibat penyisipan skrip SQL berbahaya melalui input pengguna. Selain meningkatkan keamanan, metode ini juga membuat *query* menjadi lebih terstruktur karena perintah SQL dan data yang dimasukkan oleh pengguna dipisahkan.
b. Manajemen Transaksi & Integritas Data: Menerapkan perintah commit() untuk menyimpan perubahan secara permanen ke basis data setelah seluruh proses berhasil, atau rollback() untuk membatalkan seluruh perubahan jika terjadi kegagalan di tengah transaksi (prinsip *Atomicity*). Dengan adanya manajemen transaksi, data yang tersimpan tetap konsisten dan tidak terjadi kondisi di mana hanya sebagian proses yang berhasil. Contohnya, ketika proses transaksi membutuhkan beberapa perubahan data, jika salah satu proses mengalami kegagalan maka seluruh perubahan dapat dibatalkan menggunakan rollback().
c. *Exception Handling*: Menangkap kesalahan tingkat basis data (*database error/exception*) agar jika terjadi kegagalan (misalnya duplikasi *Primary Key* atau pembatalan *Foreign Key constraint*), aplikasi tidak mengalami *crash* mendadak, melainkan memberikan respons atau pesan galat yang informatif kepada pengguna. *Exception Handling* juga membantu aplikasi menentukan tindakan yang harus dilakukan ketika terjadi kesalahan.

2.1.3 *Setup Environment, Code Editor,* Dan *Version Control System* (VCS)

1. Code Editor dan *Visual Studio Code* (VS Code)

   Dalam rekayasa perangkat lunak, pemilihan alat bantu pengembangan perangkat lunak (*software development tools)* sangat memengaruhi produktivitas dan efisiensi penulisan kode. *Code editor* adalah program teks editor yang dirancang khusus untuk menulis dan mengedit kode sumber *(source code*). Berbeda dengan *Integrated Development Environment* (IDE) penuh yang terkadang berat dan lambat, code editor modern menawarkan kompromi antara kecepatan dan fitur yang kaya.

   Salah satu *code editor* yang paling populer dan banyak digunakan saat ini adalah Visual Studio Code (VS Code). Dikembangkan oleh Microsoft, VS Code menawarkan performa yang ringan namun dilengkapi dengan *ekosistem* ekstensi yang sangat luas, integrasi *Git* secara langsung, serta fitur *debugging* yang andal. Keunggulan-keunggulan ini menjadikan VS Code sebagai pilihan utama bagi para pengembang untuk meningkatkan produktivitas dalam mengelola berbagai jenis bahasa pemrograman dan proyek perangkat lunak.

   Visual Studio *Code* (VS Code) adalah *source-code* editor gratis dan open-source yang dikembangkan oleh Microsoft. Dibangun di atas kerangka kerja Electron, VS Code sangat ringan namun menyediakan fitur-fitur kelas industri (*enterprise-grade*). Fitur utama yang menjadikannya standar industri saat ini meliputi:

   a. *IntelliSense*: Sistem *auto-complete* cerdas yang tidak hanya menyelesaikan pengetikan kata, tetapi juga memberikan saran berdasarkan tipe variabel, definisi fungsi, dan modul pustaka yang diimpor.

   b. *Integrated* Terminal: Memungkinkan pengembang menjalankan perintah *Command Line Interface* (CLI) langsung di dalam editor tanpa harus berpindah jendela aplikasi.

c. Ekstensi (*Extensions*): Kemampuan untuk menambahkan dukungan bahasa pemrograman baru (seperti Python, C++, Java), linter untuk analisis statis kode, hingga alat pemformatan teks otomatis (seperti PEP8 untuk Python).
d. *Source Control Integration*: Integrasi langsung dengan Git untuk memantau perubahan file secara *real-time*.

2. *Python Virtual Environment* (venv)

   Dalam pengembangan aplikasi skala menengah hingga besar, manajemen dependensi (*dependency management*) adalah hal yang krusial. Secara default, pustaka (*library*) pihak ketiga yang diinstal melalui pip (*Python Package Installer*) akan disimpan di direktori global sistem operasi. Pendekatan ini akan memicu fenomena yang dikenal sebagai *Dependency Hell*, yaitu konflik versi pustaka ketika beberapa proyek di satu komputer membutuhkan versi pustaka yang berbeda (misalnya, Proyek A membutuhkan *SQLAlchemy* v1.4, sedangkan Proyek B membutuhkan *SQLAlchemy* v2.0).

   Untuk mengatasi masalah ini, Python menyediakan modul bawaan bernama venv yang berfungsi menciptakan *Virtual Environment* (Lingkungan Virtual). *Virtual environment* adalah struktur direktori yang terisolasi sepenuhnya dari instalasi *Python* global. Ketika virtual environment diaktifkan, *Python* akan membuat salinan interpreter (*biner Python*) dan folder *site-packages* lokal. Dengan demikian, pengembang dapat menginstal, memperbarui, atau menghapus pustaka secara aman di dalam folder proyek tersebut tanpa memberikan dampak atau kerusakan pada proyek lain maupun pada sistem operasi utama. Praktik ini merupakan standar wajib dalam rekayasa perangkat lunak untuk menjaga konsistensi lingkungan kerja, terutama saat kode didistribusikan antar anggota tim pengembang.

   Selain *mencegah* konflik dependensi, penggunaan venv juga mempermudah proses kolaborasi dan replikasi lingkungan kerja

melalui berkas *requirements.txt.* Dalam berkas ini, seluruh daftar pustaka beserta versi spesifik yang digunakan dalam proyek dapat dicatat, sehingga pengembang lain cukup menjalankan satu perintah untuk menginstal seluruh dependensi yang identik. Hal ini memastikan bahwa aplikasi dapat berjalan dengan stabil dan konsisten di berbagai perangkat pengembangan maupun saat diunggah ke lingkungan produksi (*production environment*).

3. *Version Control System* (VCS)

   Version Control System (Sistem Pengontrol Versi) adalah sebuah sistem perangkat lunak yang merekam setiap perubahan yang terjadi pada berkas atau kumpulan berkas dari waktu ke waktu, sehingga pengembang dapat memanggil kembali versi spesifik di masa lalu jika diperlukan. VCS dirancang untuk menjawab masalah kolaborasi klasik: bagaimana puluhan atau ratusan pemrogram dapat mengedit ribuan file kode yang sama secara bersamaan tanpa saling menimpa (*overwrite*) atau merusak pekerjaan satu sama lain.

   Secara umum, VCS dibagi menjadi dua paradigma utama:

   1. *Centralized* VCS (CVCS): Menggunakan satu repositori terpusat di sebuah *server.* Semua pengembang mengambil (*checkout*) dan menyimpan (*commit*) file langsung ke server tersebut (contoh: *Subversion*/SVN). Kelemahannya, jika *server* mati, tidak ada yang bisa berkolaborasi.
   2. *Distributed* VCS (DVCS): Setiap pengembang memiliki salinan penuh dari seluruh repositori beserta riwayat perubahannya di komputer lokal masing-masing. Paradigma ini jauh lebih aman, cepat, dan memungkinkan pengembang tetap bekerja dalam keadaan *offline*.

   Implementasi paling populer dari *Distributed Version Control System* (DVCS) saat ini adalah Git, yang sering dipadukan dengan platform layanan berbasis *cloud* seperti GitHub, GitLab, atau Bitbucket. Melalui kombinasi ini, tim pengembang tidak hanya

dapat mengelola riwayat perubahan kode secara lokal, tetapi juga memanfaatkan fitur-fitur penting seperti *pull request* / *merge request*, peninjauan kode (*code reView*), pencatatan masalah (*issue tracking*), dan otomatisasi *CI/CD* untuk mempermudah proses integrasi serta pengujian perangkat lunak secara berkelanjutan.

4. Git dan GitHub

   Git adalah implementasi dari *Distributed Version Control System* (DVCS) yang diciptakan oleh Linus Torvalds pada tahun 2005. Git bekerja melalui antarmuka baris perintah (CLI) dan beroperasi secara lokal di komputer pengembang. Tiga konsep ruang kerja utama dalam Git adalah:

   a. *Working Directory:* Direktori tempat file proyek sedang dimodifikasi secara aktual.
   b. *Staging Area* (Index): Ruang transit tempat file yang telah dimodifikasi didaftarkan sebelum disimpan secara permanen.
   c. *Local Repository*: Basis data internal Git tempat riwayat perubahan disimpan secara permanen melalui perintah commit.

   Sementara itu, GitHub adalah platform layanan *cloud-based hosting* untuk repositori Git. Sering kali pemula menyamakan Git dengan GitHub, padahal keduanya berbeda; Git adalah alat pengontrol versinya, sedangkan GitHub adalah tempat menyimpan repositori Git tersebut secara daring (*online*). GitHub menyediakan antarmuka grafis (GUI) berbasis web dan menambahkan fitur kolaborasi berstandar industri seperti *Pull Requests* (mekanisme pengajuan perubahan kode antar pengembang), *Issue Tracking* (pelacakan bug dan tugas), serta integrasi berkelanjutan (CI/CD) melalui GitHub *Actions*, yang menjadikannya ekosistem pengembangan perangkat lunak terbesar di dunia. Singkatnya, Git berfungsi untuk mengelola dan merekam riwayat perubahan kode di tingkat lokal, sedangkan GitHub menjadi wadah untuk menyimpan, membagikan, dan mengolaborasikan kode tersebut secara daring bersama.

2.1.4 Perancangan Model Python Dan *Integrasi Basis Data Mysql*

Arsitektur perangkat lunak berfungsi untuk mendefinisikan struktur komponen utama sistem dan menentukan bagaimana komponen tersebut saling berinteraksi. Pola arsitektur *Model-View-Controller* (*MVC*) membagi aplikasi menjadi tiga bagian utama berlandaskan prinsip pemisahan tanggung jawab (*Separation of Concerns*). Dalam ekosistem ini, Model memiliki peran krusial dan tanggung jawab eksklusif untuk mengelola data serta logika bisnis aplikasi. Layer Model bertugas melakukan enkapsulasi struktur data, memanipulasi data, mengeksekusi transaksi, serta berinteraksi langsung dengan sistem manajemen basis data (DBMS). Model berdiri secara independen; ia tidak mempedulikan bagaimana antarmuka pengguna (*View*) ditampilkan atau bagaimana alur masukan dari pengguna diatur oleh *Controller*. Pemisahan ini membuat kode sumber (*source code*) pada bagian Model dapat digunakan ulang (*reusable*) dan lebih mudah dipelihara.

Structured Query Language (SQL), khususnya pada elemen Data Manipulation Language (DML), digunakan untuk mengolah isi data di dalam tabel melalui operasi CRUD, yakni Create (INSERT), Read (SELECT), Update (UPDATE), dan Delete (DELETE). Dalam pengimplementasian aplikasi Python berbasis *MVC*, operasi SQL ini dibungkus ke dalam paradigma Pemrograman Berorientasi Objek (OOP). Sebuah entitas tabel di basis data *Mysql* (misalnya tabel buku atau anggota) direpresentasikan sebagai sebuah Kelas (Class), operasi CRUD yang mengubah atau membaca data diimplementasikan sebagai metode (methods) di dalam kelas tersebut. Pendekatan ini membuat kode lebih modular dan mudah dibaca dibandingkan mengeksekusi instruksi SQL mentah secara acak di seluruh bagian aplikasi.

Aplikasi Python tidak dapat secara instingtif mengenali atau mengubah data di dalam server *Mysql* yang berjalan pada lingkungan Laragon. Diperlukan sebuah jembatan penghubung atau driver untuk membuka jalur komunikasi jaringan. Pustaka *Mysql -connector-python* bertindak sebagai antarmuka yang mengelola siklus hidup koneksi: mulai dari autentikasi username dan password, pengiriman perintah *SQL* (seperti query *SELECT*),

hingga penerjemahan hasil keluaran (resultset) dari *Mysql* agar kembali menjadi struktur data asli Python (seperti *List* atau *Dictionary*). Mekanisme kerjanya sangat identik dengan arsitektur PDO (PHP *Data Objects*) yang digunakan pada pengembangan berbasis PHP.

Saat Layer Model mengeksekusi kueri SQL, keamanan dan keandalan data harus dijaga secara ketat. Salah satu celah kerentanan sistem yang paling berbahaya adalah SQL Injection, yaitu penyisipan skrip SQL berbahaya melalui masukan input pengguna yang tidak divalidasi. Untuk memitigasi bahaya tersebut, pengembang wajib menggunakan Prepared Statement dan *Parameterization Query*. Mekanisme ini menggunakan placeholder—seperti tanda % pada Python (atau tanda ? dan :param pada bahasa pemrograman lain)—untuk menyisipkan nilai variabel ke dalam perintah *SQL* secara terpisah dari sintaksis utama kuerinya. Proses ini memastikan engine basis data memperlakukan input dari pengguna secara ketat murni sebagai data (value), dan menolak untuk mengeksekusinya sebagai perintah struktural, sehingga integritas basis data tetap terjamin.

Dalam menjaga integritas data relasional, eksekusi transaksi yang mengubah isi tabel (seperti *INSERT*, *UPDATE*, atau *DELETE*) harus diakhiri dengan instruksi yang tepat. Pengembang menerapkan perintah *commit*() untuk menyimpan seluruh perubahan secara permanen ke basis data setelah proses eksekusi dinilai berhasil. Jika terjadi kegagalan prosedural di tengah jalan, perintah *rollback*() digunakan untuk membatalkan seluruh perubahan agar data tidak tersimpan dalam keadaan setengah-setengah (Prinsip Atomicity). Selain itu, program yang andal wajib menerapkan Exception *Handling* (penangkapan pengecualian). Dengan menggunakan *blok try-except di Python*, pengembang menangkap kesalahan tingkat basis data (*database error/exception*) seperti duplikasi *Primary Key* saat insert data atau kegagalan koneksi ke server. Melalui penanganan ini, jika terjadi kegagalan atau interupsi, aplikasi tidak mengalami crash atau tertutup secara mendadak, melainkan tetap stabil dan memberikan respon atau pesan galat yang informatif kepada pengguna.

2.1.5 Pengembangan Antarmuka Pengguna (*View*) Berbasis *Desktop*

Dalam pola arsitektur *Model-View-Controller* (MVC), komponen *View* bertugas secara eksklusif untuk menampilkan antarmuka pengguna (*User Interface*). Layer ini berfungsi menyajikan informasi, formulir *input*, dan data hasil olahan dari Model kepada pengguna secara visual. Prinsip terpenting dalam membangun View adalah antarmuka tidak boleh mengandung logika bisnis, kueri SQL, atau operasi manipulasi basis data secara langsung. *View* dirancang untuk menjadi komponen yang "pasif" atau statis; ia hanya menyediakan tombol, kolom teks, dan tabel, namun menyerahkan respon terhadap aksi pengguna (seperti klik tombol simpan) kepada *Controller*. Pemisahan ini memungkinkan desainer antarmuka (UI/UX) dan pengembang backend (Model) untuk bekerja secara bersamaan tanpa saling mengganggu kode satu sama lain.

Selain berfungsi sebagai tampilan, *View* juga menjadi penghubung visual antara sistem dan pengguna. Dalam praktikum ini, penerapannya dilakukan menggunakan *GUI* sehingga pengguna dapat berinteraksi melalui jendela, tombol, kolom teks, dan tabel tanpa harus menjalankan perintah melalui terminal. Komponen seperti *Frame* digunakan untuk mengelompokkan elemen agar rapi, *Label* memberikan informasi atau instruksi, *Entry* digunakan untuk memasukkan data, *Button* menjalankan suatu aksi, sedangkan *Treeview* digunakan untuk menampilkan banyak data dalam bentuk tabel. Tata letak komponen tersebut dapat diatur menggunakan *Geometry Manager* seperti *.pack*() dan *.grid*(). Dengan demikian, rancangan *View* tidak hanya memperhatikan tampilan, tetapi juga mengatur agar informasi dan fungsi aplikasi dapat disajikan secara terstruktur dan mudah dipahami pengguna.

1. *Graphical User Interface* (GUI) di Python

   Berbeda dengan aplikasi CLI (*Command Line Interface*) yang dijalankan lewat teks di terminal, GUI memungkinkan pengguna berinteraksi melalui elemen visual (jendela, tombol, *ikon*). Python secara bawaan memiliki pustaka GUI bernama tkinter. Namun, untuk menghasilkan antarmuka

desktop yang lebih *modern*, elegan, dan mendukung tema gelap (*dark mode*) secara otomatis, praktikum ini menggunakan pustaka pihak ketiga bernama *customtkinter*.

2. Konsep *Layouting* dan *Widget* Komponen

   Membangun antarmuka *desktop* ibarat menyusun blok-blok bangunan (*widgets*) di dalam sebuah kanvas (*window*). Elemen-elemen dasar *GUI* meliputi:

   a. *Frame*: Kontainer atau wadah berbentuk persegi untuk mengelompokkan elemen lain agar tata letaknya rapi.
   b. *Label*: Teks statis untuk memberikan judul atau instruksi kepada pengguna.
   c. *Entry*: Kotak isian (*textbox*) satu baris tempat pengguna mengetikkan data input.
   d. *Button*: Tombol yang dapat diklik untuk memicu sebuah aksi atau *event*.
   e. *Treeview* (*DataGrid*): Komponen tabel yang digunakan untuk merender banyak baris data, sangat cocok untuk menampilkan daftar baris dari basis data.

Untuk mengatur letak komponen tersebut di layar, digunakan sistem manajemen tata letak (*Geometry Manager*) yang berfungsi menentukan posisi dan susunan setiap *widget* di dalam jendela aplikasi. Metode *.pack()* digunakan untuk menyusun *widget* secara sederhana *berdasarkan* arah tertentu, misalnya dari atas ke bawah atau dari kiri ke kanan, sehingga cocok untuk tampilan yang tidak terlalu kompleks. Sementara itu, *.grid()* menyusun *widget* berdasarkan baris dan kolom seperti sebuah tabel, sehingga lebih sesuai untuk membuat *form* yang membutuhkan posisi komponen lebih teratur, misalnya kolom label di sebelah kiri dan kolom input di sebelah kanan. Penggunaan *Geometry Manager* yang tepat membuat tampilan *GUI* menjadi lebih rapi, terstruktur, dan mudah digunakan. Materi Modul V menjelaskan bahwa *.pack()* dan *.grid()* merupakan metode umum untuk mengatur posisi *widget* pada antarmuka aplikasi desktop.

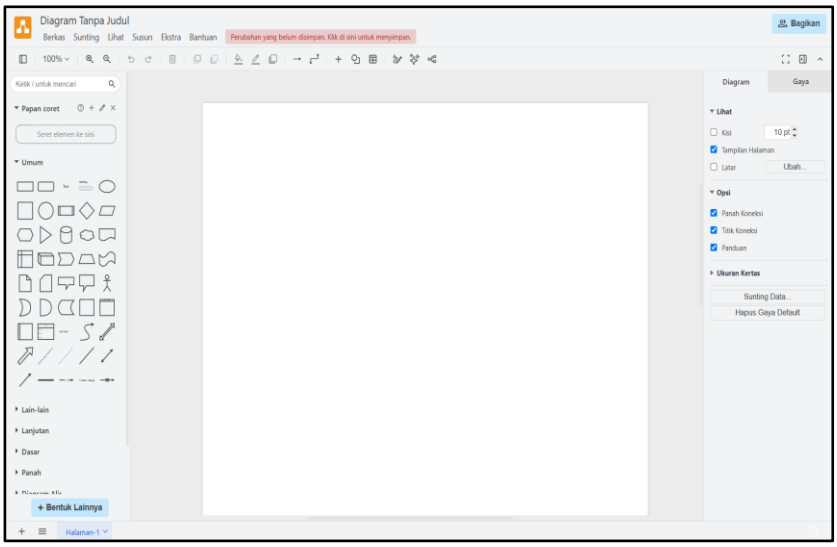
BAB III
PROSEDUR KERJA

3.1 Langkah Kerja

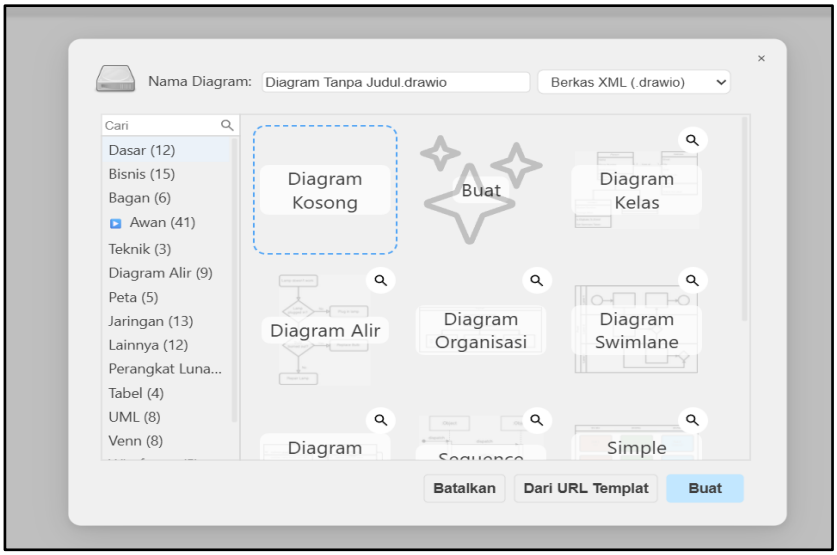
3.1.1 Konsep Dasar RPL, Pemodelan Aliran Data (DFD), *Unified Modeling Language* (UML) Menggunakan *Platform Draw.io*

1. Pembuatan Diagram Konteks di Draw.io

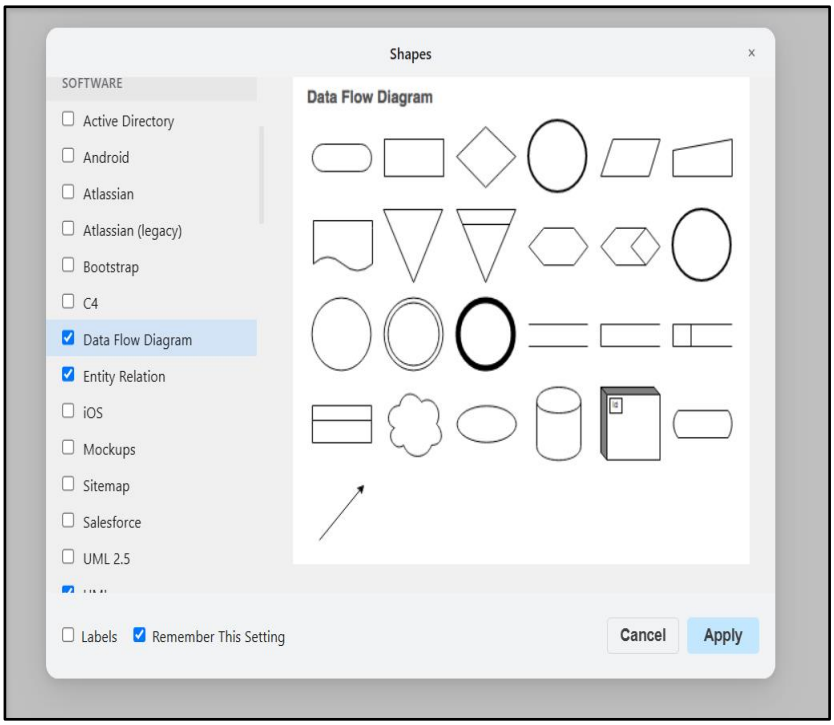
   a. Akses *Platform Draw.io* melalui web browser atau jalankan aplikasi Draw.io versi *Desktop* .



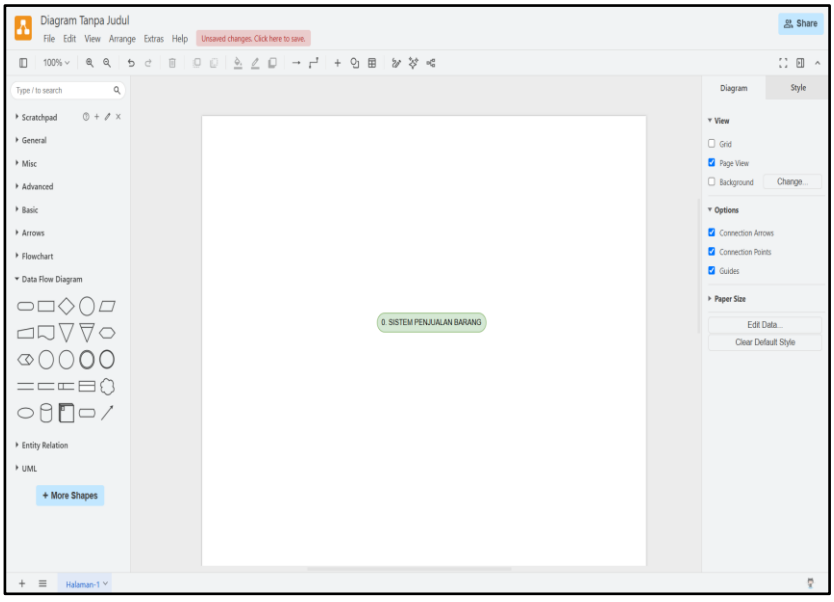
   b. Pada dialog awal, pilih '*Create New Diagram*' lalu klik opsi *'Blank Diagram'*. Klik '*Create*' untuk masuk ke bidang kerja utama.



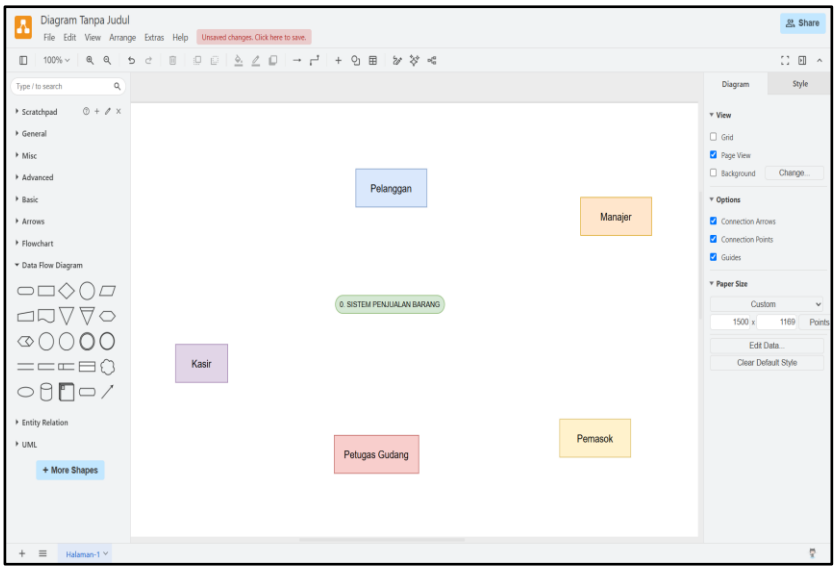
c. Lakukan konfigurasi pustaka bentuk dengan mengaktifkan kumpulan simbol yang sesuai di panel sebelah kiri (pilih kategori '*General'* atau cari *'Data Flow Diagram'*).



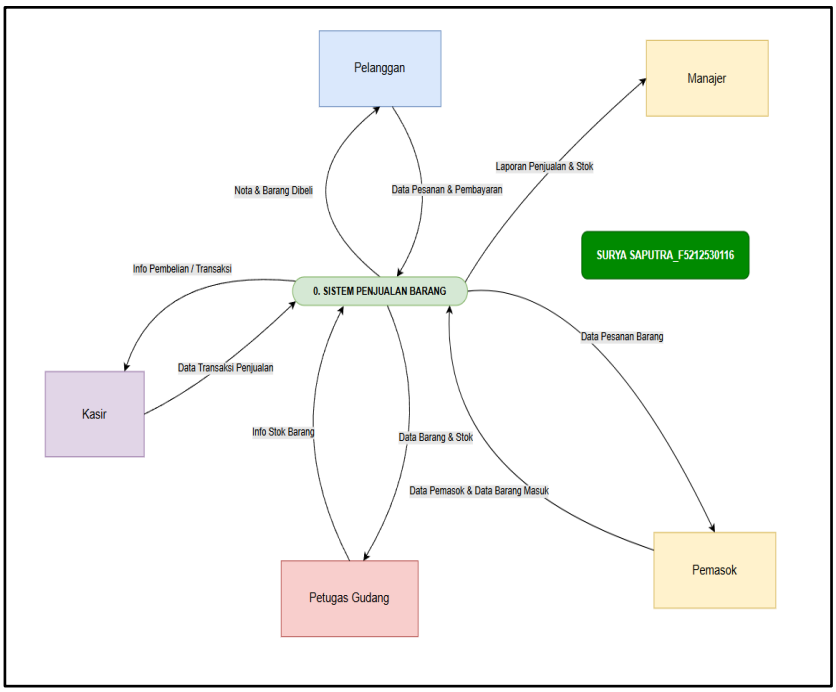
d. Tempatkan sebuah elemen proses berbentuk lingkaran atau persegi panjang bersudut tumpul di bagian tengah bidang kerja. Beri label '0. SISTEM PENJUALAN BARANG' sebagai representasi tunggal dari keseluruhan sistem.



e. Buat entitas luar di sekeliling proses pusat menggunakan bentuk persegi panjang standar. Definisikan 5 entitas eksternal utama: Pelanggan, Kasir, Petugas Gudang, Pemasok, dan Manajer.

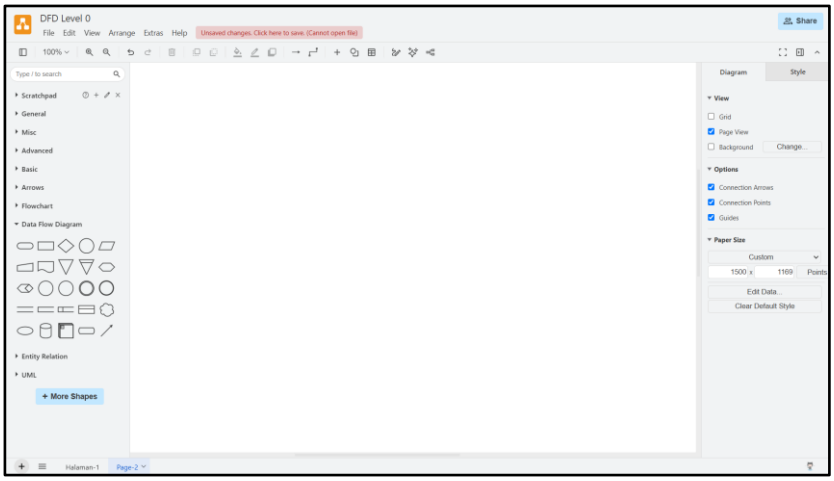


f. Hubungkan entitas eksternal ke proses pusat menggunakan garis alir berpanah (*Data Flow Line*). Berikan label teks yang jelas pada setiap garis aliran data untuk merepresentasikan informasi logis yang mengalir masuk atau keluar dari sistem seperti gambar dibawah ini



2. Pembuatan DFD Level 1 di Draw.io

a. Buat halaman baru atau diagram baru pada berkas Draw.io yang sama dengan mengklik ikon tanda tambah (+) di bagian tab bawah halaman.



b. Identifikasi dan gambarkan sub-proses utama dari sistem penjualan barang. Pada DFD Level 1 ini, sistem dipecah menjadi 5 sub-proses:

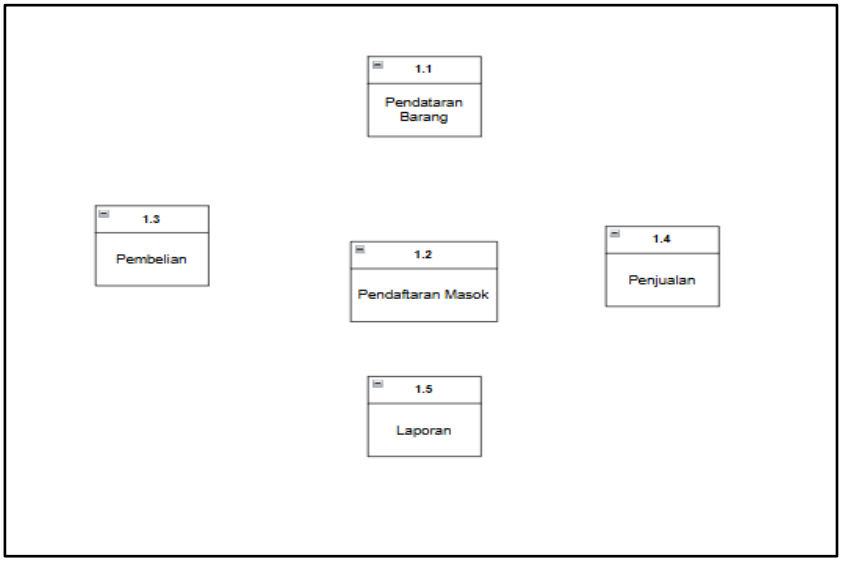
1) '1.1 Pendaftaran Barang'

2) '1.2 Pendaftaran Pemasok'

3) '1.3 Pembelian'

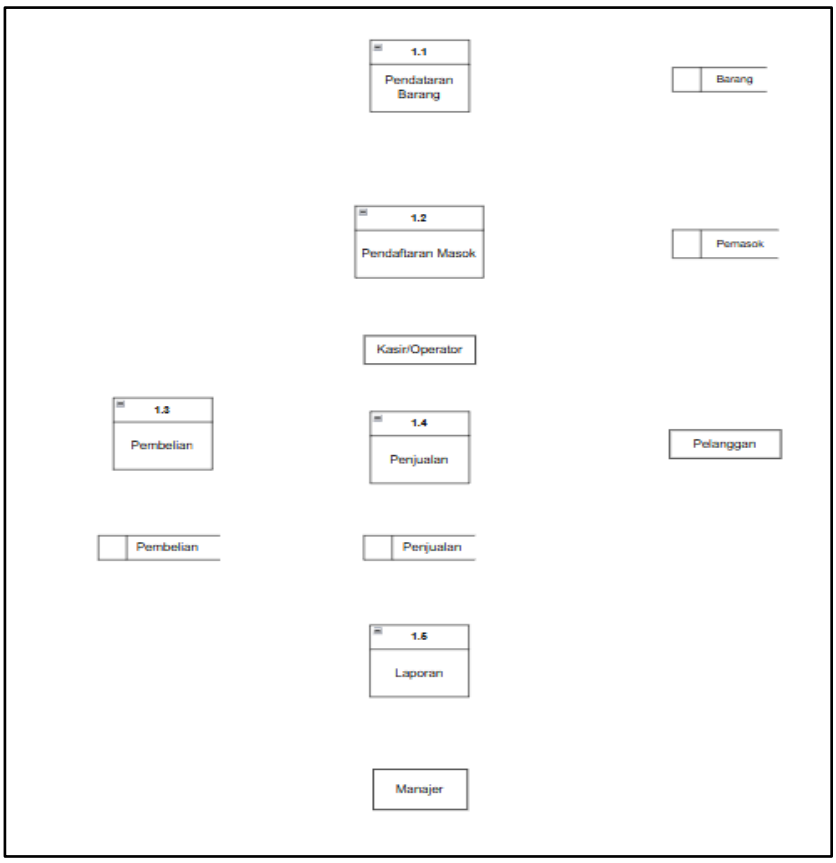
4) '1.4 Penjualan'

5) '1.5 Laporan'

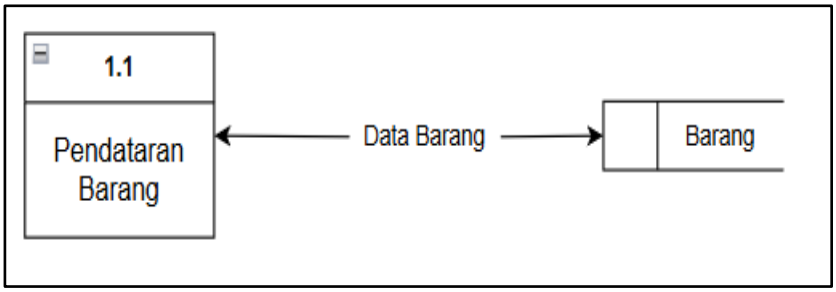


c. Gambarkan tempat penyimpanan data (Data *Store*) menggunakan bentuk garis sejajar ganda atau persegi panjang terbuka. Sediakan data *store* untuk:

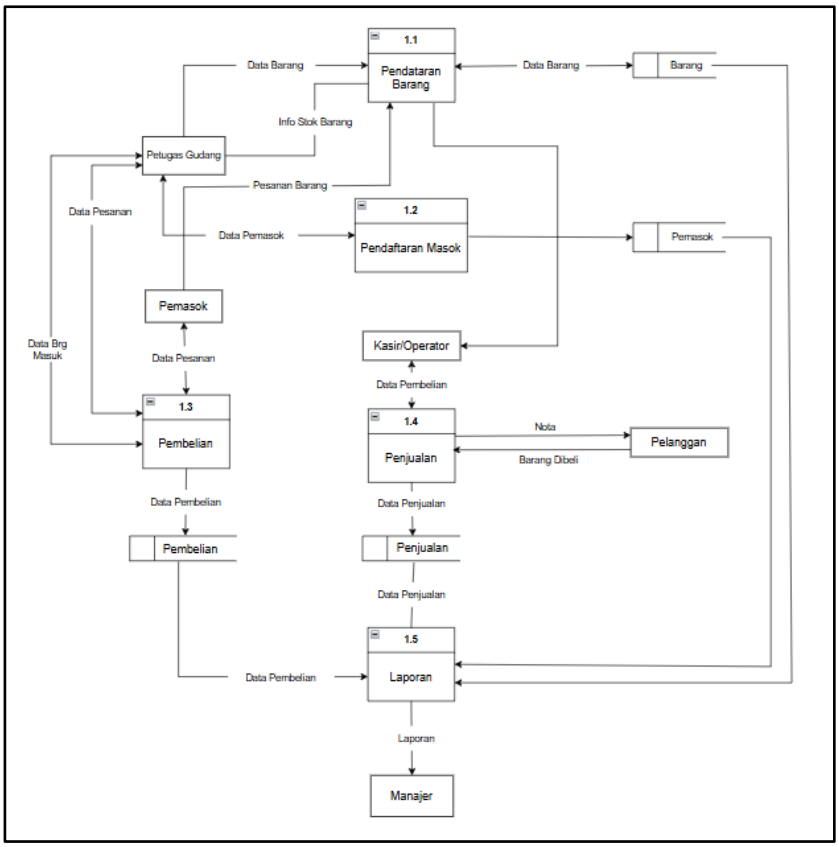
1) Simpanan 'Barang'
2) Simpanan 'Pemasok'
3) Simpanan 'Pembelian'
4) Simpanan 'Penjualan'



d. Hubungkan aliran data dari sub-proses fungsional ke simpanan data yang relevan (misalnya, aliran data barang dari sub-proses 1.1 disimpan ke dalam data *store* 'Barang').

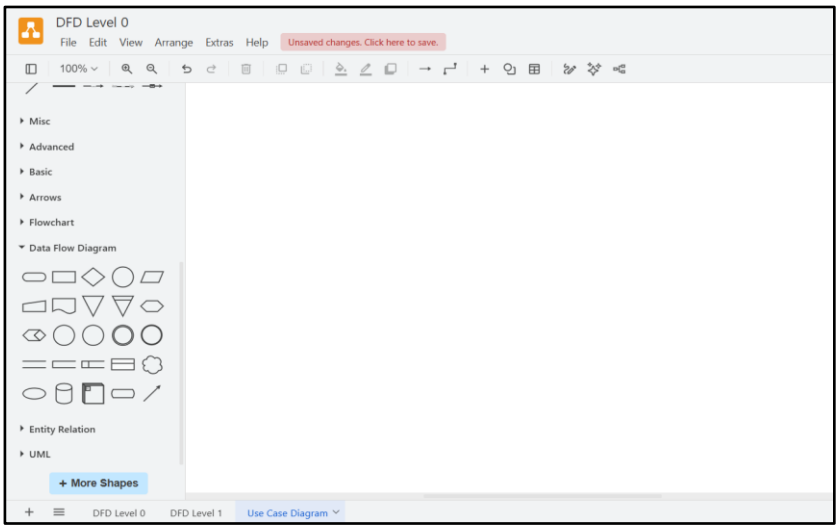


e. Pastikan semua entitas eksternal yang ada di Diagram Konteks tetap terhubung dengan sub proses yang sesuai di DFD Level 1 dibawah ini:

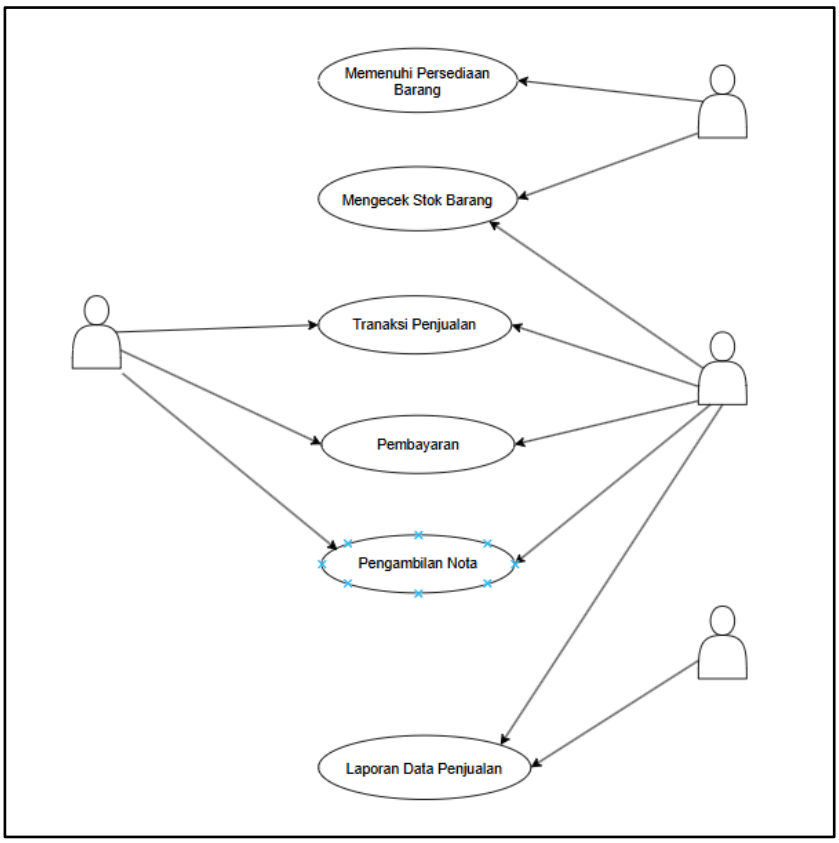


3. Pembuatan *Use Case Diagram* di Draw.io

a. Tambahkan halaman atau diagram baru dengan klik *new* di dan *blank diagram*.

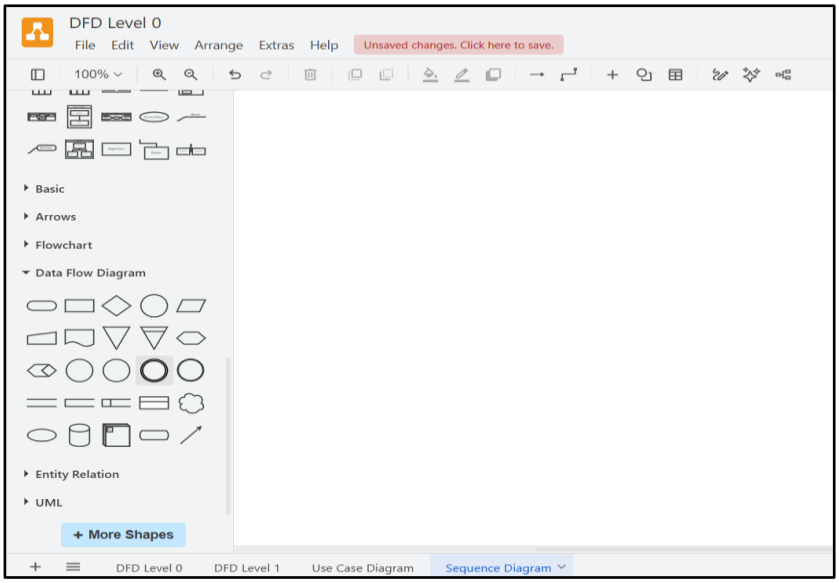


b. Buat diagram seperti gambar di bawah ini dengan aturan diagram *use case*.

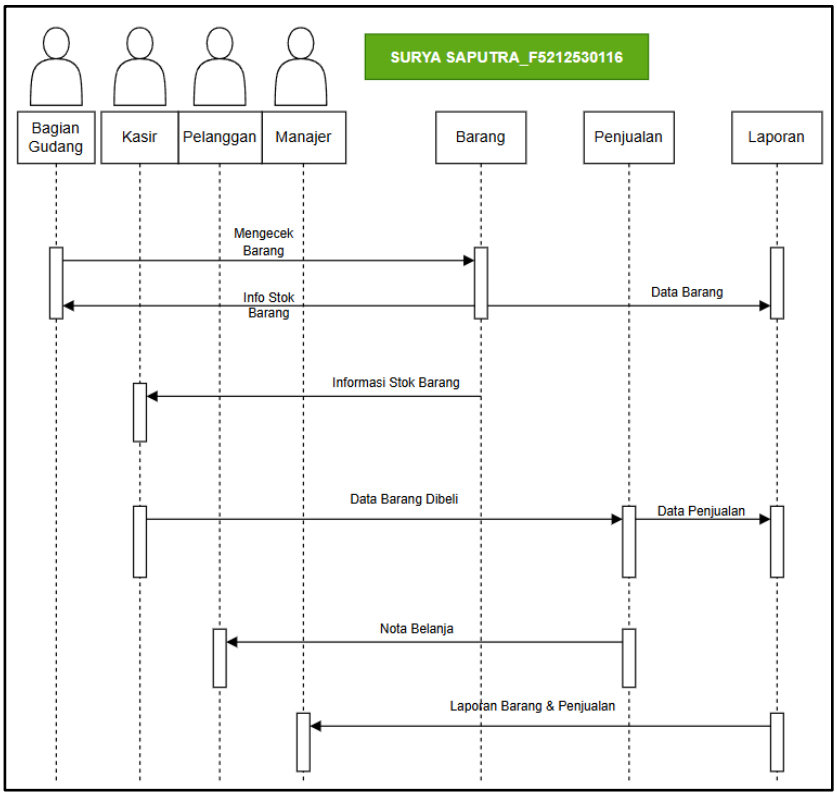


4. Pembuatan *Sequence Diagram* di Draw.io

a. Buat halaman baru dengan pilihan *blank diagram* serta beri nama, lalu tekan *create*.

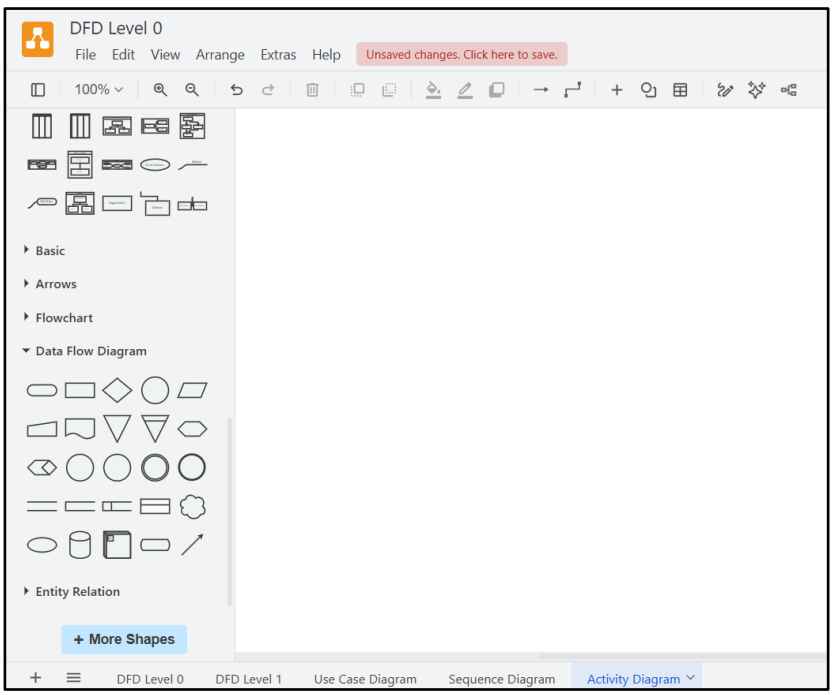


b. Buat diagram seperti di bawah ini.

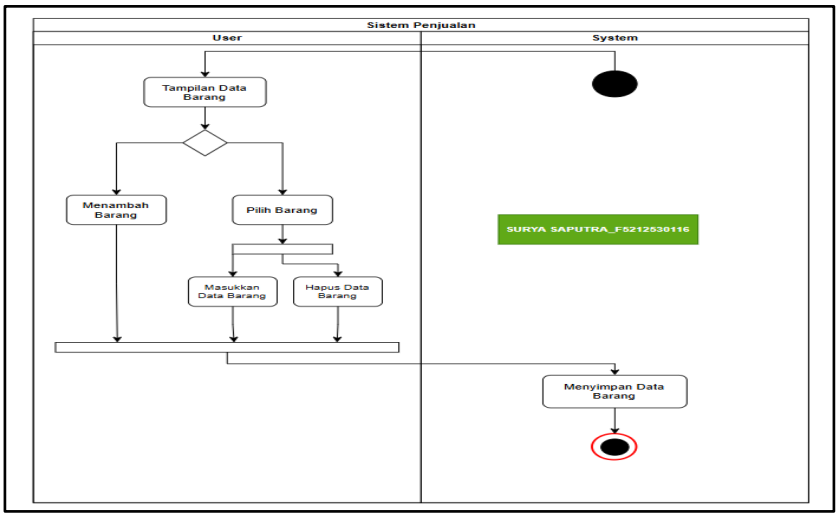


5. Pembuatan *Activity Diagram* di Draw.io

a. Buat halaman baru dengan pilih blank diagram serta beri namanya, lalu tekan *create*.

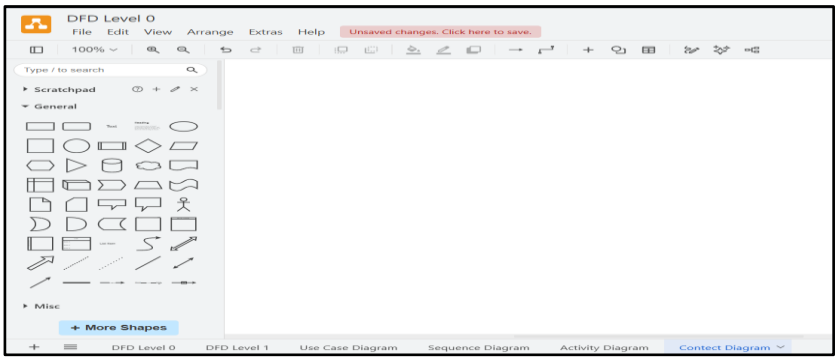


b. Buat diagram di bawah ini.

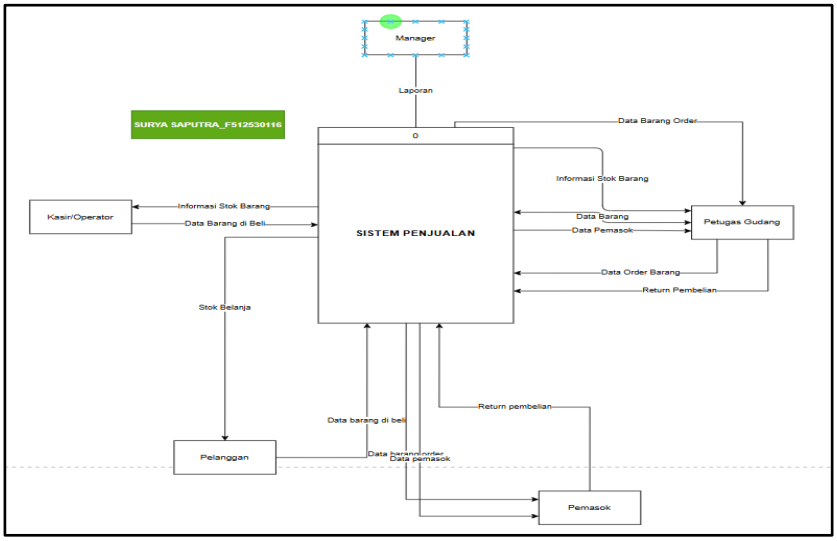


6. Pembuatan *Context Diagram* di Draw.io

a. Buat halaman baru dengan pilih *blank diagram* serta beri namanya, lalu tekan *create*

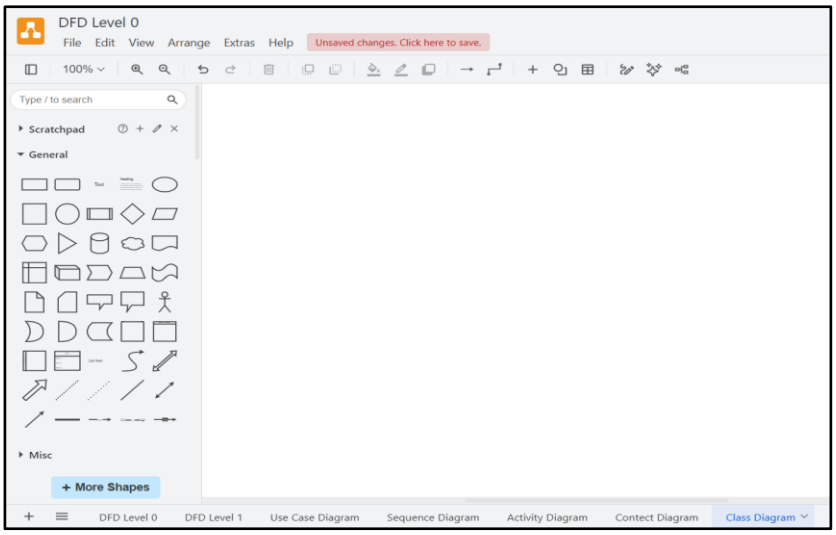


b. Buatlah diagram seperti dibawah ini

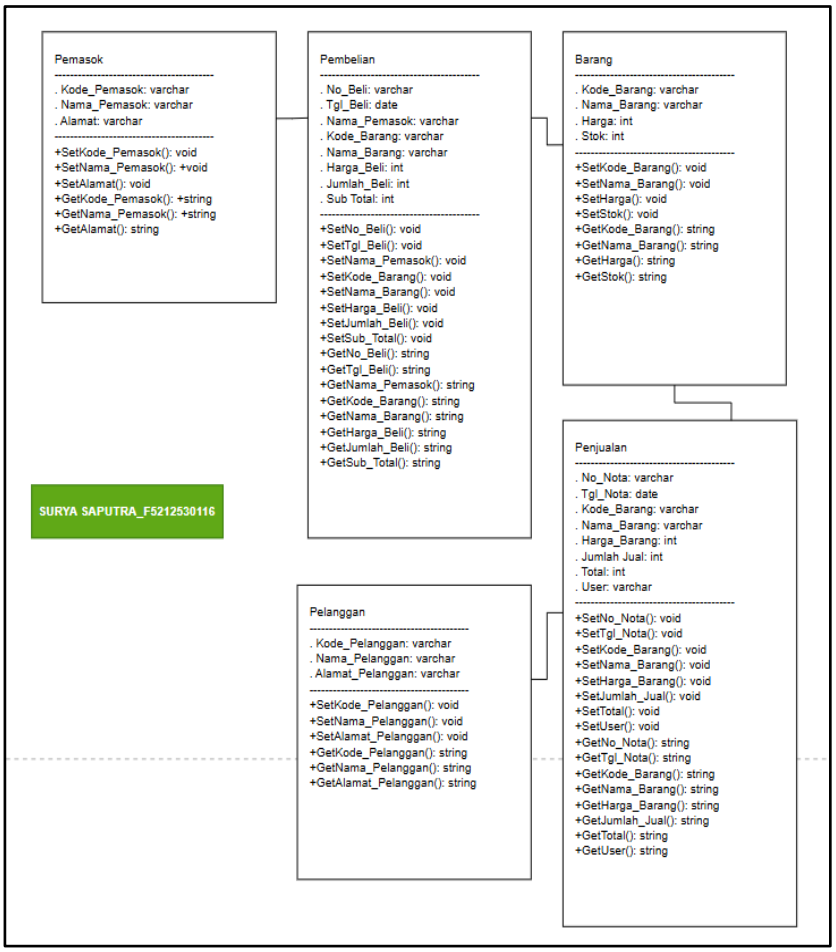


7. Pembuatan *Class Diagram* di Draw.io

a. Buat halaman baru dengan pilih *blank diagram* serta beri namanya, lalu tekan *create*.



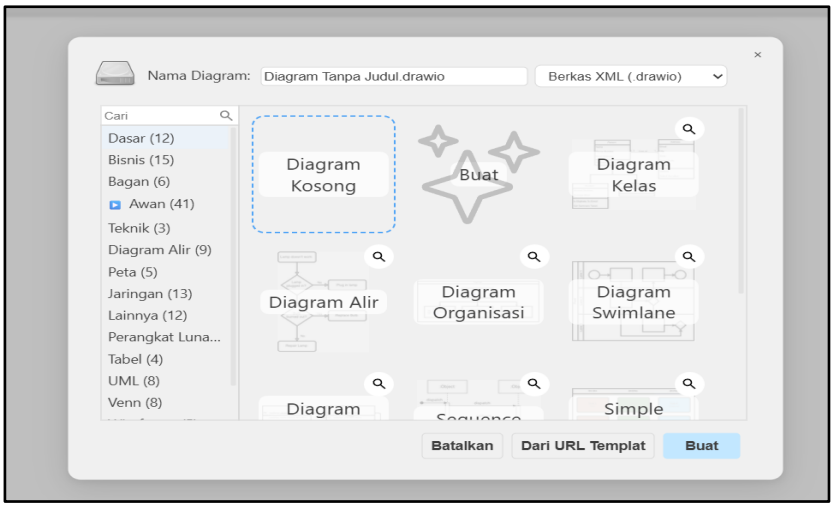
b. Buatlah diagram dibawah ini.



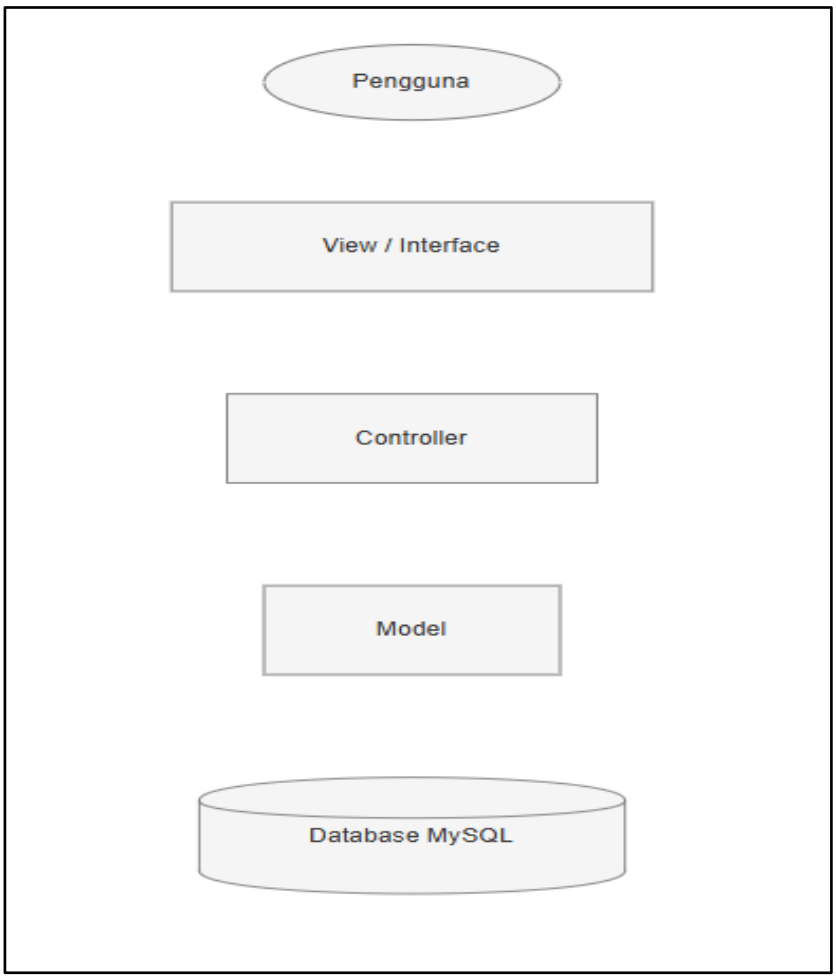
3.1.2 Perancangan Arsitektur Dan Implementasi Perangkat Lunak

1. Perancangan Diagram Arsitektur MVC di Draw.io

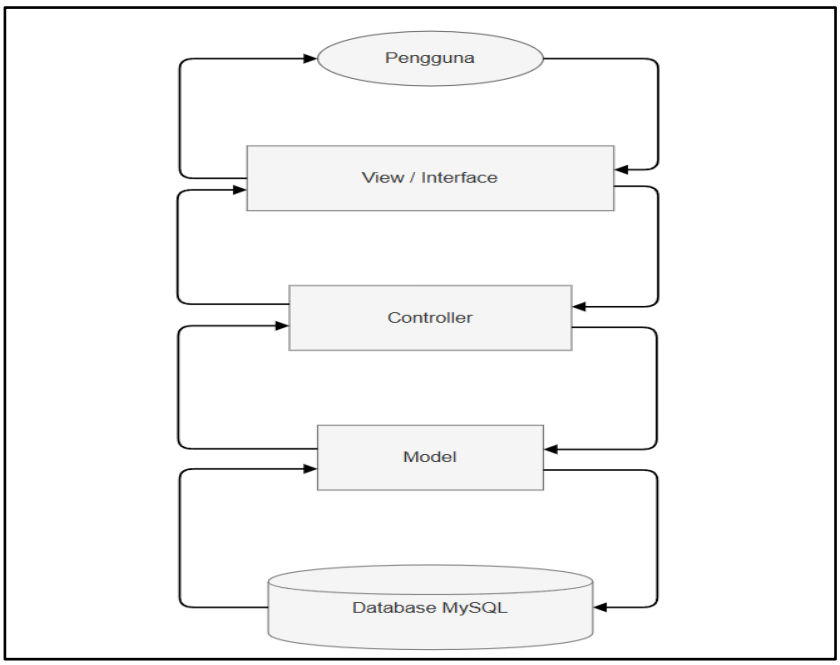
   a. Buka Draw.io dan pilih Blank Diagram.



   b. Buat empat komponen: *View*, Controller, Model, dan Database.

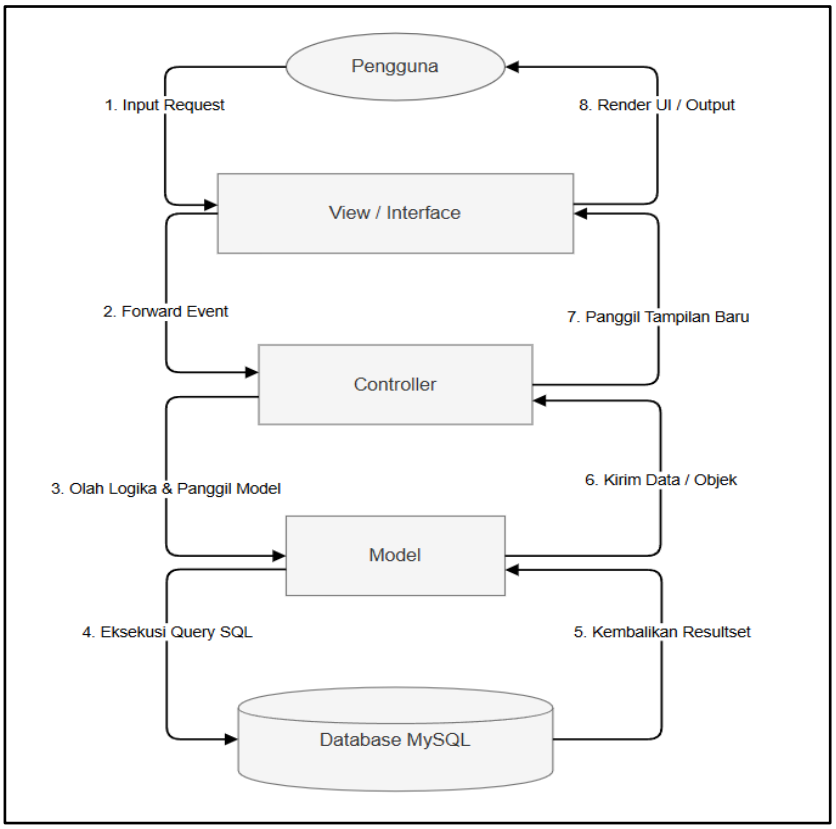


c. Hubungkan *View* ke Controller, Controller ke Model, serta Model ke Database.

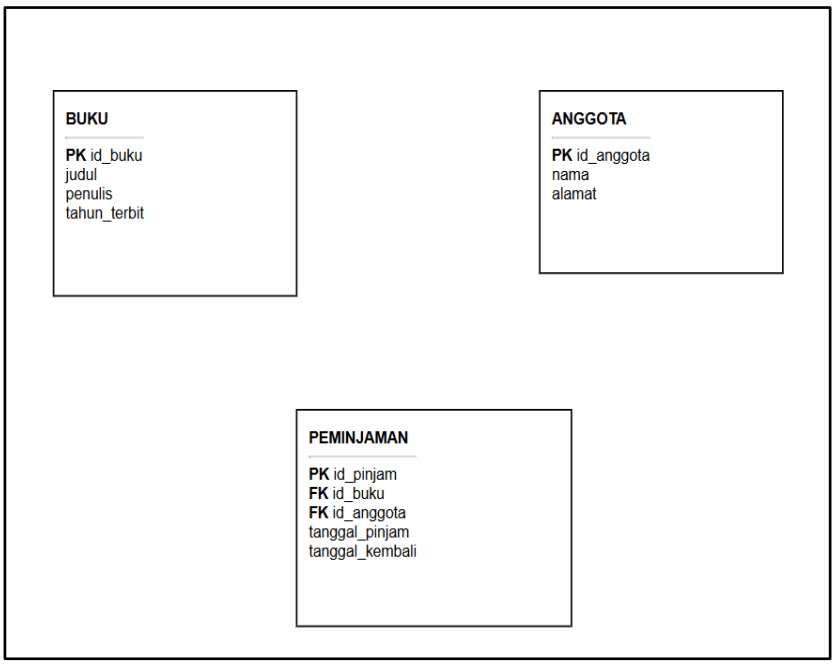


d. Tambahkan arah aliran data menggunakan connector.

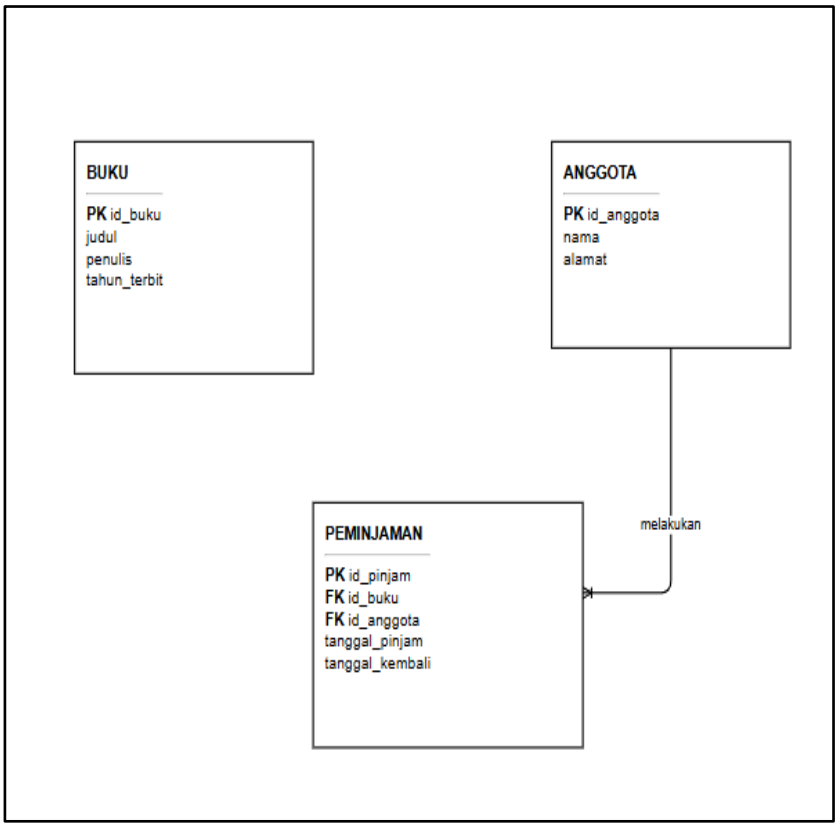
e. Simpan diagram dengan nama Arsitektur MVC.



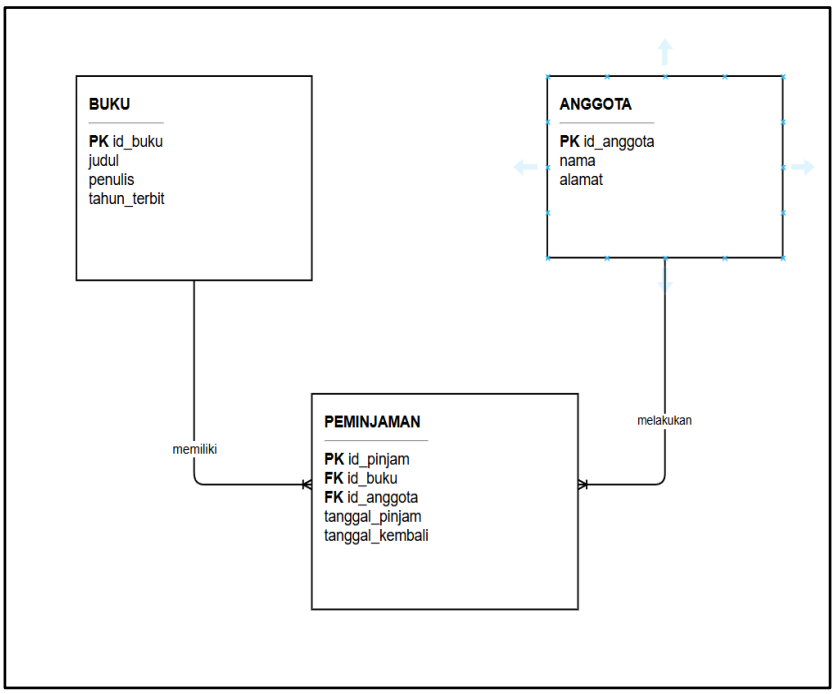
2. Perancangan *Entity Relationship Diagram* (ERD)
   a. Buat tiga entitas berikut pada Draw.io:



   b. Hubungkan Anggota dengan Peminjaman menggunakan relasi satu ke banyak.

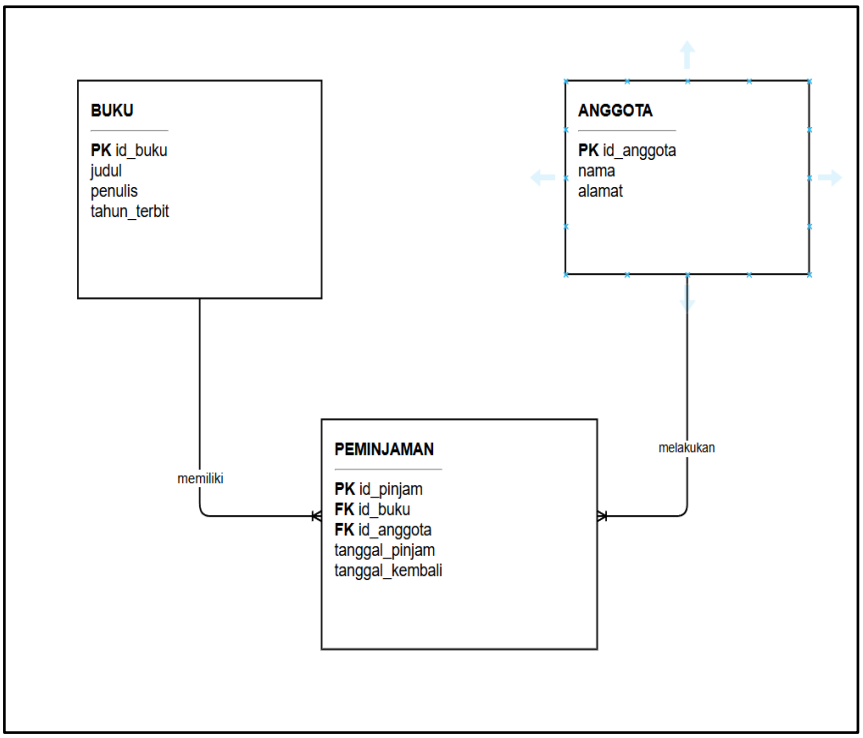


c. Hubungkan Buku dengan Peminjaman menggunakan relasi satu ke banyak.



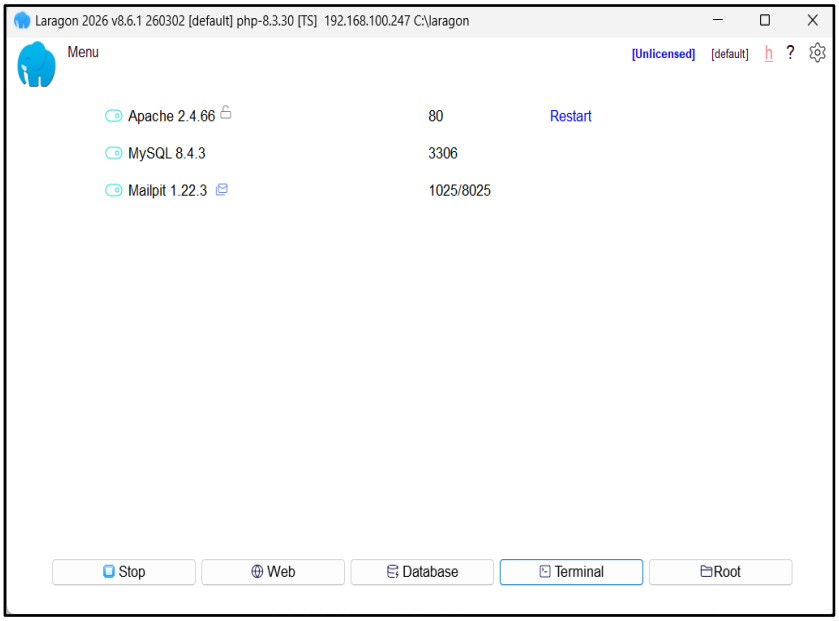
d. Pastikan setiap entitas memiliki *primary key* dan tabel Peminjaman memiliki *foreign key*.

e. Simpan diagram dengan nama ERD Perpustakaan

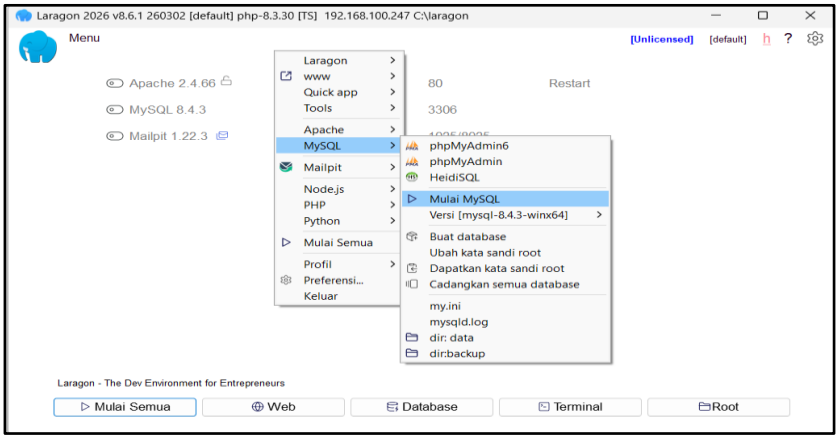


3. Mengimplementasikan *Database* (SQL DDL)

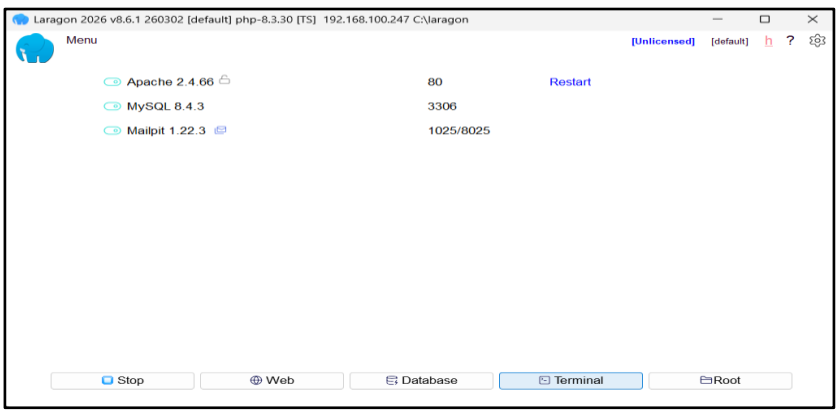
a. Buka Aplikasi Laragon



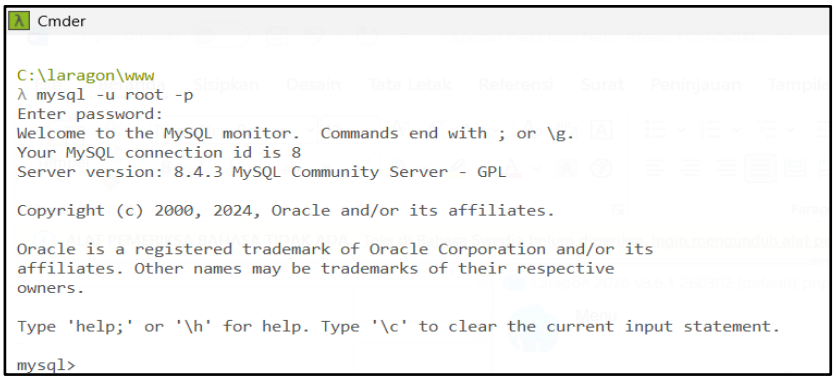
b. Klik ”Menu” lalu pilih ”*Mysql* ” dan Klik ”Mulai *Mysql* ”



c. Setelah itu Klik Menu ”*Terminal*”



d. Lalu ketik perintah berikut



e. Buat dan pilih *database* perpustakaan menggunakan perintah berikut:

CREATE DATABASE perpustakaan;

```
mysql> CREATE DATABASE perpustakaan;
Query OK, 1 row affected (0.01 sec)
```

USE perpustakaan;

```
mysql> USE perpustakaan;
Database changed
```

f. Buat tabel buku:

CREATE TABLE buku (
id_buku *INT PRIMARY KEY AUTO_INCREMENT*,
judul *VARCHAR*(100) *NOT NULL*,
penulis *VARCHAR*(100) *NOT NULL*,
tahun_terbit *YEAR*
);

```
mysql> CREATE TABLE buku (
    ->  id_buku INT PRIMARY KEY AUTO_INCREMENT,
    ->  judul VARCHAR(100) NOT NULL,
    ->  penulis VARCHAR(100) NOT NULL,
    ->  tahun_terbit YEAR
    -> );
Query OK, 0 rows affected (0.02 sec)
```

g. Buat tabel anggota:

CREATE TABLE anggota (

id_anggota *INT PRIMARY KEY AUTO_INCREMENT*,

nama *VARCHAR*(100) *NOT NULL*,

alamat *TEXT*

);

```
mysql> CREATE TABLE anggota (
    ->  id_anggota INT PRIMARY KEY AUTO_INCREMENT,
    ->  nama VARCHAR(100) NOT NULL,
    ->  alamat TEXT
    -> );
Query OK, 0 rows affected (0.01 sec)
```

h. Buat tabel peminjaman:

CREATE TABLE peminjaman (

id_pinjam *INT PRIMARY KEY AUTO_INCREMENT*,

id_buku *INT NOT NULL*,

id_anggota *INT NOT NULL*,

tanggal_pinjam *DATE NOT NULL*,

tanggal_kembali *DATE*,

FOREIGN KEY (id_buku) *REFERENCES* buku(id_buku),

FOREIGN KEY (id_anggota) *REFERENCES* anggota(id_anggota)

);

```
mysql> CREATE TABLE peminjaman (
    ->  id_pinjam INT PRIMARY KEY AUTO_INCREMENT,
    ->  id_buku INT NOT NULL,
    ->  id_anggota INT NOT NULL,
    ->  tanggal_pinjam DATE NOT NULL,
    ->  tanggal_kembali DATE,
    ->  FOREIGN KEY (id_buku) REFERENCES buku(id_buku),
    ->  FOREIGN KEY (id_anggota) REFERENCES     anggota(id_anggota)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

i. Melihat seluruh tabel yang telah dibuat:

```
mysql> SHOW TABLES;
+-------------------------+
| Tables_in_perpustakaan  |
+-------------------------+
| anggota                 |
| buku                    |
| peminjaman              |
+-------------------------+
3 rows in set (0.00 sec)
```

j. Masukkan data anggota ke tabel anggota:

```
mysql> INSERT INTO anggota (nama, alamat) VALUES
    -> ('Dimas Anggara', 'Jl. Diponegoro No. 15'),
    -> ('Citra Kirana', 'Jl. Sudirman No. 88');
Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

k. Masukkan data buku ke tabel buku:

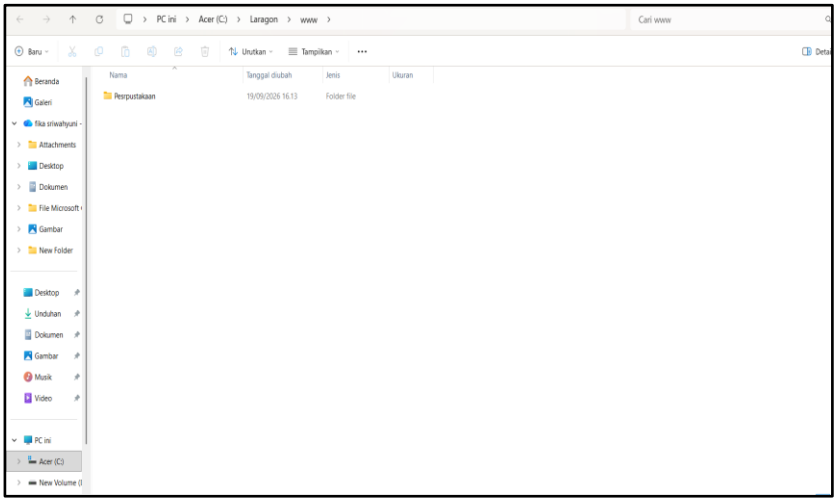
```
mysql> INSERT INTO buku (judul, penulis, tahun_terbit) VALUES
    -> ('Negeri 5 Menara', 'Ahmad Fuadi', 2009),
    -> ('Cantik Itu Luka', 'Eka Kurniawan', 2002);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

l. Masukkan data peminjaman ke tabel peminjaman:

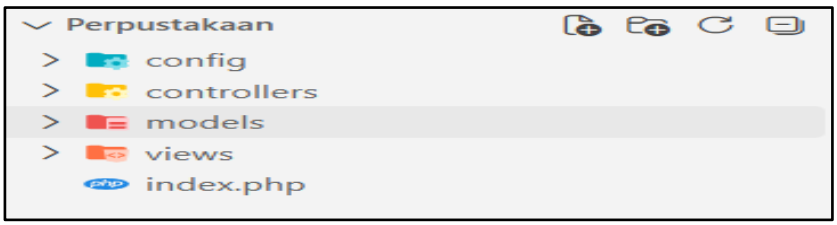
```
mysql> INSERT INTO peminjaman (id_anggota, id_buku, tanggal_pinjam, tanggal_kembali) VALUES
    -> (1, 2, '2026-03-01', '2026-03-08'),
    -> (2, 1, '2026-03-05', '2026-03-12');
Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

4. Membuat Struktur Proyek MVC

a. Buat foder baru di dalam folder *www* dan beri nama foldernya



b. Buat folder proyek perpustakaan dengan struktur direktori sebagai berikut:



c. Konfigurasi Koneksi Basis Data (`*config/database.php*`)



```php
<?php
class Database {
    private $host = "localhost";
    private $db_name = "perpustakaan";
    private $username = "root";
    private $password = "";
    public $conn;

    public function getConnection() {
        $this->conn = null;
        try {
            $this->conn = new PDO("mysql:host=" . $this->host . ";dbname=" . $this->db_name, $this->username, $this->password);
            $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $exception) {
            echo "Koneksi Gagal: " . $exception->getMessage();
        }
        return $this->conn;
    }
}
?>
```

d. Implementasi *Model* (models/Buku*Model.php*)



```php
<?php
require_once "config/database.php";

class BukuModel {
    private $conn;
    private $table_name = "buku";

    public function __construct() {
        $database = new Database();
        $this->conn = $database->getConnection();
    }

    public function getAllBuku() {
        $query = "SELECT * FROM " . $this->table_name;
        $stmt = $this->conn->prepare($query);
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    public function createBuku($judul, $penulis, $tahun_terbit) {
        $query = "INSERT INTO " . $this->table_name . " (judul, penulis, tahun_terbit) VALUES (:judul, :penulis, :tahun)";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(":judul", $judul);
        $stmt->bindParam(":penulis", $penulis);
        $stmt->bindParam(":tahun", $tahun_terbit);
        return $stmt->execute();
    }
}
```

e. Implementasi *Model* (*models*/Anggota*Model.php*)



```php
<?php
require_once "config/database.php";

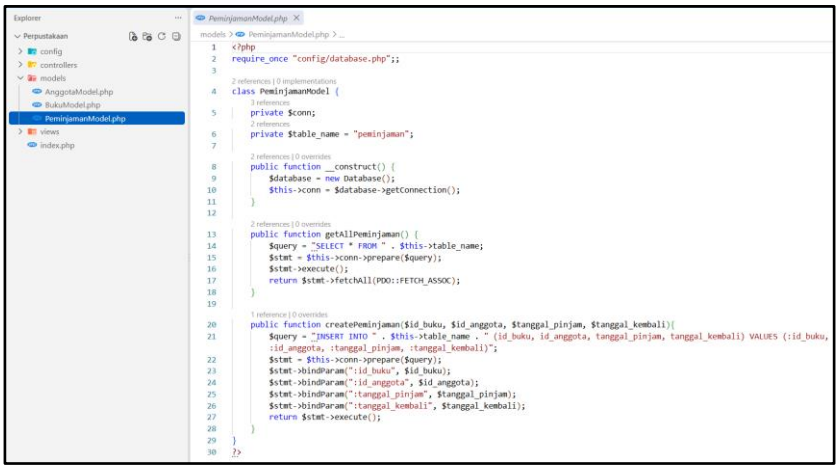
class AnggotaModel{
    private $conn;
    private $table_name = "anggota";

    public function __construct(){
        $database = new Database();
        $this->conn = $database->getConnection();
    }

    public function getAllAnggota() {
        $query = "SELECT * FROM " . $this->table_name;
        $stmt = $this->conn->prepare($query);
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    public function createAnggota($nama, $alamat) {
        $query = "INSERT INTO " . $this->table_name . " (nama, alamat) VALUES (:nama, :alamat)";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(":nama", $nama);
        $stmt->bindParam(":alamat", $alamat);
        return $stmt->execute();
    }
}
?>
```

f. Implementasi *Model* (*models*/Peminjaman*Model.php*)



```php
<?php
require_once "config/database.php";;

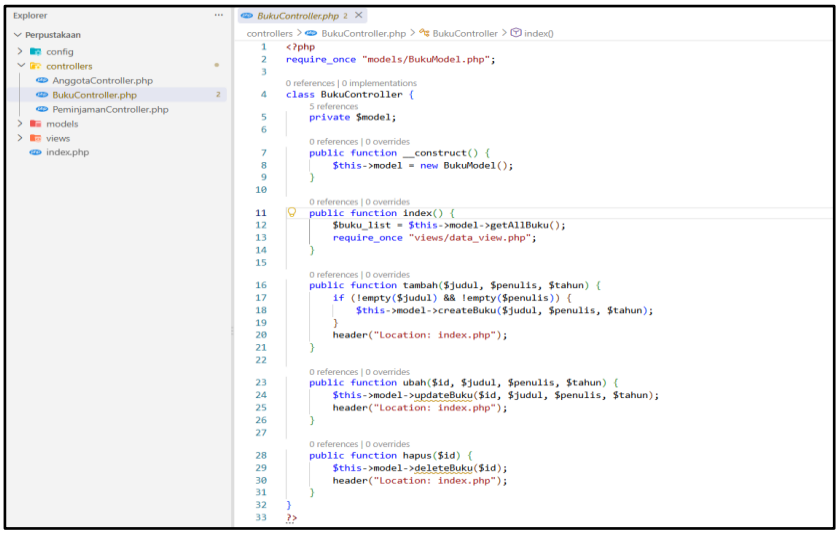
class PeminjamanModel {
    private $conn;
    private $table_name = "peminjaman";

    public function __construct() {
        $database = new Database();
        $this->conn = $database->getConnection();
    }

    public function getAllPeminjaman() {
        $query = "SELECT * FROM " . $this->table_name;
        $stmt = $this->conn->prepare($query);
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    public function createPeminjaman($id_buku, $id_anggota, $tanggal_pinjam, $tanggal_kembali){
        $query = "INSERT INTO " . $this->table_name . " (id_buku, id_anggota, tanggal_pinjam, tanggal_kembali) VALUES (:id_buku,
        :id_anggota, :tanggal_pinjam, :tanggal_kembali)";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(":id_buku", $id_buku);
        $stmt->bindParam(":id_anggota", $id_anggota);
        $stmt->bindParam(":tanggal_pinjam", $tanggal_pinjam);
        $stmt->bindParam(":tanggal_kembali", $tanggal_kembali);
        return $stmt->execute();
    }
}
?>
```

g. Implementasi *Controller* (*controllers*/Buku*Controller.php*)



```php
<?php
require_once "models/BukuModel.php";

class BukuController {
    private $model;

    public function __construct() {
        $this->model = new BukuModel();
    }

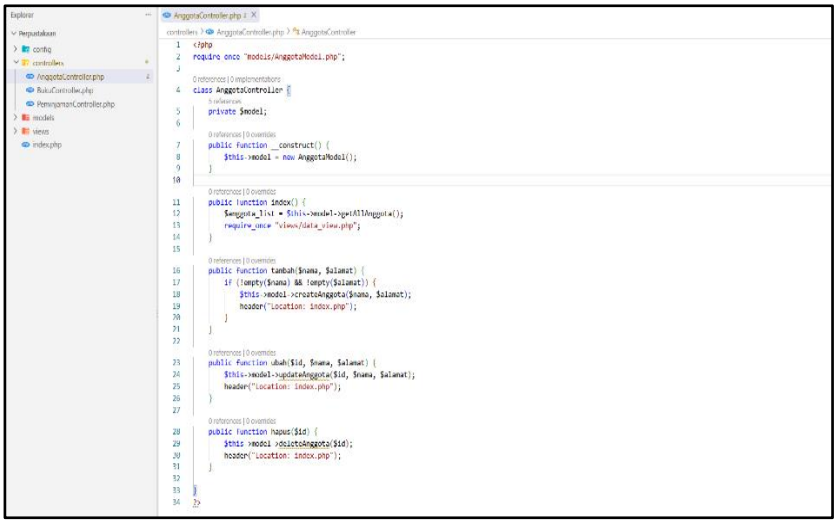
    public function index() {
        $buku_list = $this->model->getAllBuku();
        require_once "views/data_view.php";
    }

    public function tambah($judul, $penulis, $tahun) {
        if (!empty($judul) && !empty($penulis)) {
            $this->model->createBuku($judul, $penulis, $tahun);
        }
        header("Location: index.php");
    }

    public function ubah($id, $judul, $penulis, $tahun) {
        $this->model->updateBuku($id, $judul, $penulis, $tahun);
        header("Location: index.php");
    }

    public function hapus($id) {
        $this->model->deleteBuku($id);
        header("Location: index.php");
    }
}
?>
```

h. Implementasi *Controller* (*controllers*/Anggota*Controller.php*)



```php
<?php
require_once "models/AnggotaModel.php";

class AnggotaController {
    private $model;

    public function __construct() {
        $this->model = new AnggotaModel();
    }

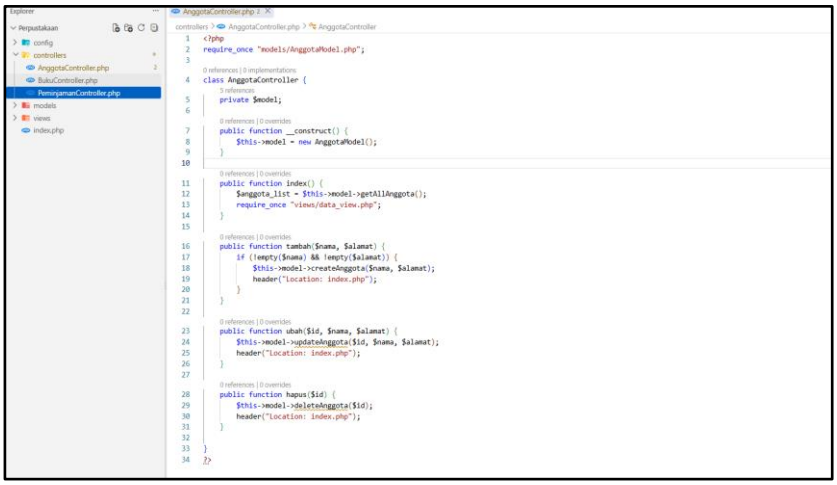
    public function index() {
        $anggota_list = $this->model->getAllAnggota();
        require_once "views/data_view.php";
    }

    public function tambah($nama, $alamat) {
        if (!empty($nama) && !empty($alamat)) {
            $this->model->createAnggota($nama, $alamat);
            header("Location: index.php");
        }
    }

    public function ubah($id, $nama, $alamat) {
        $this->model->updateAnggota($id, $nama, $alamat);
        header("Location: index.php");
    }

    public function hapus($id) {
        $this->model->deleteAnggota($id);
        header("Location: index.php");
    }
}
?>
```

i. Implementasi Controller (*controllers*/Peminjaman*Controller.php*



```php
<?php
require_once "models/AnggotaModel.php";

class AnggotaController {
    private $model;

    public function __construct() {
        $this->model = new AnggotaModel();
    }

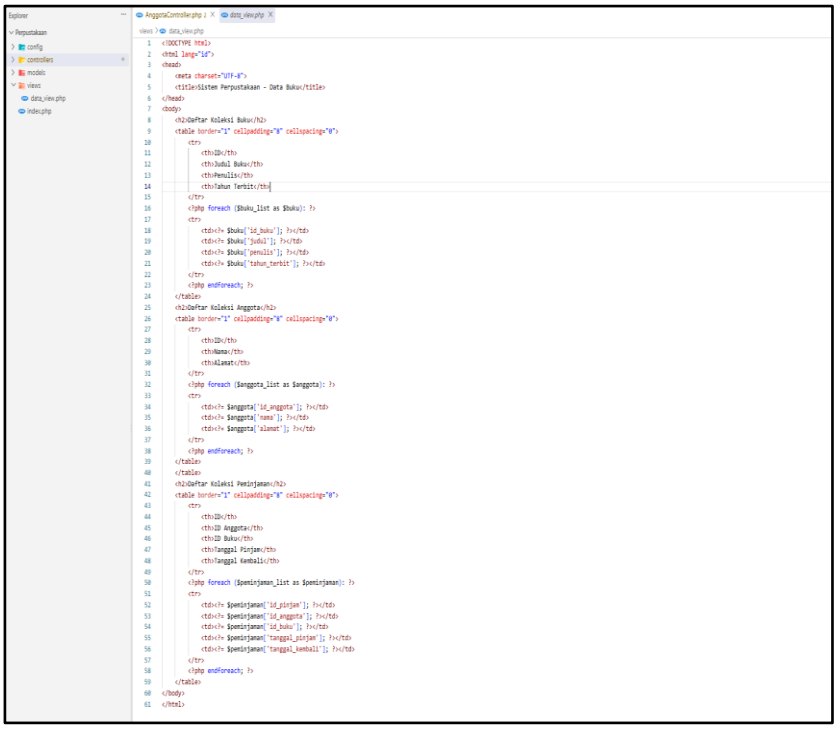
    public function index() {
        $anggota_list = $this->model->getAllAnggota();
        require_once "views/data_view.php";
    }

    public function tambah($nama, $alamat) {
        if (!empty($nama) && !empty($alamat)) {
            $this->model->createAnggota($nama, $alamat);
            header("Location: index.php");
        }
    }

    public function ubah($id, $nama, $alamat) {
        $this->model->updateAnggota($id, $nama, $alamat);
        header("Location: index.php");
    }

    public function hapus($id) {
        $this->model->deleteAnggota($id);
        header("Location: index.php");
    }
}
?>
```

j. Implementasi *View* (*Views/*data_*View.php*)



```php
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Sistem Perpustakaan - Data Buku</title>
</head>
<body>
    <h2>Daftar Koleksi Buku</h2>
    <table border="1" cellpadding="8" cellspacing="0">
        <tr>
            <th>ID</th>
            <th>Judul Buku</th>
            <th>Penulis</th>
            <th>Tahun Terbit</th>
        </tr>
        <?php foreach ($buku_list as $buku): ?>
        <tr>
            <td><?= $buku['id_buku']; ?></td>
            <td><?= $buku['judul']; ?></td>
            <td><?= $buku['penulis']; ?></td>
            <td><?= $buku['tahun_terbit']; ?></td>
        </tr>
        <?php endforeach; ?>
    </table>
    <h2>Daftar Koleksi Anggota</h2>
    <table border="1" cellpadding="8" cellspacing="0">
        <tr>
            <th>ID</th>
            <th>Nama</th>
            <th>Alamat</th>
        </tr>
        <?php foreach ($anggota_list as $anggota): ?>
        <tr>
            <td><?= $anggota['id_anggota']; ?></td>
            <td><?= $anggota['nama']; ?></td>
            <td><?= $anggota['alamat']; ?></td>
        </tr>
        <?php endforeach; ?>
    </table>
    <h2>Daftar Koleksi Peminjaman</h2>
    <table border="1" cellpadding="8" cellspacing="0">
        <tr>
            <th>ID</th>
            <th>ID Anggota</th>
            <th>ID Buku</th>
            <th>Tanggal Pinjam</th>
            <th>Tanggal Kembali</th>
        </tr>
        <?php foreach ($peminjaman_list as $peminjaman): ?>
        <tr>
            <td><?= $peminjaman['id_pinjam']; ?></td>
            <td><?= $peminjaman['id_anggota']; ?></td>
            <td><?= $peminjaman['id_buku']; ?></td>
            <td><?= $peminjaman['tanggal_pinjam']; ?></td>
            <td><?= $peminjaman['tanggal_kembali']; ?></td>
        </tr>
        <?php endforeach; ?>
    </table>
</body>
</html>
```

k. Titik Masuk Utama (*index.php*)



```php
<?php
require_once "models/BukuModel.php";
require_once "models/AnggotaModel.php";
require_once "models/PeminjamanModel.php";

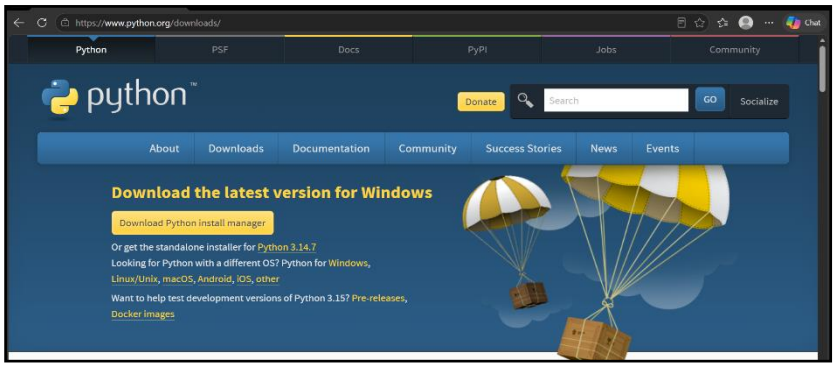
$bukuModel = new BukuModel();
$anggotaModel = new AnggotaModel();
$peminjamanModel = new PeminjamanModel();

$buku_list = $bukuModel->getAllBuku();
$anggota_list = $anggotaModel->getAllAnggota();
$peminjaman_list = $peminjamanModel->getAllPeminjaman();

require_once "views/data_view.php";
```

3.1.3 *Setup Environment, Code Editor,* Dan Versioan Control System (VCS)

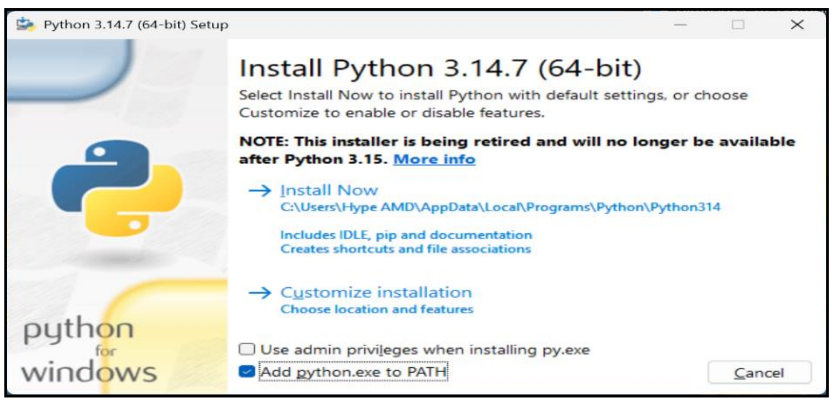
1. Instalasi Python
   a. Buka *web browser* dan unduh *installer Python* versi stabil terbaru dari situs resmi (https://www.python.org/downloads/



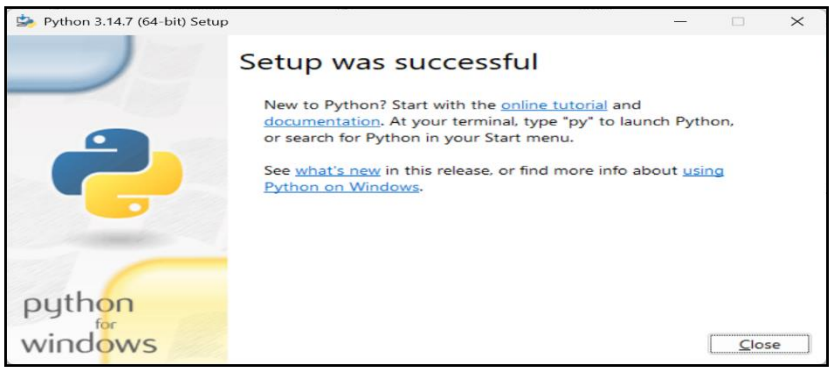
   b. Jalankan file *installer* yang telah diunduh.



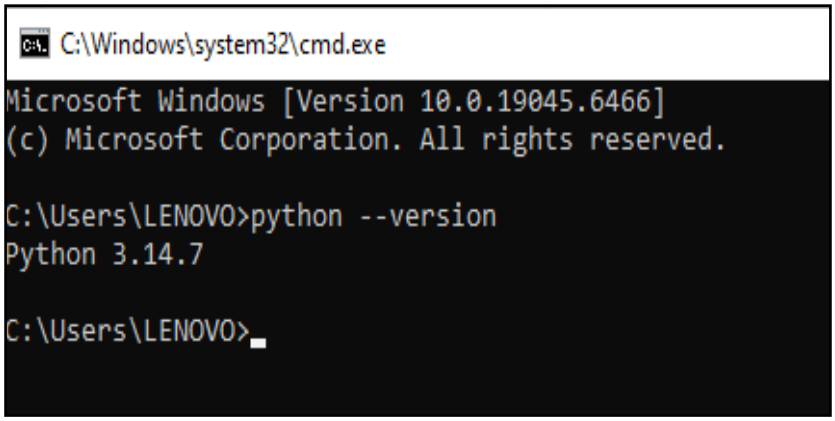
   c. Langkah Krusial: Pada jendela awal instalasi, pastikan Anda mencentang kotak bertuliskan *"Add Python to* PATH" atau "*Add python.exe to PATH*" yang berada di bagian bawah. (Ini wajib agar perintah Python dikenali di terminal).



d. Klik *"Install Now"* dan tunggu hingga proses instalasi selesai, lalu klik *"Close"*.

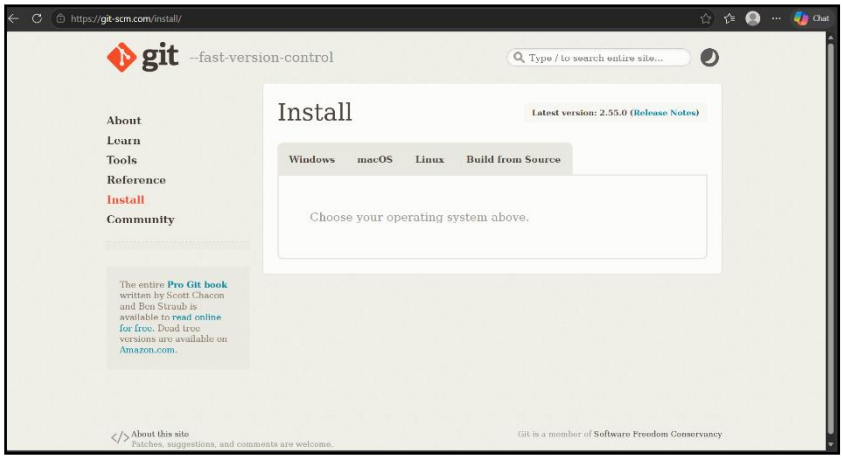


e. Buka *Command Prompt* (CMD) atau *PowerShell,* lalu ketik python --version dan tekan *Enter.* Jika muncul *versi* Python (misal: Python 3.14.x), instalasi berhasil.

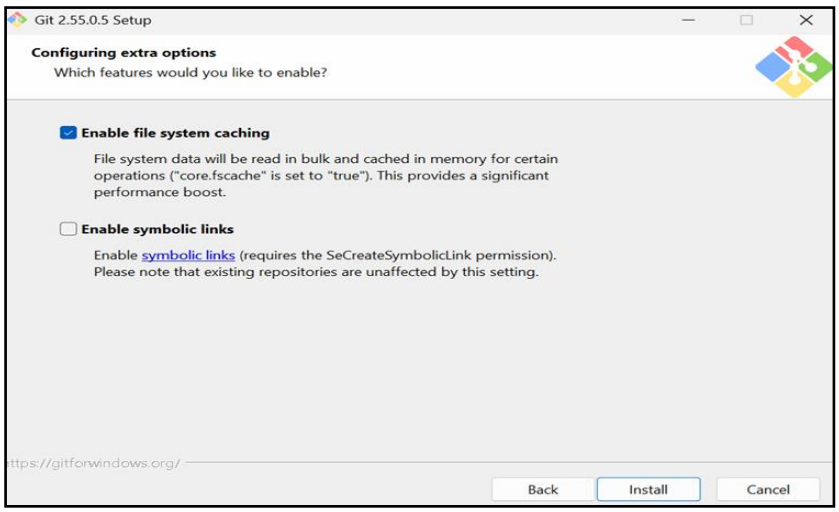


2. Instalasi Git dan Konfigurasi Awal

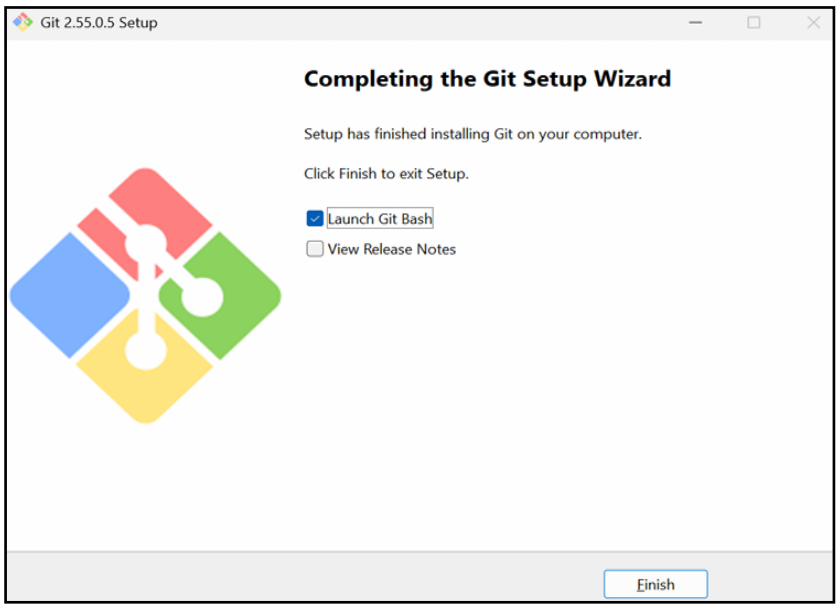
a. Buka *web browser* dan unduh *installer* Git untuk *Windows* dari situs resmi (https://git-scm.com/downloads).



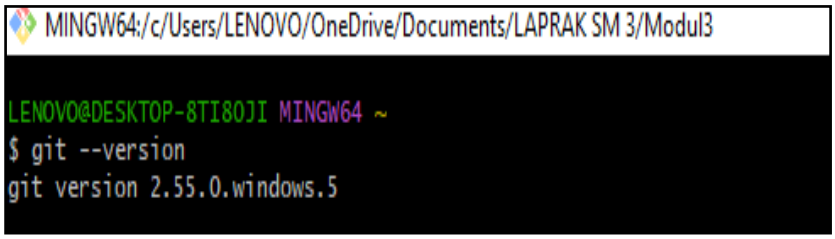
b. Jalankan file *installer*. Anda cukup mengikuti pengaturan bawaan (default) dengan mengklik *Next* terus menerus hingga klik *Install*.



c. Setelah instalasi selesai, buka kembali *Command Prompt* atau PowerShell.



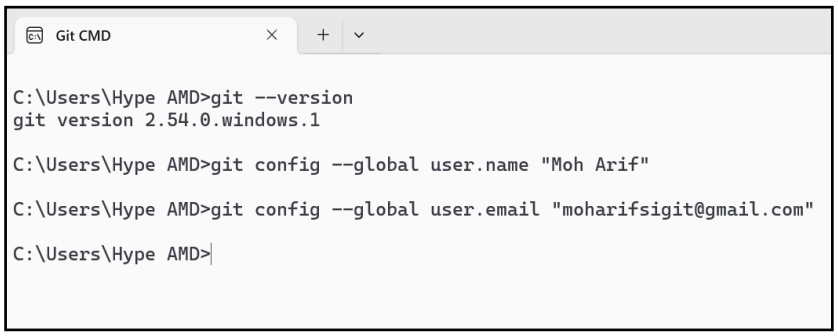
d. Ketik *git --version* untuk memverifikasi bahwa Git sudah terinstal (akan muncul versi Git).

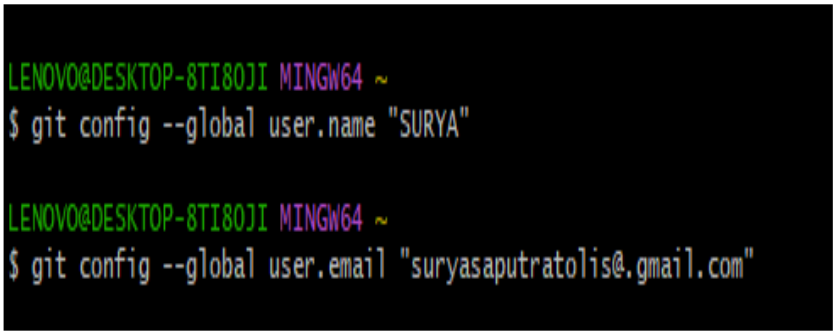


e. Lakukan *konfigurasi* identitas global Git agar setiap perubahan kode memiliki nama pengarangnya. Ketikkan dua baris perintah ini secara berurutan di terminal:

git config --global user.name "Nama Lengkap Anda"

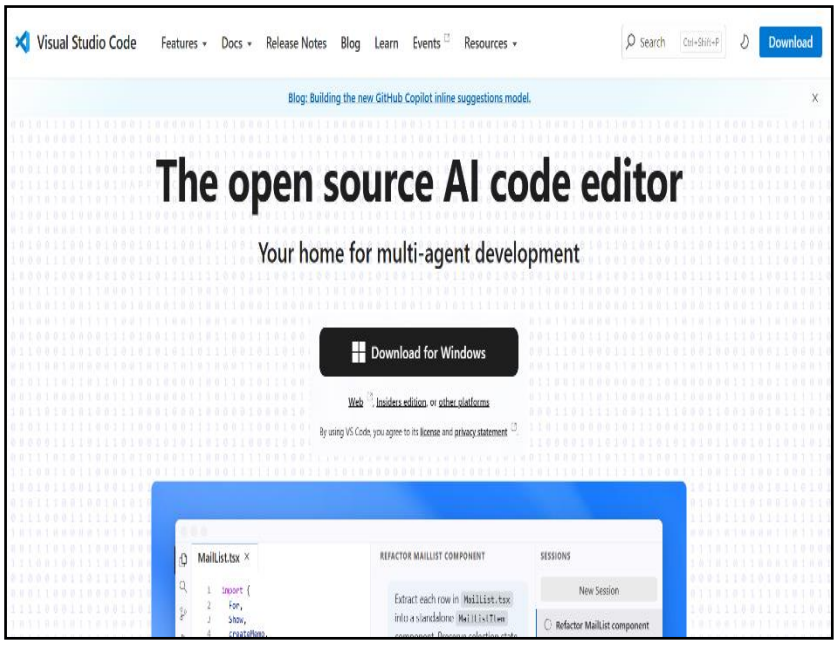
git config --global user.email email.anda@gmail.com



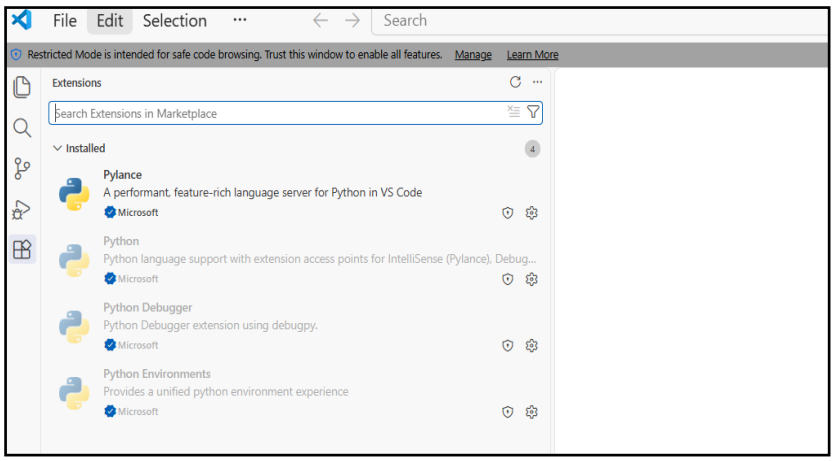


3. Konfigurasi Code Editor (VS Code)

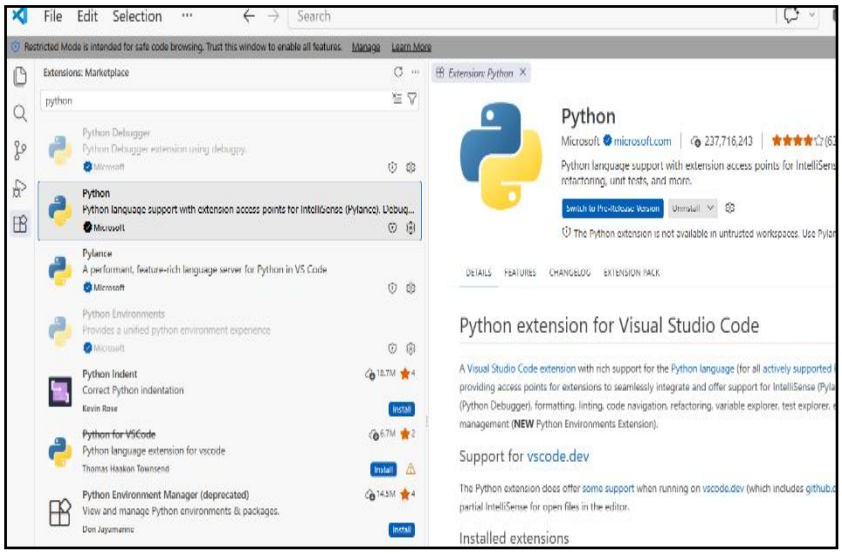
a. Unduh dan *instal Visual Studio Code* dari situs resminya.



b. Buka VS Code, navigasikan ke menu *Extensions* di panel sebelah kiri (atau tekan Ctrl+Shift+X).

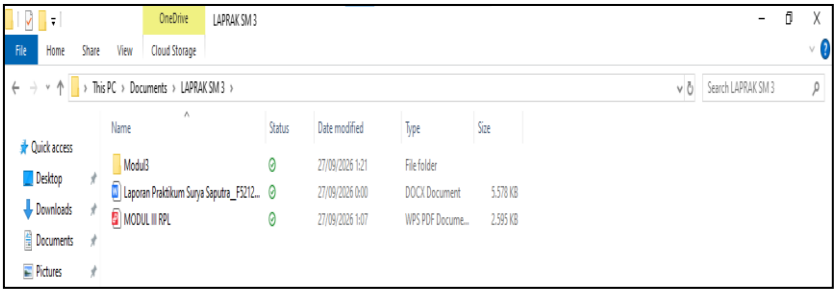


c. Ketik "*Python*" pada bilah pencarian, pilih ekstensi resmi buatan Microsoft, lalu klik *Install*.



4. Pembuatan *Virtual Environment Python*

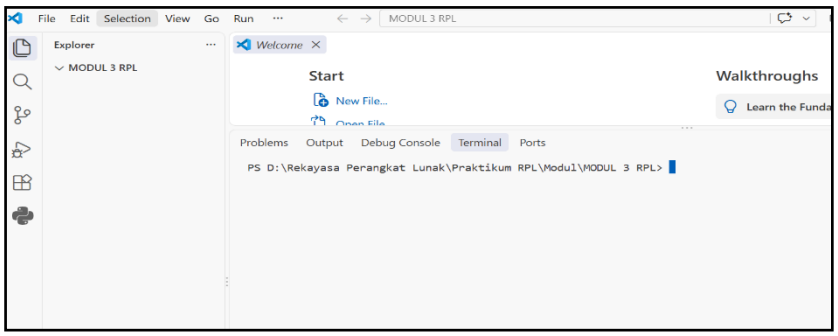
a. Buat folder baru di komputer untuk proyek praktikum (misal: Modul3_RPL).



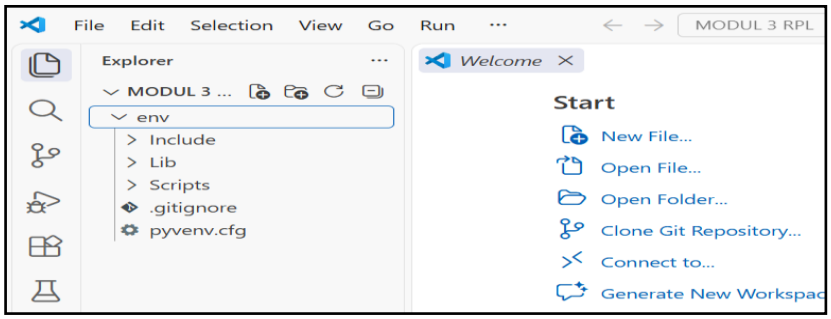
b. Buka VS Code, klik File -> *Open* Folder, dan pilih folder yang baru saja dibuat.



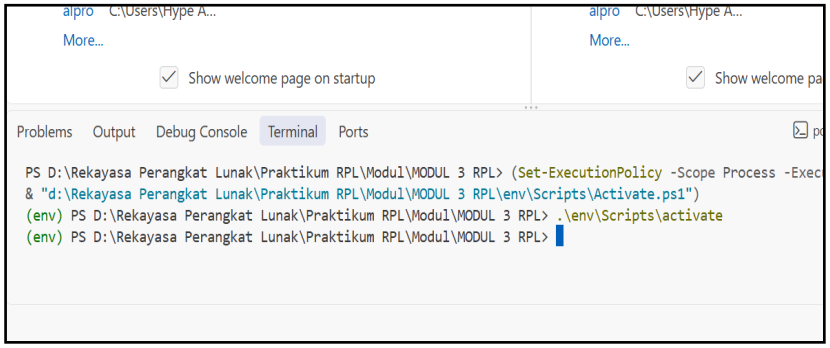
c. Buka terminal terintegrasi dengan mengklik Terminal -> *New* Terminal.



d. Ketik perintah python -m venv env lalu tekan *Enter*. Perintah ini akan membuat folder *virtual environment* baru bernama env.

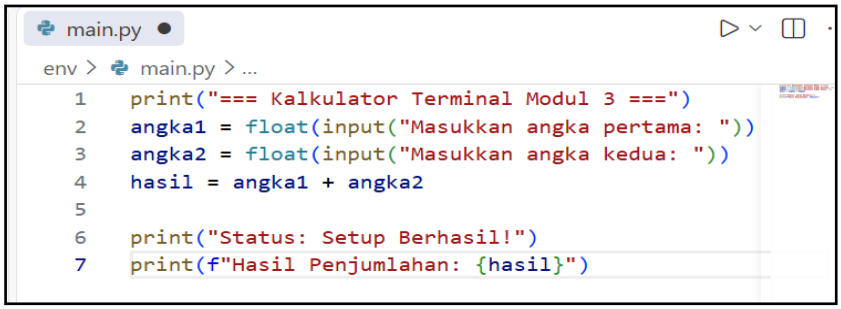


e. Aktifkan virtual environment dengan mengetikkan perintah .\env\Scripts\activate (untuk pengguna Windows).

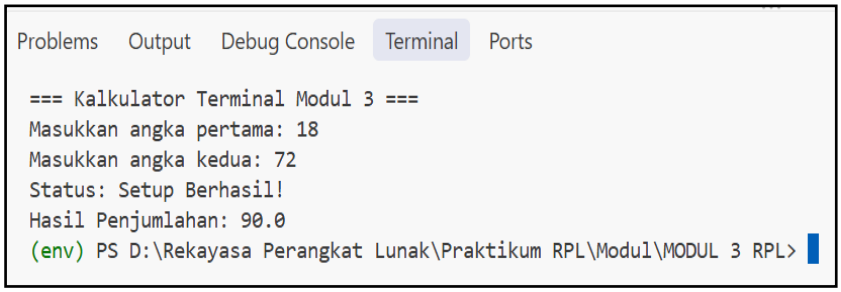


5. Pembuatan Program Awal

   a. Buat file baru di dalam proyek dengan nama main.py dan ketikkan baris kode kalkulator CLI sederhana berikut di dalam file tersebut:

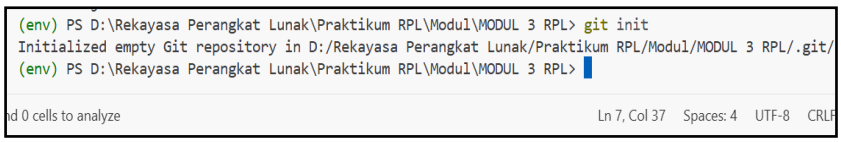


   b. Simpan file (Ctrl+S) dan jalankan kode dengan mengetik perintah python main.py di terminal.

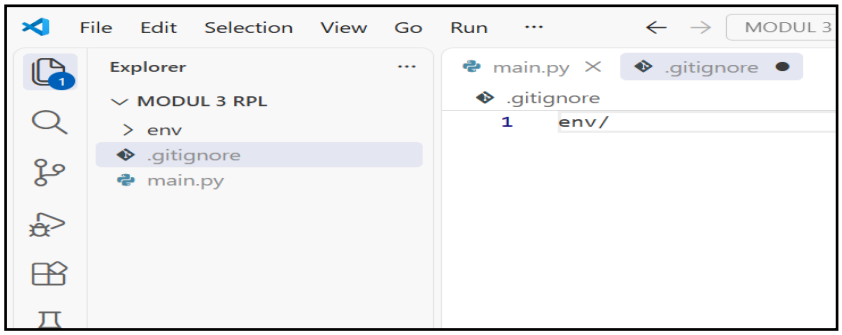


6. Inisialisasi Git dan GitHub

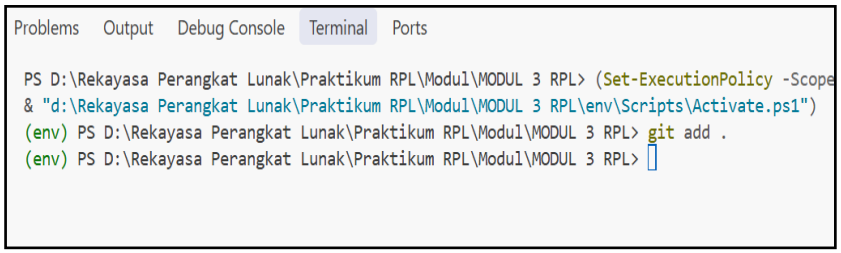
   a. Ketik perintah git init untuk menginisialisasi repositori Git lokal.



   b. Buat file baru bernama .gitignore. Isi file tersebut dengan teks env/ lalu simpan, agar folder virtual environment tidak ikut terunggah ke GitHub.



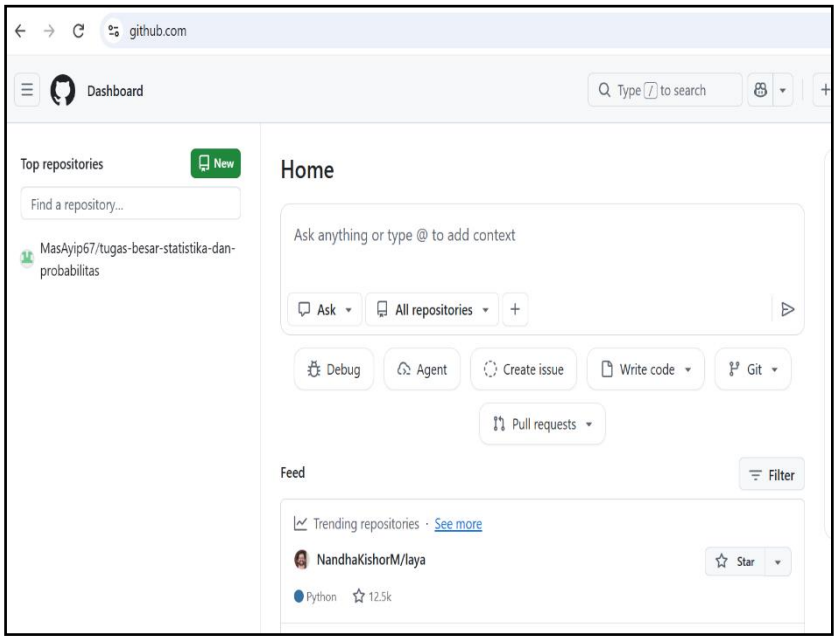
c. Tambahkan seluruh perubahan ke Git dengan perintah git *add* .



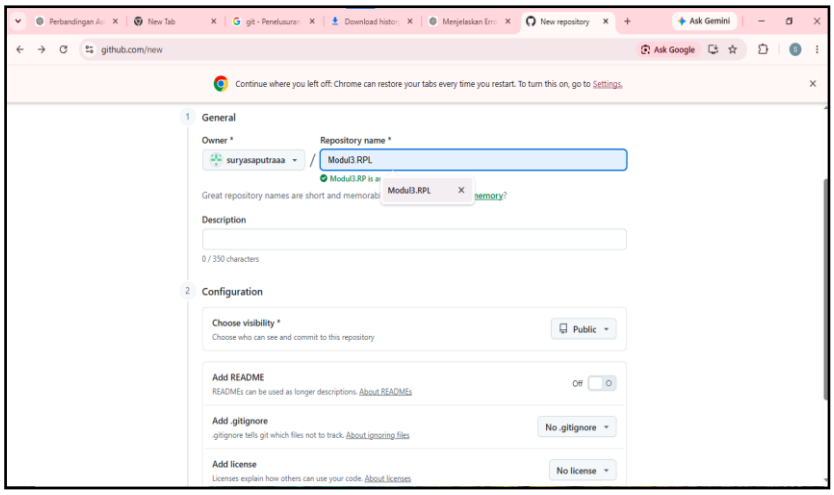
d. Lakukan commit pertama dengan perintah git commit -m "Initial commit modul 3"



e. Buka web browser, masuk ke akun GitHub Anda, dan klik New Repository.



f. Beri nama *repositori* (misal: Modul3_RPL), biarkan *publik*, jangan centang penambahan *README*, lalu klik *Create repository*.



g. Salin perintah yang ada di bagian *"push an existing repository from the command line"*, dan jalankan di terminal VS Code Anda:

git remote add origin https://github.com/username-anda/Modul3_RPL.git

git branch -M main

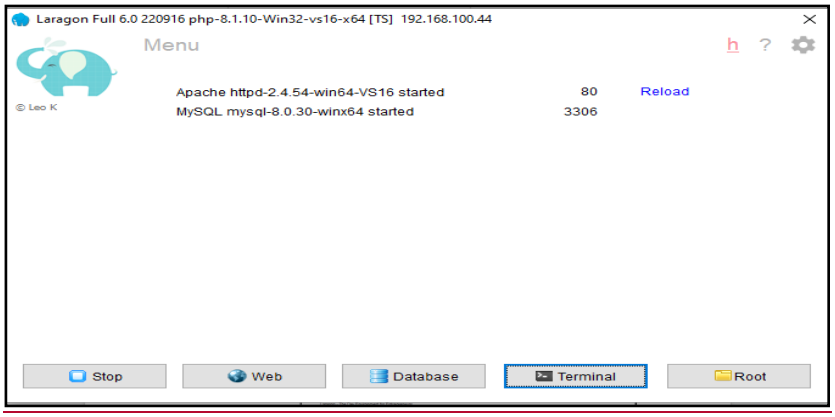
git push -u origin main




```
(env) PS C:\Users\asus\Downloads\Praktikum\Modul3> git remote add origin https://github.com/MasAyip67_RPL.git
(env) PS C:\Users\asus\Downloads\Praktikum\Modul3> git branch -M main
(env) PS C:\Users\asus\Downloads\Praktikum\Modul3> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 416 bytes | 208.00 KiB/s, done.
Total 4 (delta 0), reued 0) (delta 0), pack-reused 0 (from 0)
To https://github.com/MasAyip67/Modul3_RPL.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(env) PS C:\Users\asus\Downloads\Praktikum\Modul3>
```

3.1.4 Perancangan Model Python Dan Integrasi Basis Data *Mysql*

1. Persiapan Layanan Basis Data (Laragon)
   a. Buka aplikasi Laragon, lalu klik tombol Mulai Semua untuk menjalankan service Apache dan *Mysql* .

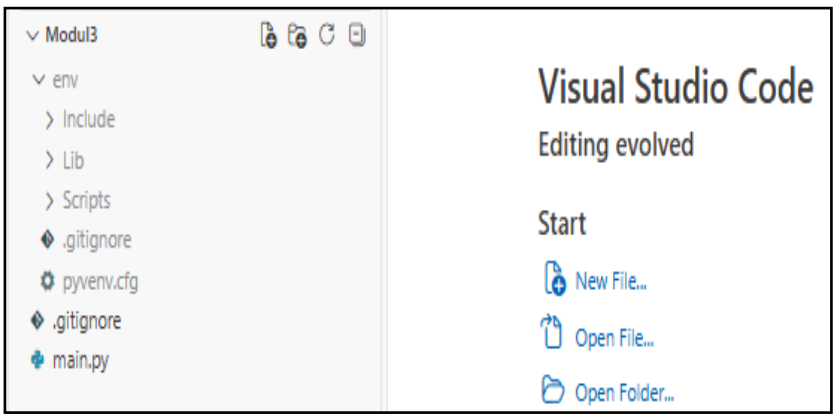


   b. Pastikan basis data perpustakaan beserta tabel buku, anggota, dan peminjaman yang telah dikerjakan pada Modul II masih tersedia dan dapat diakses.

```
mysql> show tables;
+----------------------------+
| Tables_in_db_perpustakaan |
+----------------------------+
| anggota                    |
| buku                       |
| peminjaman                 |
+----------------------------+
3 rows in set (0.00 sec)

mysql>
```

2. Instalasi Pustaka Penghubung Python
   a. Buka folder proyek Python Anda di Visual Studio Code (lanjutan dari Modul 3).



b. Pastikan Virtual Environment (env) sudah aktif di terminal.

c. Instal pustaka konektor *Mysql* dengan mengetik perintah berikut di terminal:

pip install *Mysql* -connector-python.



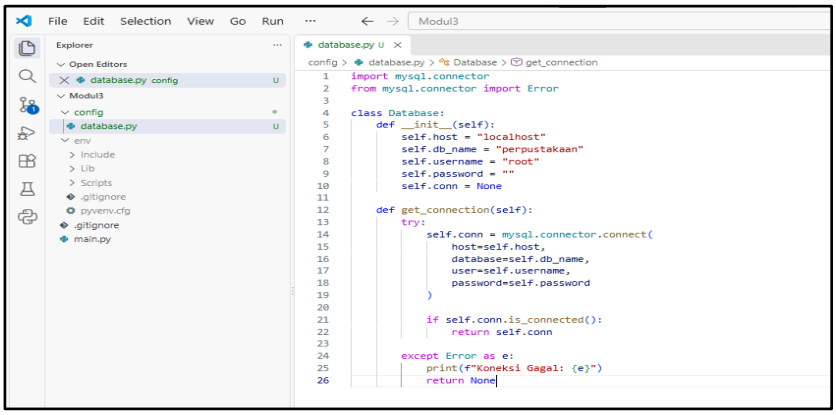
3. Konfigurasi Koneksi Basis Data

a. Buat folder baru di dalam proyek dengan nama config (meniru struktur direktori pada Modul II).



b. Di dalam folder config, buat file bernama database.py

c. Tulis skrip koneksi berikut untuk menghubungkan Python ke database perpustakaan di Laragon:

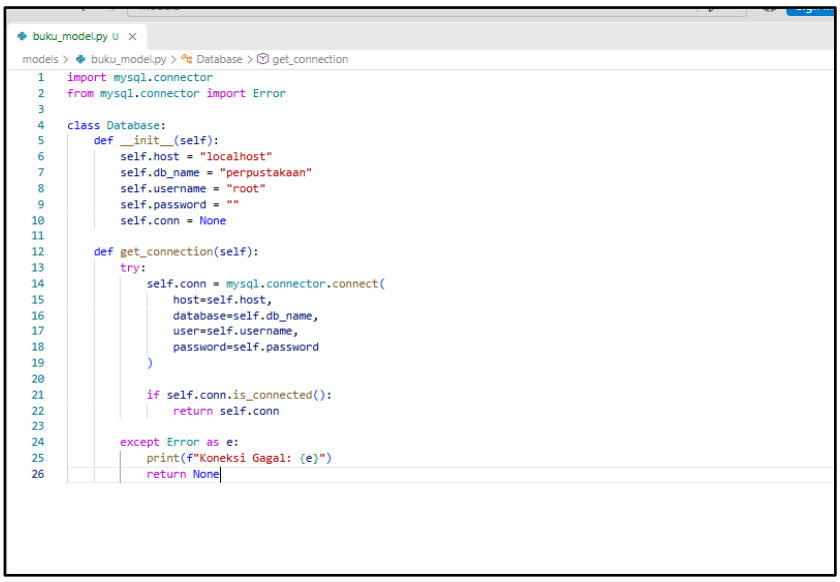


4. Implementasi Model Buku di Python

a. Buat folder models di direktori proyek Anda.

b. Di dalam folder models, buat file buku_model.py.

c. Tulis kode berikut yang disesuaikan dengan struktur kolom judul, penulis, dan tahun_terbit dari tabel buku pada Modul II:



```python
import mysql.connector
from mysql.connector import Error

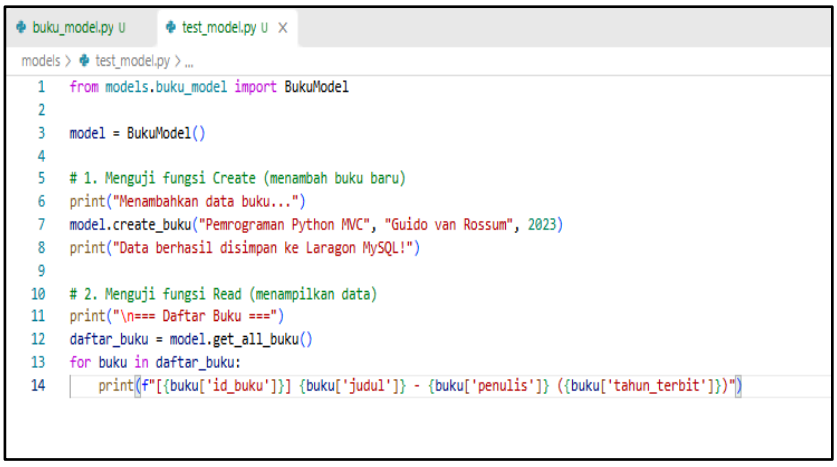
class Database:
    def __init__(self):
        self.host = "localhost"
        self.db_name = "perpustakaan"
        self.username = "root"
        self.password = ""
        self.conn = None

    def get_connection(self):
        try:
            self.conn = mysql.connector.connect(
                host=self.host,
                database=self.db_name,
                user=self.username,
                password=self.password
            )

            if self.conn.is_connected():
                return self.conn

        except Error as e:
            print(f"Koneksi Gagal: {e}")
            return None
```

5. Pengujian Model (Testing)
   a. Buat file test_model.py di luar folder models dan config.
   b. Uji fungsionalitas Model Python Anda dengan skrip berikut:



```python
from models.buku_model import BukuModel

model = BukuModel()

# 1. Menguji fungsi Create (menambah buku baru)
print("Menambahkan data buku...")
model.create_buku("Pemrograman Python MVC", "Guido van Rossum", 2023)
print("Data berhasil disimpan ke Laragon MySQL!")

# 2. Menguji fungsi Read (menampilkan data)
print("\n=== Daftar Buku ===")
daftar_buku = model.get_all_buku()
for buku in daftar_buku:
    print(f"[{buku['id_buku']}] {buku['judul']} - {buku['penulis']} ({buku['tahun_terbit']})")
```

   c. Jalankan perintah python test_model.py di terminal VS Code Anda.



```
(env) PS C:\Users\LENOVO\OneDrive\Documents\LAPRAK SM 3\Modul3> python test_model.py
```

6. Menyimpan Perubahan ke Repositori Git (Commit & Push)
   a. Cek status perubahan file di dalam proyek Anda dengan mengetik perintah:

git status

```
(env) PS C:\Users\LENOVO\OneDrive\Documents\LAPRAK SM 3\Modul3> git status
On branch main
Your branch is up to date with 'origin/main'.
```

b. Tambahkan seluruh file baru (database.py, buku_model.py, dan test_model.py) ke dalam staging area Git menggunakan perintah:
git add

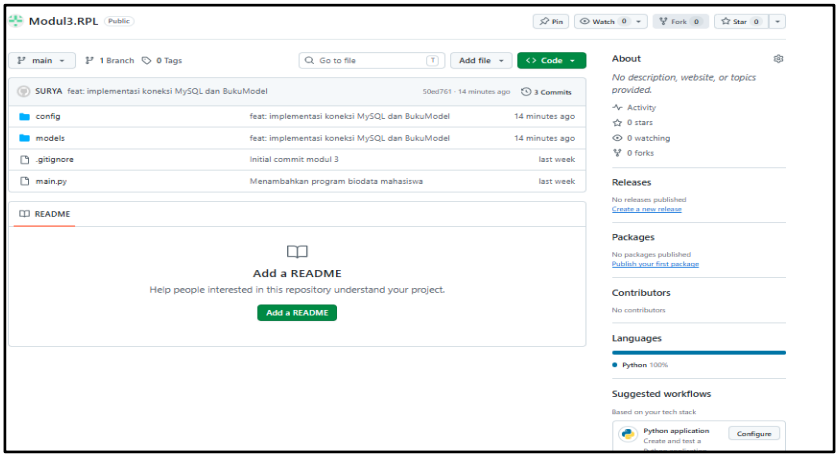
c. Rekam perubahan tersebut ke dalam riwayat repositori lokal dengan perintah:
git commit -m "feat: implementasi koneksi *Mysql* dan BukuModel"

```
(env) PS C:\Users\LENOVO\OneDrive\Documents\LAPRAK SM 3\Modul3> git commit -m "feat: implementasi koneksi MySQL dan BukuModel"
[main 50ed761] feat: implementasi koneksi MySQL dan BukuModel
 5 files changed, 68 insertions(+)
 create mode 100644 config/database.py
 create mode 100644 models/__pycache__/buku_model.cpython-313.pyc
 create mode 100644 models/__pycache__/test_model.cpython-313.pyc
 create mode 100644 models/buku_model.py
 create mode 100644 models/test_model.py
(env) PS C:\Users\LENOVO\OneDrive\Documents\LAPRAK SM 3\Modul3>
```

d. Unggah pembaruan proyek Anda ke repositori *GitHub* yang telah Anda buat pada Modul 3 dengan perintah:
git push origin main

```
(env) PS C:\Users\LENOVO\OneDrive\Documents\LAPRAK SM 3\Modul3> git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 3.03 KiB | 517.00 KiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/suryasaputraaa/Tugas_Modul3_F5212530116.git
   040de38..50ed761  main -> main
(env) PS C:\Users\LENOVO\OneDrive\Documents\LAPRAK SM 3\Modul3>
```

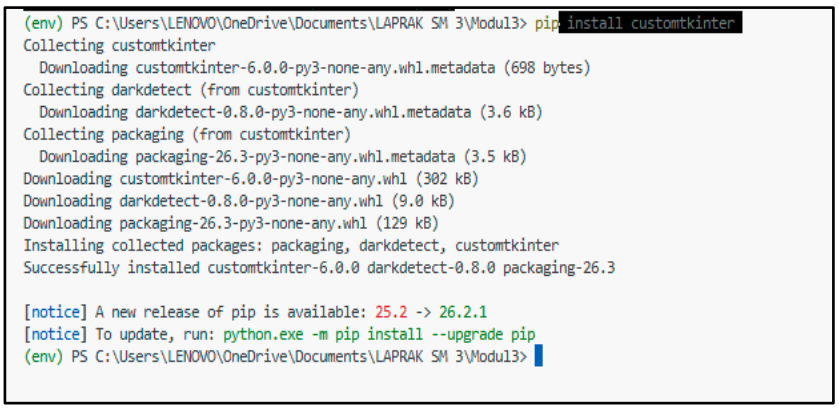
e. Buka repositori GitHub Anda di browser dan pastikan struktur folder config dan models sudah muncul.



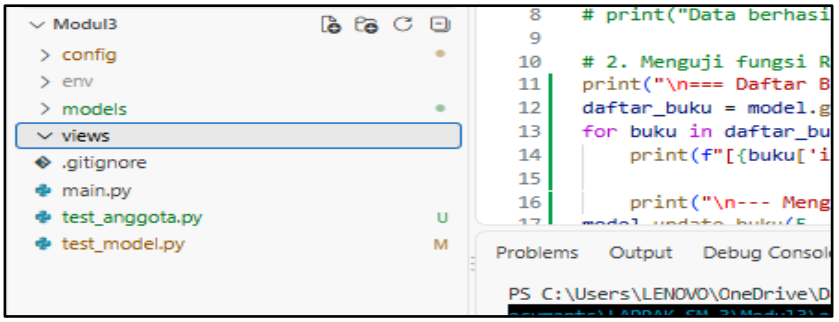
3.1.5 Pengembangan Antarmuka Pengguna (*View*) Berbasis *Desktop*

1. Instalasi Pustaka Antarmuka Grafis
   a. Buka folder proyek Anda di VS Code dan pastikan Virtual Environment (env) aktif.
   b. Ketik perintah berikut di terminal untuk menginstal pustaka GUI modern:

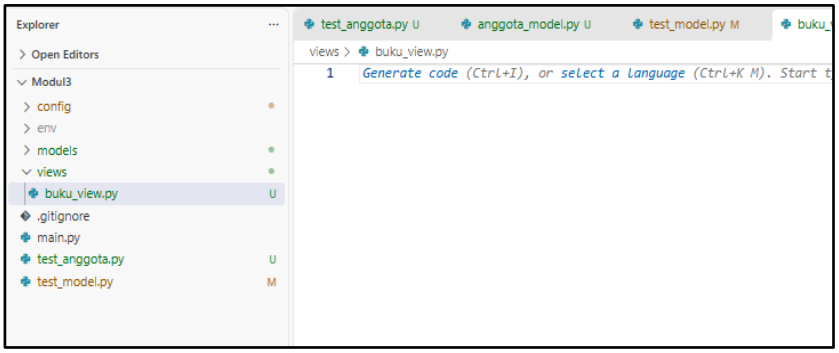
      pip install customtkinter



2. Membuat Struktur Folder View
   a. Buat folder baru di dalam direktori proyek Anda dengan nama views (menyesuaikan arsitektur proyek MVC).

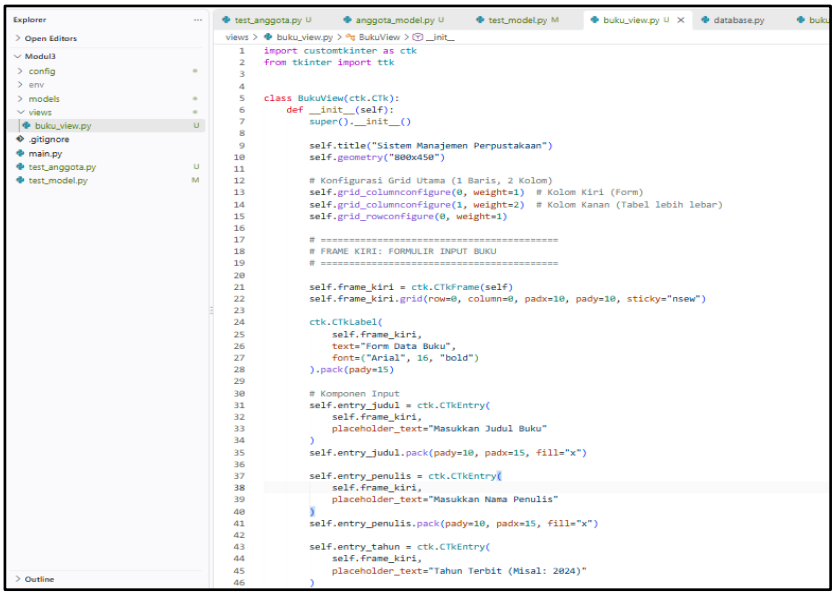


   b. Di dalam folder views, buat file baru bernama buku_view.py.



3. Implementasi Antarmuka Buku

   a. Tulis kode berikut di dalam file buku_view.py. Antarmuka ini dibagi menjadi dua bagian (Frame Kiri untuk Form Input, dan Frame Kanan untuk Tabel Data).



```
import customtkinter as ctk
from tkinter import ttk


class BukuView(ctk.CTk):
    def __init__(self):
        super().__init__()

        self.title("Sistem Manajemen Perpustakaan")
        self.geometry("800x450")

        # Konfigurasi Grid Utama (1 Baris, 2 Kolom)
        self.grid_columnconfigure(0, weight=1)  # Kolom Kiri (Form)
        self.grid_columnconfigure(1, weight=2)  # Kolom Kanan (Tabel lebih lebar)
        self.grid_rowconfigure(0, weight=1)

        # ========================================
        # FRAME KIRI: FORMULIR INPUT BUKU
        # ========================================

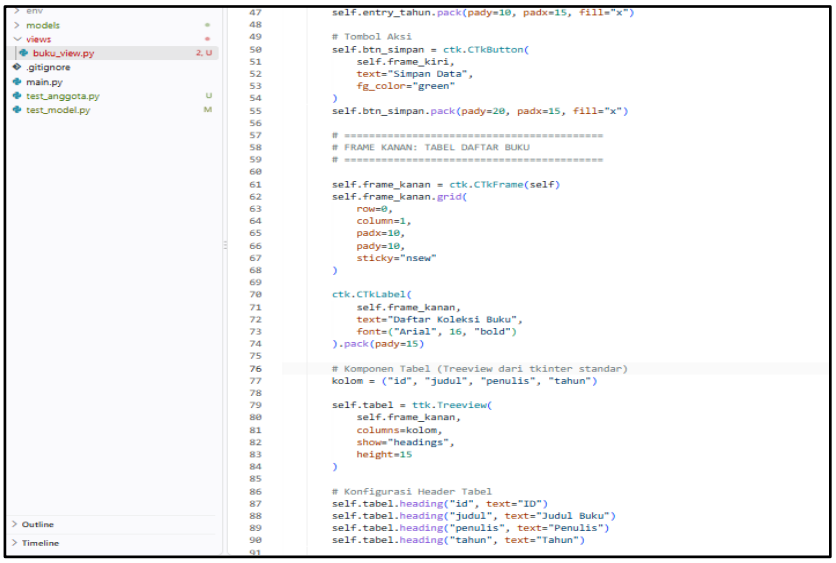
        self.frame_kiri = ctk.CTkFrame(self)
        self.frame_kiri.grid(row=0, column=0, padx=10, pady=10, sticky="nsew")

        ctk.CTkLabel(
            self.frame_kiri,
            text="Form Data Buku",
            font=("Arial", 16, "bold")
        ).pack(pady=15)

        # Komponen Input
        self.entry_judul = ctk.CTkEntry(
            self.frame_kiri,
            placeholder_text="Masukkan Judul Buku"
        )
        self.entry_judul.pack(pady=10, padx=15, fill="x")

        self.entry_penulis = ctk.CTkEntry(
            self.frame_kiri,
            placeholder_text="Masukkan Nama Penulis"
        )
        self.entry_penulis.pack(pady=10, padx=15, fill="x")

        self.entry_tahun = ctk.CTkEntry(
            self.frame_kiri,
            placeholder_text="Tahun Terbit (Misal: 2024)"
        )
```



```
        self.entry_tahun.pack(pady=10, padx=15, fill="x")

        # Tombol Aksi
        self.btn_simpan = ctk.CTkButton(
            self.frame_kiri,
            text="Simpan Data",
            fg_color="green"
        )
        self.btn_simpan.pack(pady=20, padx=15, fill="x")

        # ========================================
        # FRAME KANAN: TABEL DAFTAR BUKU
        # ========================================

        self.frame_kanan = ctk.CTkFrame(self)
        self.frame_kanan.grid(
            row=0,
            column=1,
            padx=10,
            pady=10,
            sticky="nsew"
        )

        ctk.CTkLabel(
            self.frame_kanan,
            text="Daftar Koleksi Buku",
            font=("Arial", 16, "bold")
        ).pack(pady=15)

        # Komponen Tabel (Treeview dari tkinter standar)
        kolom = ("id", "judul", "penulis", "tahun")

        self.tabel = ttk.Treeview(
            self.frame_kanan,
            columns=kolom,
            show="headings",
            height=15
        )

        # Konfigurasi Header Tabel
        self.tabel.heading("id", text="ID")
        self.tabel.heading("judul", text="Judul Buku")
        self.tabel.heading("penulis", text="Penulis")
        self.tabel.heading("tahun", text="Tahun")
```

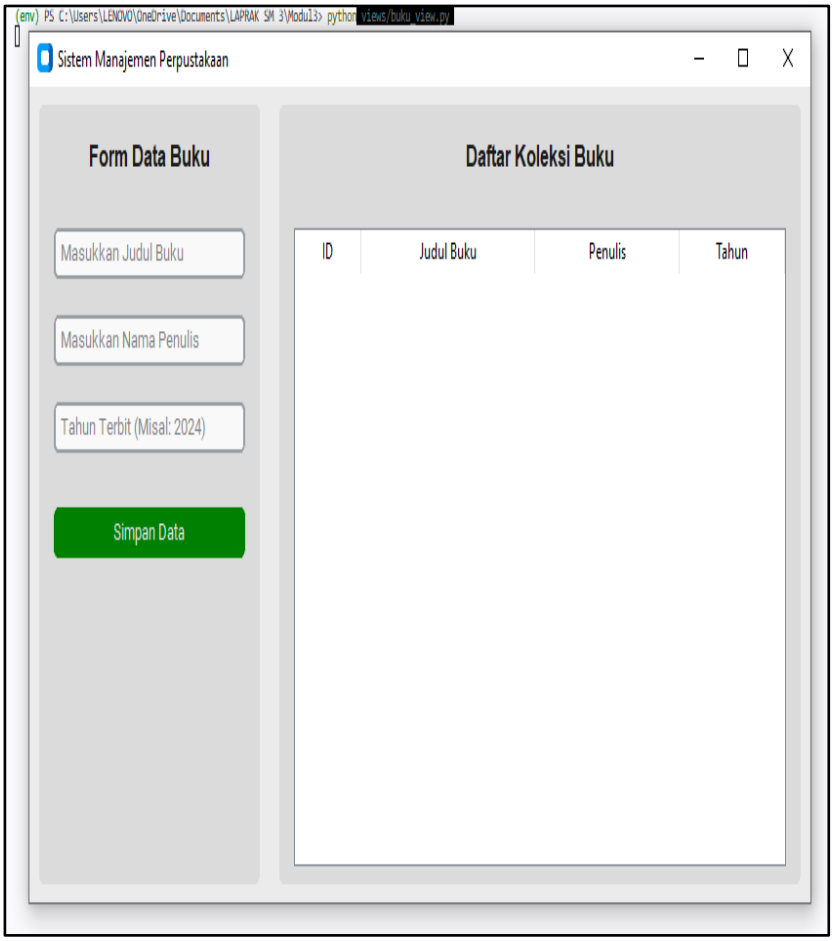


```
        # Konfigurasi Lebar Kolom
        self.tabel.column("id", width=40, anchor="center")
        self.tabel.column("judul", width=150)
        self.tabel.column("penulis", width=120)
        self.tabel.column("tahun", width=80, anchor="center")

        self.tabel.pack(
            fill="both",
            expand=True,
            padx=15,
            pady=10
        )


# Blok eksekusi untuk menguji tampilan grafis
if __name__ == "__main__":
    app = BukuView()
    app.mainloop()
```

4. Pengujian Antarmuka Pengguna
    a. Pastikan Virtual Environment aktif di terminal VS Code.
    b. Jalankan skrip View secara mandiri dengan perintah:
    python views/buku_view.py



    c. Akan muncul sebuah jendela aplikasi desktop. Periksa apakah letak form input, tombol, dan tabel sudah rapi sesuai dengan konfigurasi geometri. (Catatan: Saat ini, tombol simpan belum berfungsi karena belum dihubungkan ke Controller).
5. Menyimpan Perubahan ke Repositori Git a
    a. Cek status proyek dengan git status.
    b. Tambahkan file tampilan ke Git: git add views/buku_view.py
    c. Simpan riwayat perubahan: git commit -m "feat: merancang UI form dan tabel untuk View Buku"
    d. Unggah ke GitHub: git push origin main

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil dan Pembahasan

4.1.1 Konsep Dasar RPL, Pemodelan Aliran Data (DFD), *Unified Modeling Language* (UML) Menggunakan *Platform Draw.io*

1. Pembuatan Diagram Konteks di Draw.io

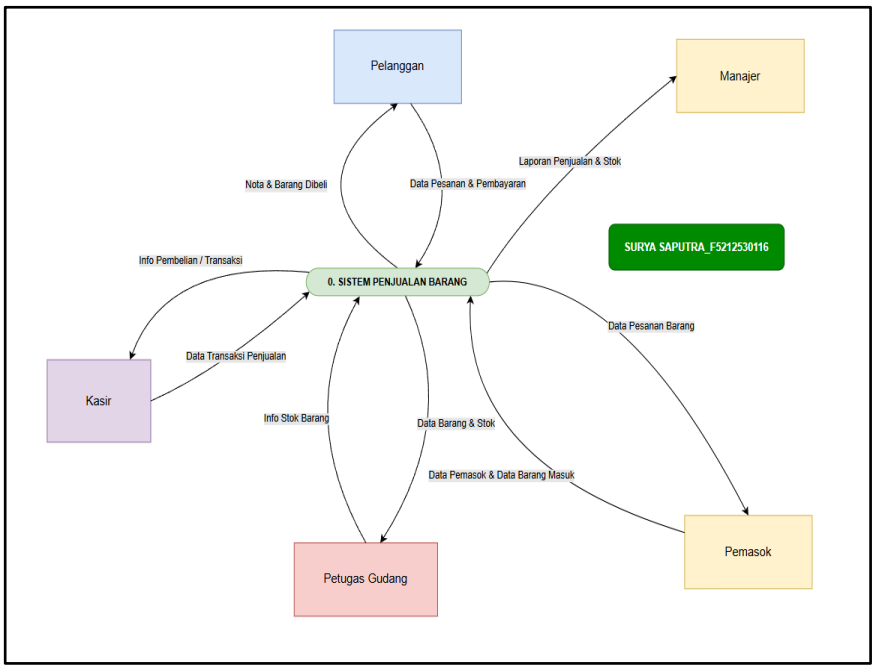


Diagram Konteks (Level 0) ini menggambarkan batasan operasional dari Sistem Penjualan Barang yang menghubungkan sistem utama dengan lima entitas eksternal, yaitu Pelanggan, Kasir, Petugas Gudang, Pemasok, dan Manajer. Pelanggan memberikan data pesanan dan pembayaran, kemudian menerima nota dan barang yang dibeli. Kasir memasukkan data transaksi penjualan dan menerima informasi pembelian, sedangkan Petugas Gudang memberikan serta menerima informasi mengenai data barang dan stok. Pemasok menerima data pesanan barang dari sistem dan memberikan data pemasok serta barang yang masuk. Sementara itu, Manajer menerima laporan penjualan dan stok untuk memantau kegiatan operasional. Seluruh aliran data tersebut terpusat pada Sistem

Penjualan Barang sehingga proses transaksi, pengelolaan stok, pemesanan barang, dan pelaporan dapat dilakukan secara lebih terstruktur dan mudah dipantau.

2. Pembuatan DFD Level 1 di Draw.io

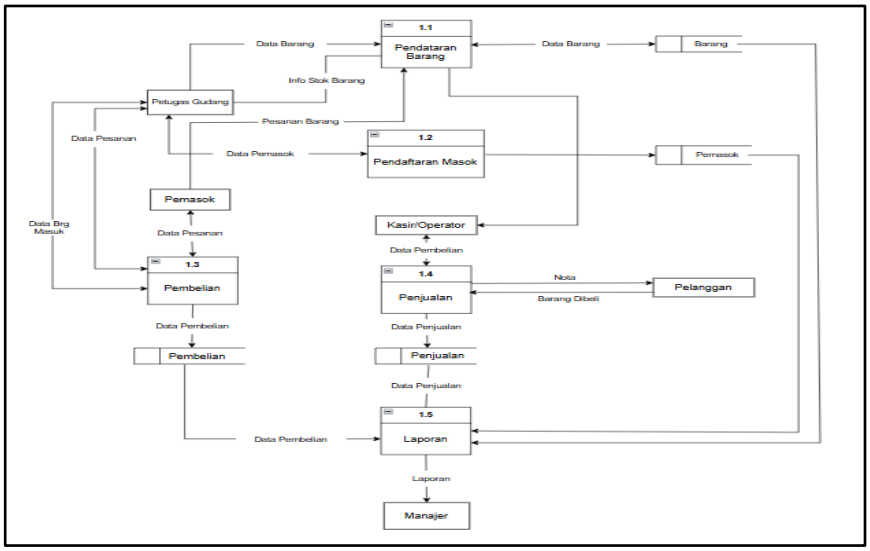


Diagram DFD Level 1 ini menggambarkan pemecahan dari proses utama Sistem Penjualan Barang menjadi beberapa proses yang saling berhubungan, yaitu Pendataan Barang, Pendataan Masuk, Pembelian, Penjualan, dan Laporan. Pada proses pendataan barang, Petugas Gudang memberikan data barang dan sistem mengelola informasi stok yang kemudian disimpan pada data store Barang. Proses pendataan masuk menerima data pemasok dan informasi barang yang masuk untuk dicatat pada data store Pemasok dan memperbarui persediaan barang. Selanjutnya, proses Pembelian mengelola data pesanan dan data pembelian yang melibatkan Pemasok serta Petugas Gudang, kemudian menyimpan data pembelian sebagai riwayat transaksi. Pada proses Penjualan, Kasir/Operator memasukkan data pembelian pelanggan, kemudian sistem menghasilkan nota dan informasi barang yang dibeli untuk Pelanggan serta menyimpan data penjualan. Seluruh data pembelian, penjualan, dan persediaan kemudian digunakan dalam proses

Laporan untuk menghasilkan laporan yang diberikan kepada Manajer sebagai bahan pemantauan dan pengambilan keputusan.

3. Pembuatan *Use Case Diagram* di Draw.io

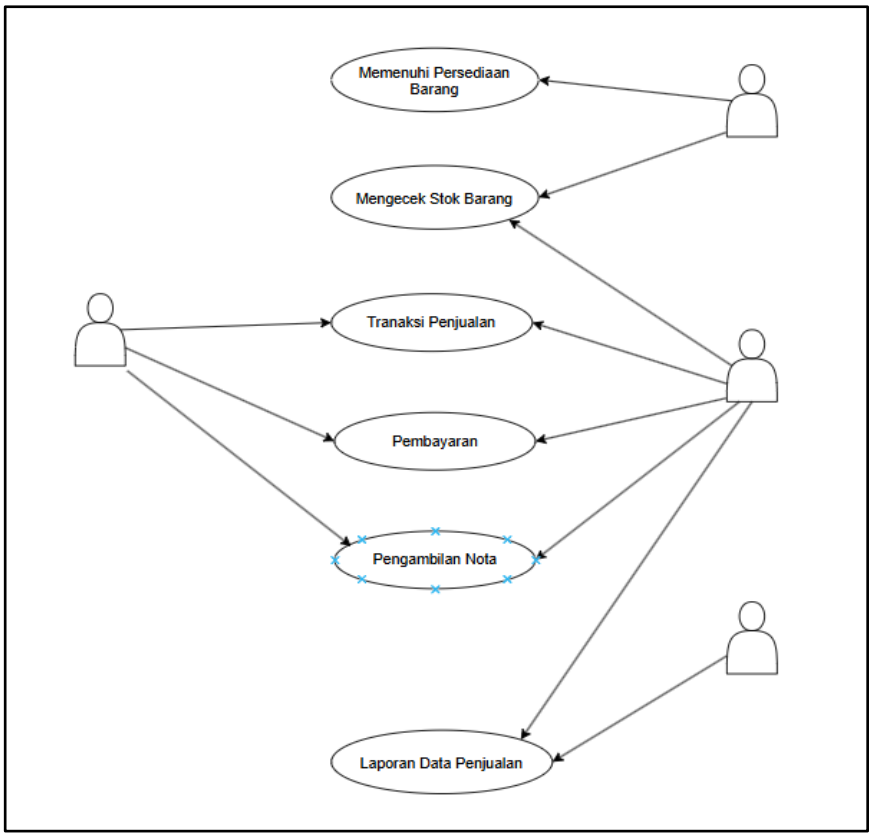


Diagram *Use Case* ini menggambarkan interaksi pengguna dengan Sistem Penjualan Barang melalui beberapa fungsi utama, yaitu Memenuhi Persediaan Barang, Mengecek Stok Barang, Transaksi Penjualan, Pembayaran, Pengambilan Nota, dan Laporan Data Penjualan. Petugas Gudang bertugas memenuhi persediaan dan mengecek stok barang, sedangkan Pelanggan melakukan transaksi penjualan, pembayaran, dan pengambilan nota sebagai bukti pembelian. Kasir/Operator membantu mengelola transaksi, pembayaran, nota, serta pengecekan stok dan laporan penjualan. Sementara itu, Manajer menerima Laporan Data Penjualan untuk memantau hasil penjualan dan kondisi operasional. Secara keseluruhan, diagram ini menunjukkan pembagian fungsi setiap pengguna agar proses penjualan, pengelolaan stok, pembayaran, dan

pelaporan dapat berjalan secara terstruktur, efektif, dan sesuai dengan tanggung jawab masing-masing pihak.

4. Pembuatan *Sequence Diagram* di Draw.io

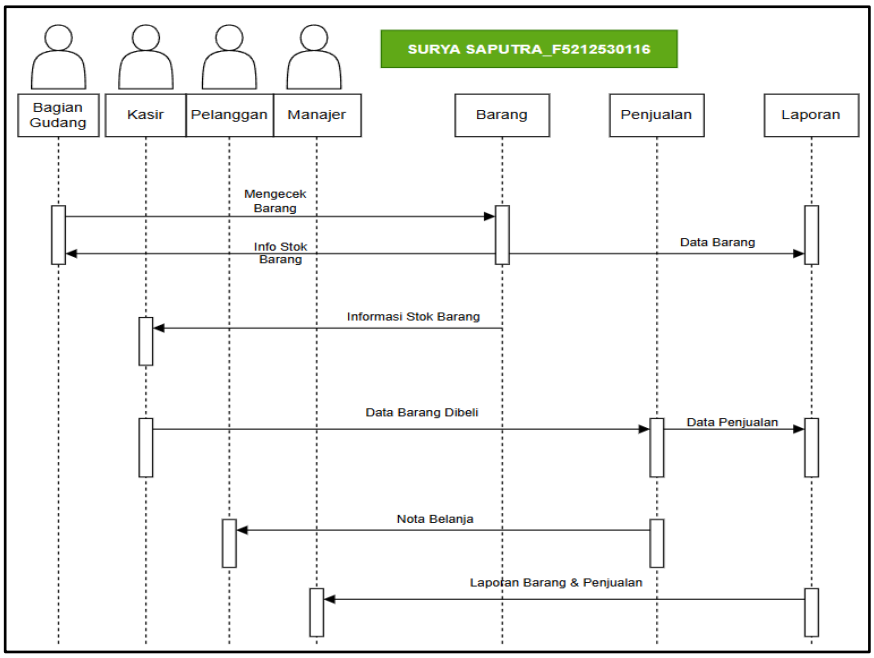


Diagram *Sequence* ini menggambarkan urutan interaksi antara Bagian Gudang, Kasir, Pelanggan, Manajer, serta sistem Barang, Penjualan, dan Laporan dalam proses penjualan barang. Proses dimulai ketika Bagian Gudang melakukan pengecekan barang melalui sistem Barang, kemudian sistem memberikan informasi stok barang yang tersedia. Informasi tersebut juga diteruskan kepada Kasir sebagai bahan untuk mengetahui ketersediaan barang saat melayani pelanggan. Selanjutnya, Kasir memasukkan data barang yang dibeli ke dalam sistem Penjualan, kemudian data penjualan diteruskan ke sistem Laporan untuk dicatat. Setelah transaksi berhasil diproses, sistem Penjualan memberikan nota belanja kepada Pelanggan sebagai bukti transaksi. Pada tahap akhir, sistem Laporan menghasilkan laporan barang dan penjualan yang diberikan kepada Manajer untuk memantau aktivitas penjualan dan kondisi persediaan. Secara keseluruhan, diagram ini menunjukkan aliran komunikasi dan urutan proses antarbagian sehingga kegiatan

pengecekan stok, transaksi, pencatatan penjualan, pemberian nota, dan pelaporan dapat berjalan secara terstruktur dan terkoordinasi.

5. Pembuatan *Activity Diagram* di Draw.io

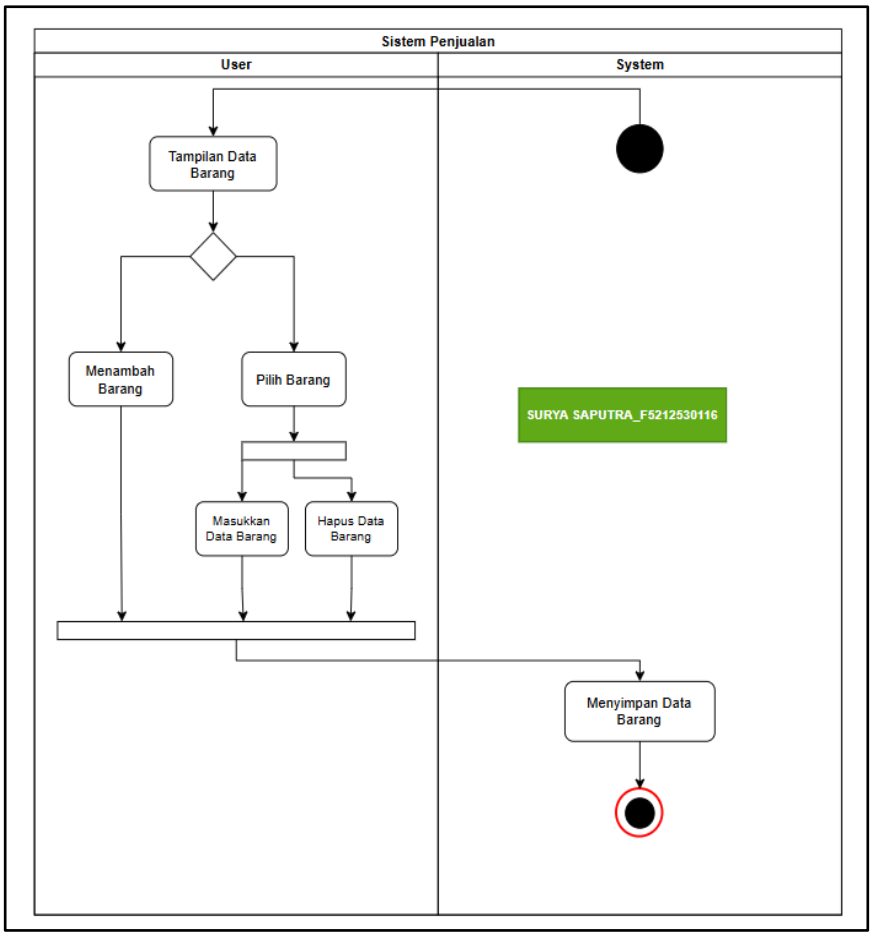


Diagram *Activity* ini menggambarkan alur proses pengelolaan data barang pada Sistem Penjualan yang melibatkan *User* dan *System*. Proses dimulai ketika *User* melihat tampilan data barang, kemudian sistem memberikan pilihan untuk menambah atau memilih barang yang tersedia. Jika memilih menambah barang, *User* memasukkan data barang baru, sedangkan jika memilih barang yang sudah ada, *User* dapat memasukkan perubahan atau menghapus data barang. Setelah proses tersebut selesai, data barang dikirim ke sistem untuk disimpan. Sistem kemudian melakukan proses penyimpanan data barang dan mengakhiri proses. Secara keseluruhan, diagram ini menunjukkan tahapan pengelolaan data barang mulai dari melihat

data, menambah, memilih, mengubah, atau menghapus data hingga menyimpan data ke dalam sistem secara terstruktur.

6. Pembuatan *Context Diagram* di Draw.io

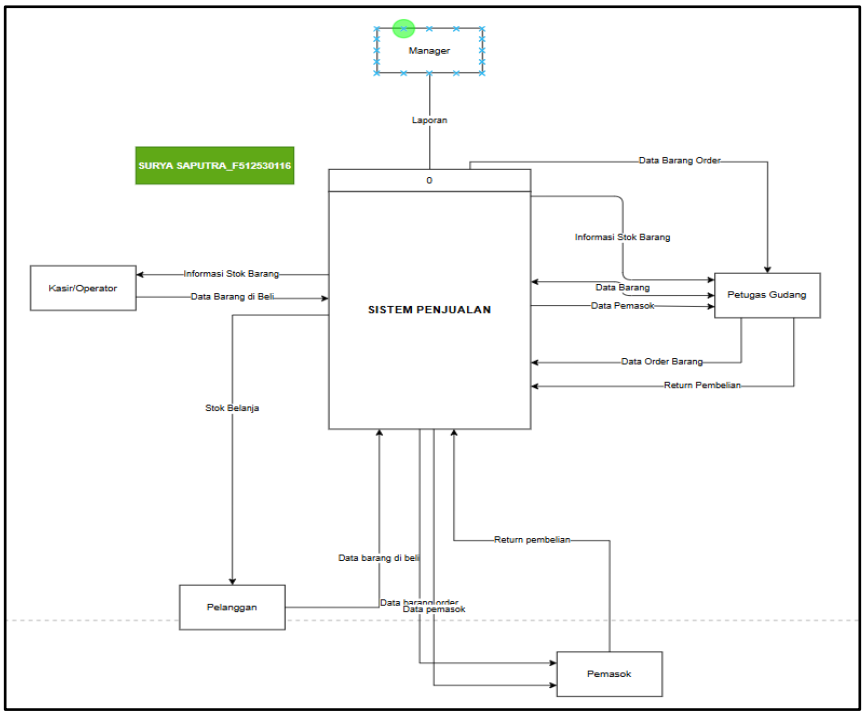


Diagram Konteks ini menggambarkan hubungan antara Sistem Penjualan dengan beberapa entitas yang terlibat dalam proses penjualan dan pengelolaan barang, yaitu Kasir/Operator, Pelanggan, Pemasok, Petugas Gudang, dan Manajer. Sistem menerima data barang yang dibeli dari Kasir/Operator serta memberikan informasi stok barang untuk membantu proses transaksi. Pelanggan menerima informasi dan barang yang dibeli serta berhubungan dengan sistem dalam proses pembelian. Petugas Gudang bertugas mengelola data barang, menerima data barang dan informasi stok, serta menangani data order dan retur pembelian. Pemasok memberikan data pemasok dan berhubungan dengan sistem dalam proses pemenuhan serta retur pembelian barang. Sementara itu, Manajer menerima laporan dari sistem sebagai bahan untuk memantau kegiatan penjualan dan persediaan. Secara keseluruhan, diagram ini menunjukkan aliran data antara setiap pihak dengan Sistem Penjualan agar proses

transaksi, pengelolaan stok, pemesanan barang, dan pembuatan laporan dapat berjalan secara terstruktur dan terkoordinasi.

7. Pembuatan *Class Diagram* di Draw.io

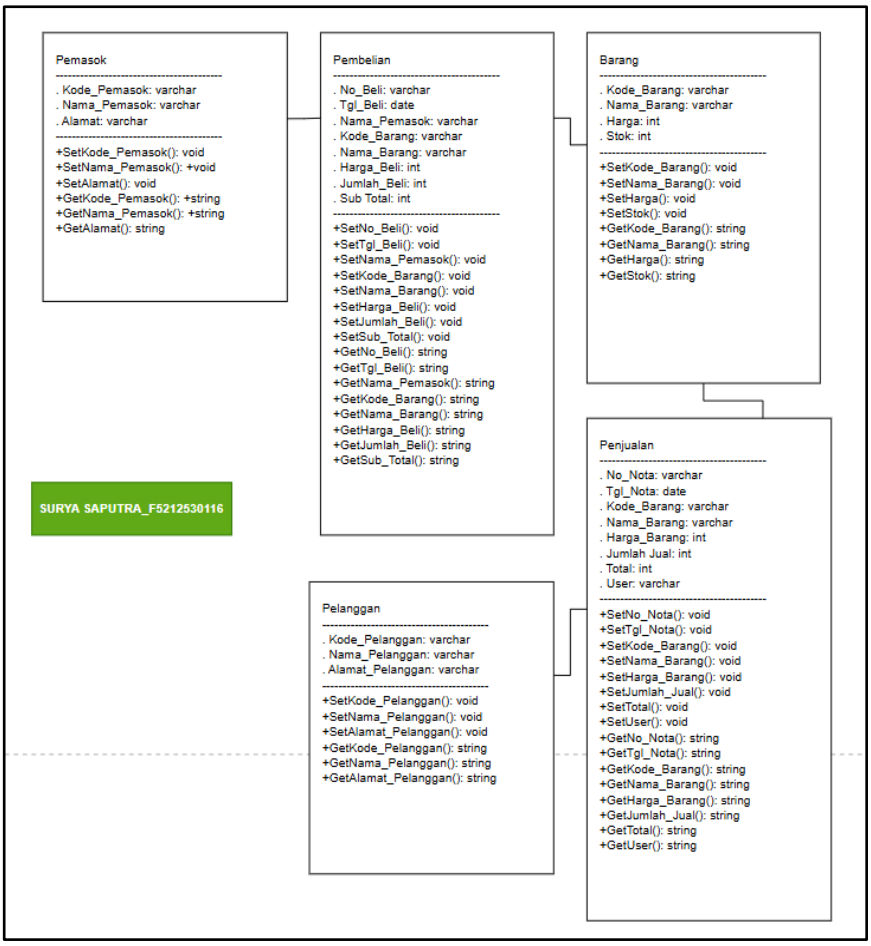
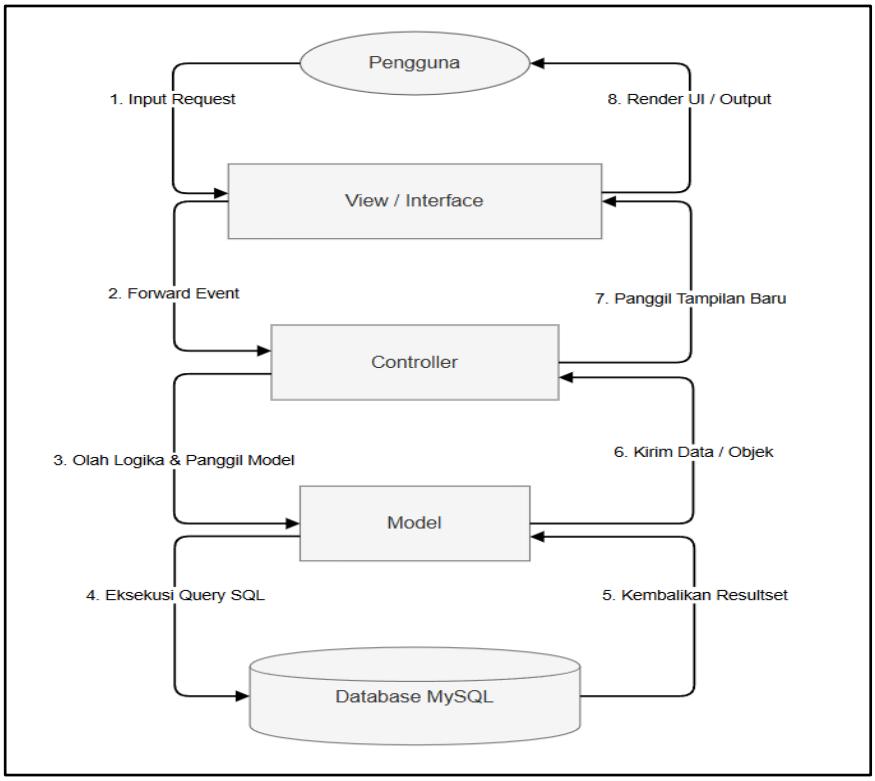


Diagram *Class* ini menggambarkan struktur data dan hubungan antar-*class* yang digunakan dalam Sistem Penjualan Barang. Terdapat lima *class* utama, yaitu Pemasok, Pembelian, Barang, Pelanggan, dan Penjualan. *Class* Pemasok menyimpan informasi mengenai kode, nama, dan alamat pemasok yang dilengkapi fungsi untuk mengatur dan mengambil data tersebut. *Class* Pembelian digunakan untuk mencatat proses pembelian barang dari pemasok, seperti nomor beli, tanggal, nama pemasok, kode dan nama barang, harga, jumlah, serta subtotal. *Class* Barang berfungsi menyimpan data barang berupa kode, nama, harga, dan stok yang dapat digunakan untuk mengelola persediaan. Selanjutnya, *class*

Pelanggan menyimpan informasi pelanggan seperti kode, nama, dan Alamat.

4.1.2 Perancangan Arsitektur dan Implementasi Perangkat Lunak

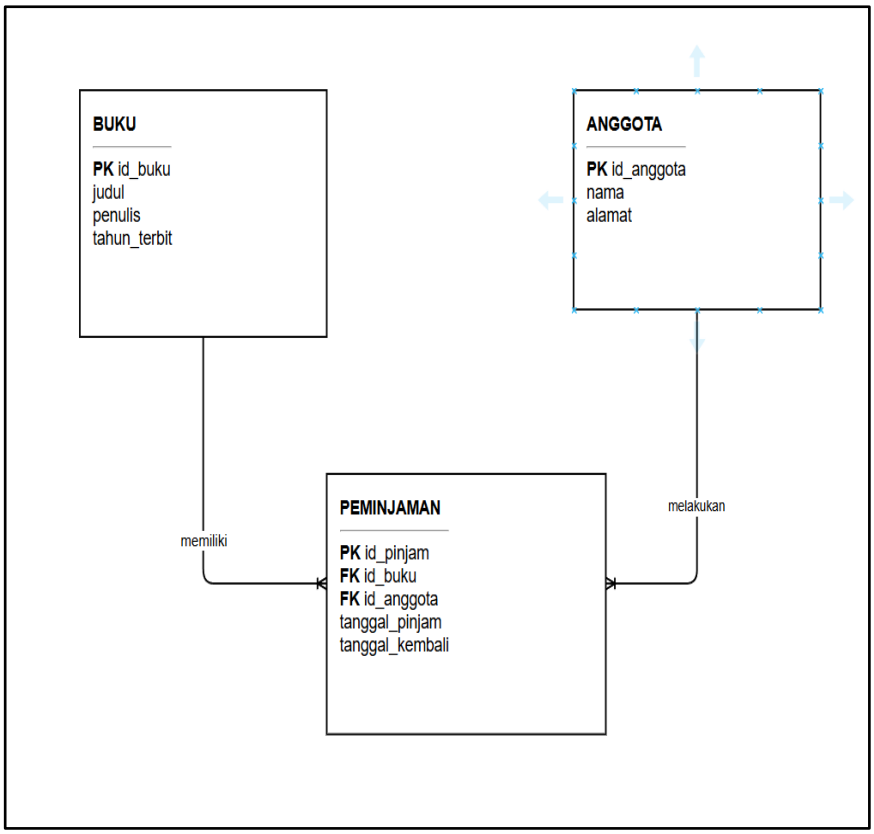
1. Hasil dari Perancangan Diagram Arsitektur MVC di Draw.io



Gambar tersebut merupakan diagram arsitektur *MVC (Model View Controller)* yang menjelaskan alur kerja antara pengguna, *View / Interface*, *Controller*, *Model*, dan *Database Mysql* . Pengguna memberikan permintaan melalui *View*, kemudian *View* meneruskan permintaan tersebut kepada *Controller*. Selanjutnya, *Controller* mengolah logika sistem dan memanggil *Model* untuk memproses data. *Model* kemudian menjalankan *query SQL* ke *Database Mysql* untuk mengambil atau mengolah data. Setelah proses selesai, *Database* mengembalikan *resultset* kepada *Model*, kemudian *Model* mengirimkan data atau objek kepada *Controller*. Setelah itu, *Controller* memanggil tampilan yang sesuai melalui *View*, dan *View* melakukan *render UI / output* untuk menampilkan hasil akhir kepada pengguna. Jadi, alur keseluruhan pada diagram tersebut adalah

pengguna → *View* → *Controller* → *Model* → *Database*, kemudian hasilnya kembali melalui *Model* → *Controller* → *View* → pengguna.

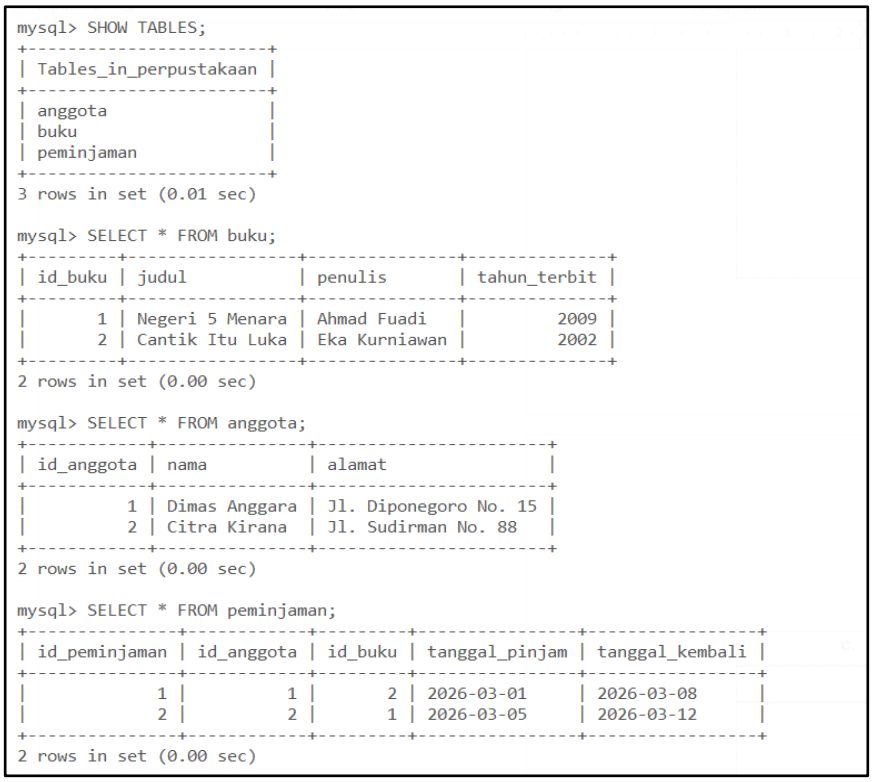
2. Hasil dari Perancangan *Entity Relationship Diagram* (ERD)



Gambar tersebut merupakan ERD (*Entity Relationship Diagram*) untuk sistem perpustakaan yang terdiri dari tiga entitas, yaitu *Buku*, *Anggota*, dan *Peminjaman*. Entitas *Buku* memiliki *primary key* id_buku serta atribut judul, penulis, dan tahun_terbit, sedangkan entitas *Anggota* memiliki *primary key* id_anggota dengan atribut nama dan alamat. Entitas *Peminjaman* memiliki *primary key* id_pinjam serta *foreign key* id_buku dan id_anggota, kemudian memiliki atribut tanggal_pinjam dan tanggal_kembali. Hubungan antara *Buku* dan *Peminjaman* adalah satu ke banyak (1:N), yang berarti satu buku dapat memiliki beberapa data peminjaman, sedangkan hubungan antara *Anggota* dan *Peminjaman* juga satu ke banyak (1:N), yang berarti satu anggota dapat melakukan beberapa peminjaman. Dengan demikian, entitas *Peminjaman* berfungsi

sebagai penghubung antara *Buku* dan *Anggota* melalui *foreign key* id_buku dan id_anggota.

3. Hasil dari Mengimplementasikan *Database* (SQL DDL)



```
mysql> SHOW TABLES;
+------------------------+
| Tables_in_perpustakaan |
+------------------------+
| anggota                |
| buku                   |
| peminjaman             |
+------------------------+
3 rows in set (0.01 sec)

mysql> SELECT * FROM buku;
+---------+-----------------+---------------+--------------+
| id_buku | judul           | penulis       | tahun_terbit |
+---------+-----------------+---------------+--------------+
|       1 | Negeri 5 Menara | Ahmad Fuadi   |         2009 |
|       2 | Cantik Itu Luka | Eka Kurniawan |         2002 |
+---------+-----------------+---------------+--------------+
2 rows in set (0.00 sec)

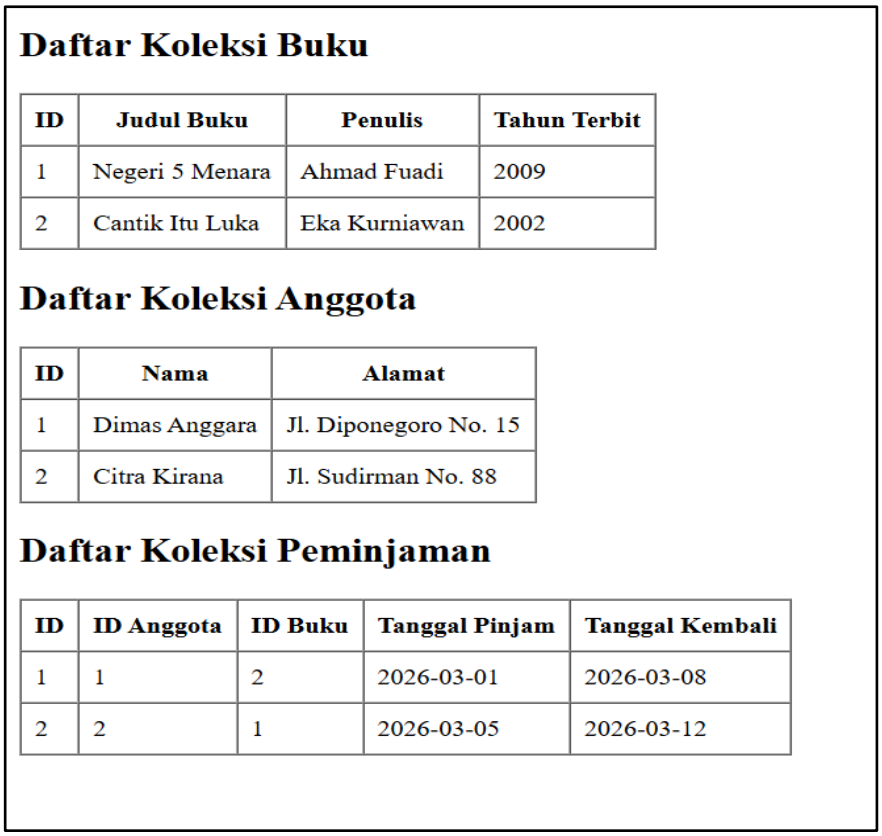
mysql> SELECT * FROM anggota;
+------------+---------------+-----------------------+
| id_anggota | nama          | alamat                |
+------------+---------------+-----------------------+
|          1 | Dimas Anggara | Jl. Diponegoro No. 15 |
|          2 | Citra Kirana  | Jl. Sudirman No. 88   |
+------------+---------------+-----------------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM peminjaman;
+---------------+------------+---------+----------------+-----------------+
| id_peminjaman | id_anggota | id_buku | tanggal_pinjam | tanggal_kembali |
+---------------+------------+---------+----------------+-----------------+
|             1 |          1 |       2 | 2026-03-01     | 2026-03-08      |
|             2 |          2 |       1 | 2026-03-05     | 2026-03-12      |
+---------------+------------+---------+----------------+-----------------+
2 rows in set (0.00 sec)
```

Gambar tersebut menunjukkan hasil penerapan basis data sistem perpustakaan menggunakan *Mysql* . Pada bagian awal, perintah SHOW TABLES; digunakan untuk melihat tabel yang tersedia dalam basis data *perpustakaan*, yaitu tabel *anggota*, *buku*, dan *peminjaman*. Selanjutnya, perintah SELECT * FROM buku; menampilkan data buku yang terdiri dari id_buku, judul, penulis, dan tahun_terbit, dengan dua data buku yang tersimpan. Perintah SELECT * FROM anggota; menampilkan data anggota berupa id_anggota, nama, dan alamat, yang juga memiliki dua data anggota. Terakhir, perintah SELECT * FROM peminjaman; menampilkan data transaksi peminjaman yang terdiri dari id_peminjaman, id_anggota, id_buku, tanggal_pinjam, dan tanggal_kembali. Dari hasil tersebut dapat dilihat bahwa tabel *peminjaman* menghubungkan data anggota dan buku melalui id_anggota dan

id_buku, sehingga struktur basis data yang dibuat sudah sesuai dengan ERD sebelumnya.

4. Hasil dari Membuat Struktur Proyek MVC



Daftar Koleksi Buku

ID	Judul Buku	Penulis	Tahun Terbit
1	Negeri 5 Menara	Ahmad Fuadi	2009
2	Cantik Itu Luka	Eka Kurniawan	2002

Daftar Koleksi Anggota

ID	Nama	Alamat
1	Dimas Anggara	Jl. Diponegoro No. 15
2	Citra Kirana	Jl. Sudirman No. 88

Daftar Koleksi Peminjaman

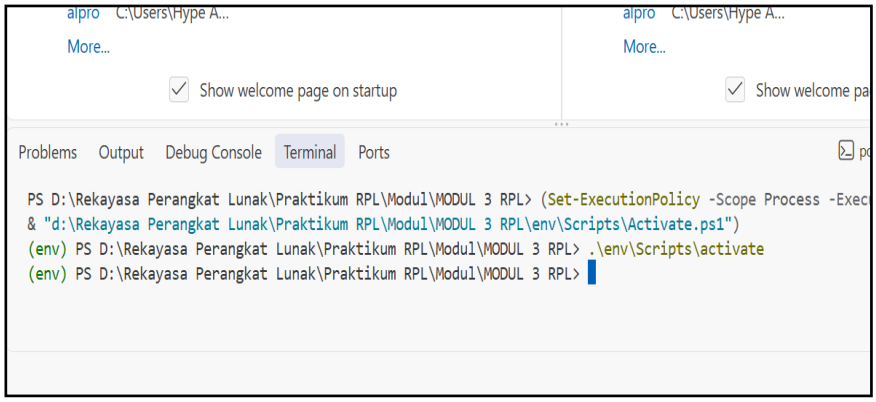
ID	ID Anggota	ID Buku	Tanggal Pinjam	Tanggal Kembali
1	1	2	2026-03-01	2026-03-08
2	2	1	2026-03-05	2026-03-12

Gambar tersebut menunjukkan tampilan halaman sistem perpustakaan yang menampilkan tiga bagian utama, yaitu *Daftar Koleksi Buku*, *Daftar Koleksi Anggota*, dan *Daftar Koleksi Peminjaman*. Pada bagian *Daftar Koleksi Buku* ditampilkan data id, judul buku, penulis, dan tahun terbit yang berisi dua buku. Bagian *Daftar Koleksi Anggota* menampilkan id, nama, dan alamat dari dua anggota perpustakaan. Sementara itu, bagian *Daftar Koleksi Peminjaman* menampilkan id, id anggota, id buku, tanggal pinjam, dan tanggal kembali. Data tersebut menunjukkan hubungan antara anggota dan buku melalui proses peminjaman, misalnya anggota dengan ID 1 meminjam buku dengan ID 2 pada tanggal 2026-03-01 dan mengembalikannya pada 2026-03-08. Tampilan ini digunakan untuk menyajikan data dari basis data dalam bentuk

tabel sehingga informasi buku, anggota, dan peminjaman dapat dilihat dengan lebih mudah.

4.1.3 *Setup Environment, Code Editor,* Dan *Version Control System* (VCS)

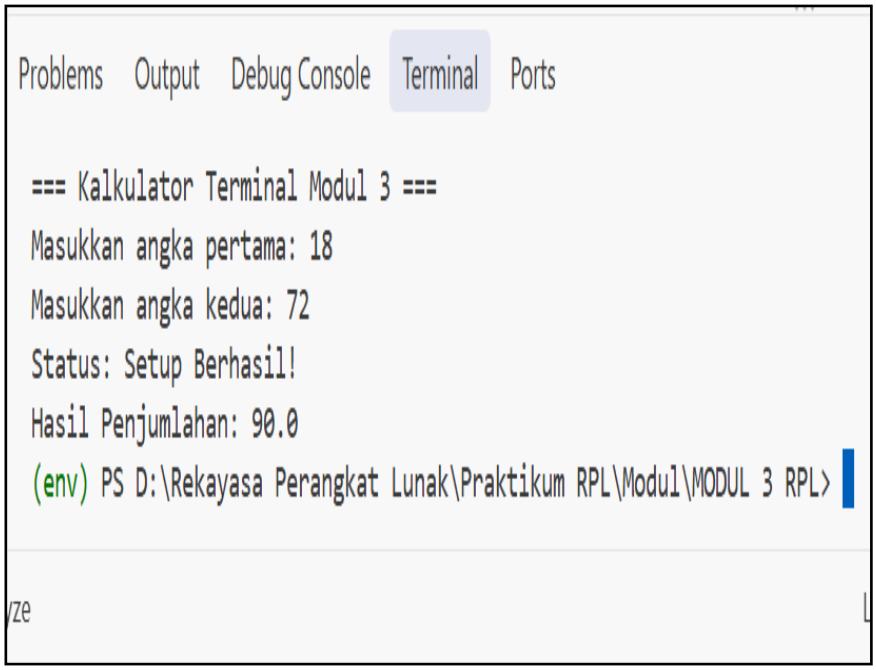
1. Tampilan env telah aktif



Pada gambar tersebut terlihat bahwa program *Kalkulator Terminal Modul 3* telah berhasil dijalankan melalui *terminal* pada VS Code. Program pertama-tama menampilkan judul === Kalkulator Terminal Modul 3 === sebagai penanda bahwa aplikasi yang dijalankan merupakan program kalkulator untuk praktikum Modul 3. Selanjutnya, program meminta pengguna memasukkan dua nilai, yaitu angka pertama 18 dan angka kedua 72. Kedua nilai tersebut berhasil diterima oleh program tanpa adanya *error*. Setelah input diberikan, muncul keterangan *Status: Setup Berhasil!* yang menunjukkan bahwa proses persiapan dan konfigurasi program telah berjalan dengan baik. Program kemudian melakukan proses perhitungan dengan menjumlahkan kedua angka tersebut, yaitu 18 + 72, sehingga menghasilkan nilai 90.0. Hasil tersebut menunjukkan bahwa fungsi penjumlahan pada program dapat bekerja sesuai dengan input yang diberikan. Angka ditampilkan dalam bentuk desimal karena hasil perhitungan diproses sebagai tipe data *float*. Pada bagian bawah juga masih terlihat (env), yang menunjukkan bahwa *virtual environment* Python tetap aktif selama program dijalankan. Secara keseluruhan, tampilan tersebut memperlihatkan

bahwa program berhasil dieksekusi, menerima input pengguna, melakukan proses perhitungan, dan menampilkan hasil penjumlahan dengan benar.

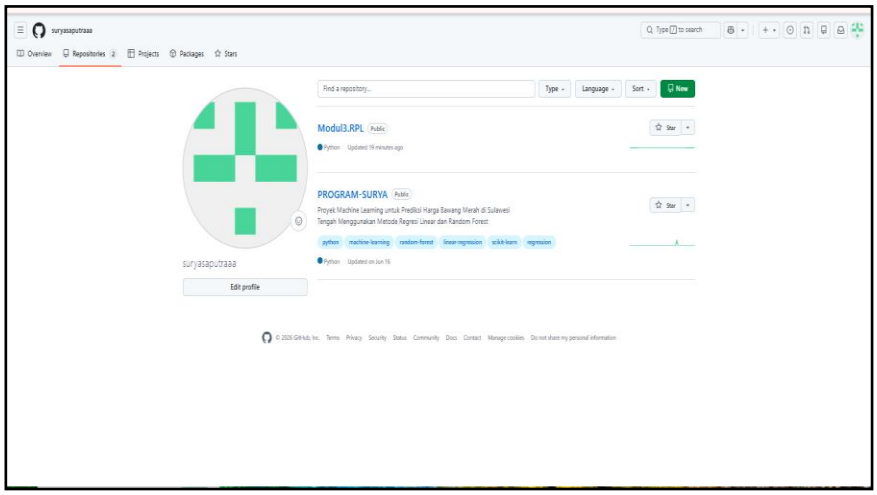
2. Tampilan terminal hasil output



Pada gambar tersebut terlihat bahwa program *Kalkulator Terminal Modul 3* berhasil dijalankan melalui terminal VS Code. Program diawali dengan menampilkan identitas aplikasi melalui tulisan === Kalkulator Terminal Modul 3 ===, sehingga dapat diketahui bahwa program yang dijalankan merupakan bagian dari praktikum Modul 3. Setelah itu, program meminta pengguna memasukkan dua nilai sebagai *input*, yaitu angka pertama sebesar 18 dan angka kedua sebesar 72. Kedua nilai tersebut berhasil diterima oleh program dan kemudian diproses untuk melakukan operasi penjumlahan. Program juga menampilkan *Status: Setup Berhasil!* yang menunjukkan bahwa proses *setup* telah berjalan dengan baik dan tidak ditemukan kendala saat program dijalankan. Setelah proses tersebut selesai, program menghasilkan *Hasil Penjumlahan: 90.0*. Nilai tersebut sesuai dengan perhitungan 18 + 72 = 90, sehingga dapat diketahui bahwa fungsi penjumlahan pada program bekerja dengan benar.

Hasil ditampilkan sebagai 90.0 karena nilai keluaran menggunakan format *float* atau bilangan desimal. Selain itu, pada bagian bawah terminal masih terdapat tulisan (env), yang menandakan bahwa *virtual environment* Python yang sebelumnya dibuat masih dalam kondisi aktif. Hal ini menunjukkan bahwa program dijalankan menggunakan lingkungan Python khusus untuk praktikum, sehingga konfigurasi dan *package* yang digunakan dapat terpisah dari instalasi Python utama pada komputer. Secara keseluruhan, hasil pada gambar menunjukkan bahwa program telah berhasil dijalankan, mampu menerima *input* dari pengguna.

3. Tampilan repositori github



halaman dasbor profil akun platform *cloud hosting* GitHub yang memuat repositori publik dengan nama modul3_RPL. Tampilan ini menandai keberhasilan integrasi antara sistem pengontrol versi lokal (Git) dengan repositori jarak jauh (*remote repository*). Melalui serangkaian perintah Git seperti git init, git add, git commit, serta penambahan *remote origin*, seluruh riwayat perubahan struktur folder, berkas kode program, dan catatan komit (*commit history*) praktikum dari komputer lokal telah berhasil diunggah (*push*) ke peladen daring GitHub. Hal ini memastikan kode sumber (*source code*) tersimpan secara aman sebagai cadangan (*backup*).

4.1.4 Perancangan Model Python Dan Integrasi Basis Data *Mysql*

1. Tampilan output daftar buku di Python.

```
=== Daftar Buku ===
[1] Bumi Manusia - Pramoedya Ananta Toer (1980)
[2] Negeri 5 Menara - Ahmad Fuadi (2009)
[3] Laskar Pelangi - Andrea Hirata (2005)
[4] Filosofi Teras - Henry Manampiring (2018)
[5] Pemrograman Python MVC - Guido van Rossum (2023)
```

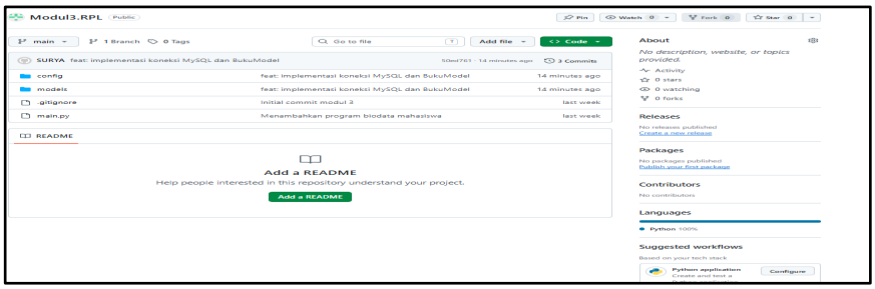
Output pada gambar tersebut adalah hasil program *Python* yang sukses menampilkan daftar data buku dari database *Mysql* dengan format yang rapi. Tampilan tersebut membuktikan bahwa koneksi program ke database dan fungsi pengambilan data telah berhasil berjalan dengan baik.

2. Tampilan output daftar buku di *Mysql*

```
+----------+--------------------------+--------------------------+--------------+
| id_buku | judul_buku                | penulis                  | tahun_terbit |
+----------+--------------------------+--------------------------+--------------+
|        1 | Bumi Manusia             | Pramoedya Ananta Toer    |         1980 |
|        2 | Negeri 5 Menara          | Ahmad Fuadi              |         2009 |
|        3 | Laskar Pelangi           | Andrea Hirata            |         2005 |
|        4 | Filosofi Teras           | Henry Manampiring        |         2018 |
|        5 | Pemrograman Python MVC   | Guido van Rossum         |         2023 |
+----------+--------------------------+--------------------------+--------------+
5 rows in set (0.00 sec)
```

Output pada gambar tersebut adalah hasil terminal *Mysql* yang menampilkan data tabel buku secara langsung dengan format tabel bergaris. Tampilan tersebut membuktikan bahwa perintah penarikan data berhasil dijalankan dengan sangat baik dan menampilkan lima baris rekaman informasi secara utuh.

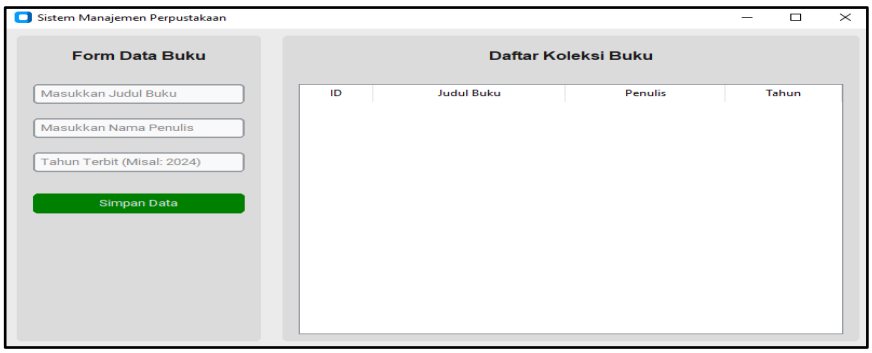
3. Tampilan repositori setelah commit terbaru



Tampilan pada gambar tersebut adalah halaman repositori *GitHub* untuk proyek perangkat lunak bernama *Modul3.RPL*. Halaman tersebut menunjukkan struktur direktori yang mencakup folder *config*, folder *models*, berkas *.gitignore*, serta berkas *main.py* yang telah berhasil diunggah dengan pesan commit terkait implementasi koneksi *Mysql* dan *BukuModel*.

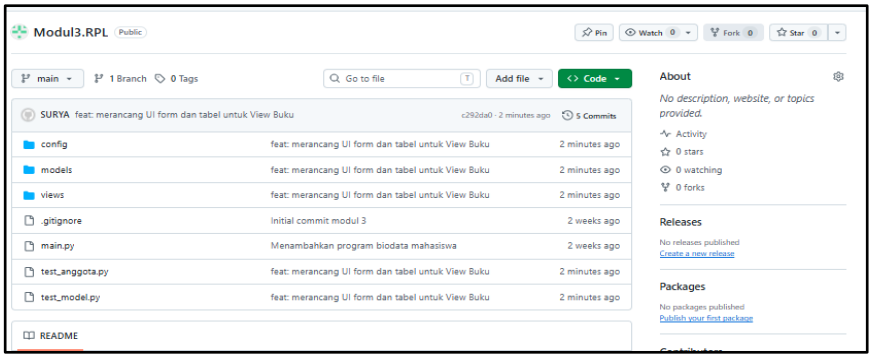
4.1.5 Pengembangan Antarmuka Pengguna (*View*) Berbasis *Desktop*

1. Tampilan antarmuka aplikasi



Antarmuka aplikasi Sistem Manajemen Perpustakaan dibuat sederhana dan mudah digunakan. Bagian kiri berisi *Form* Data Buku untuk memasukkan judul, penulis, dan tahun terbit, serta tombol Simpan Data. Bagian kanan menampilkan Daftar Koleksi Buku dalam bentuk tabel berisi *ID*, Judul Buku, Penulis, dan Tahun. Pembagian ini memudahkan pengguna dalam memasukkan dan melihat data buku.

2. Tampilan *repositori github*



Tampilan tersebut merupakan repositori GitHub *Modul3.RPL* yang digunakan untuk menyimpan dan mengelola source code aplikasi. Di dalamnya terdapat folder *config*, *models*, dan *views*, serta *file* seperti *main.py*, *test_anggota.py*, dan *test_model.py*. Terdapat juga *.gitignore* dan *README* sebagai pendukung proyek. Riwayat commit menunjukkan pengembangan *UI form* dan tabel buku telah berhasil disimpan. *Repository* menggunakan *branch main* dan bersifat publik.

BAB V
PENUTUP

5.1 Kesimpulan

5.1.1 Konsep Dasar RPL, Pemodelan Aliran Data (DFD), *Unified Modeling Language* (UML) Menggunakan *Platform Draw.io*

Rekayasa Perangkat Lunak (RPL) merupakan proses yang digunakan untuk merancang dan mengembangkan perangkat lunak secara sistematis agar menghasilkan sistem yang sesuai dengan kebutuhan pengguna. Dalam perancangan sistem, pemodelan sangat diperlukan untuk menggambarkan proses, aliran data, struktur, serta hubungan antarbagian dalam sistem. *Data Flow Diagram* (DFD) digunakan untuk menggambarkan aliran data mulai dari Diagram Konteks hingga DFD Level 1, sedangkan *Unified Modeling Language* (UML) digunakan untuk menggambarkan sistem dari sisi pengguna maupun struktur sistem melalui *Use Case Diagram, Sequence Diagram, Activity Diagram,* dan *Class Diagram*. Penggunaan *Platform Draw.io* membantu proses pembuatan berbagai diagram tersebut menjadi lebih mudah, terstruktur, dan mudah dipahami. Dengan adanya pemodelan DFD dan UML, rancangan Sistem Penjualan Barang dapat digambarkan dengan lebih jelas sehingga dapat menjadi dasar dalam proses pengembangan sistem yang terarah dan sesuai dengan kebutuhan.

5.1.2 *Perancangan* Arsitektur dan Implementasi Perangkat Lunak

praktikum mempelajari perancangan sistem menggunakan arsitektur *MVC*, pembuatan ERD, serta implementasi basis data menggunakan *Mysql* . Dalam praktikum juga diterapkan *primary key*, *foreign key*, relasi antartabel, dan operasi *CRUD* untuk mengelola data perpustakaan. Dengan kegiatan tersebut, dapat dipahami bagaimana *View*, *Controller*, dan *Model* saling terhubung dengan basis data sehingga sistem dapat berjalan secara terstruktur dan terorganisir. Selain itu, praktikum memberikan pemahaman tentang pengelolaan data, hubungan antarbagian sistem, serta cara membangun aplikasi perpustakaan sederhana yang dapat menampilkan dan mengelola data buku, anggota, dan peminjaman.

5.1.3 *Setup Environment, Code Editor,* Dan *Version Control System* (VCS)

Kesimpulan dari praktikum Modul III adalah bahwa proses *setup environment*, penggunaan *code editor*, pembuatan *virtual environment*, serta penerapan *Version Control System* menggunakan Git dan GitHub telah berhasil dilakukan. *Virtual environment* digunakan untuk membuat lingkungan Python yang terpisah sehingga *dependency* proyek dapat dikelola dengan lebih aman, sedangkan VS Code digunakan sebagai *code editor* sekaligus *integrated terminal* untuk menjalankan program. Selanjutnya, Git digunakan untuk mengelola perubahan pada berkas melalui proses git init, git add, dan git commit, kemudian repositori lokal dihubungkan dengan GitHub sebagai *remote repository* melalui proses *push*. Hasil praktikum menunjukkan bahwa program Python sederhana dapat dijalankan melalui terminal dan berkas proyek berhasil diunggah ke repositori GitHub. Dengan demikian, praktikum ini memberikan pemahaman dasar mengenai persiapan lingkungan pengembangan dan pengelolaan kode menggunakan Git serta GitHub sesuai tujuan praktikum.

5.1.4 Perancangan Model Python Dan Integrasi Basis Data *Mysql*

Praktikum ini berhasil membuktikan bahwa lingkungan kerja *Python* dapat diintegrasikan secara optimal dengan layanan basis data *Mysql* pada *Laragon* menggunakan pola arsitektur *Model-View-Controller*. Komponen *Model* terbukti efektif dalam mengenkapsulasi logika bisnis dan penanganan operasi *CRUD* melalui pemanfaatan *Object-Oriented Programming* serta pustaka *Mysql -connector-python*. Selain itu, penerapan *Prepared Statement* dan *Parameterized Query* memastikan keamanan sistem dari kerentanan injeksi SQL, sementara pengelolaan transaksi secara tepat menjaga integritas data relasional aplikasi. Seluruh rangkaian pengembangan perangkat lunak ini juga telah berhasil disinkronkan dan diunggah ke dalam repositori *GitHub* secara terstruktur sehingga proses pengembangan dan pengujian dapat dilakukan secara lebih terorganisir dan terdokumentasi dengan baik, serta mudah dipantau oleh pengguna dan pengembang.

5.1.5 Pengembangan Antarmuka Pengguna (*View*) Berbasis *Desktop*

Praktikum ini berhasil membuktikan bahwa komponen *View* dalam arsitektur *Model-View-Controller (MVC)* berperan penting dalam menyediakan antarmuka pengguna yang sederhana dan mudah digunakan. Pengembangan GUI menggunakan *Python* dan pustaka *customtkinter* berhasil menerapkan *layouting*, *input form*, *button*, dan tabel untuk menampilkan data secara visual. Pemisahan *View* dari logika bisnis dan basis data juga membantu menjaga struktur aplikasi agar lebih terorganisir, sehingga proses pengembangan antarmuka dapat dilakukan secara terpisah dari *Controller* dan *Model*.